
大模型时代下的存储系统



崔立骁



目录

- 大模型训练中的存储
- 大模型推理中的存储
- 大模型指导存储系统设计





目录

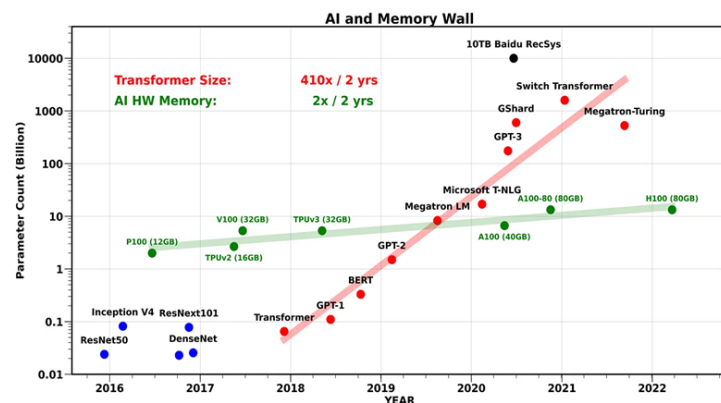
- 大模型训练中的存储
- 大模型推理中的存储
- 大模型指导存储系统设计



模型训练为什么需要存储



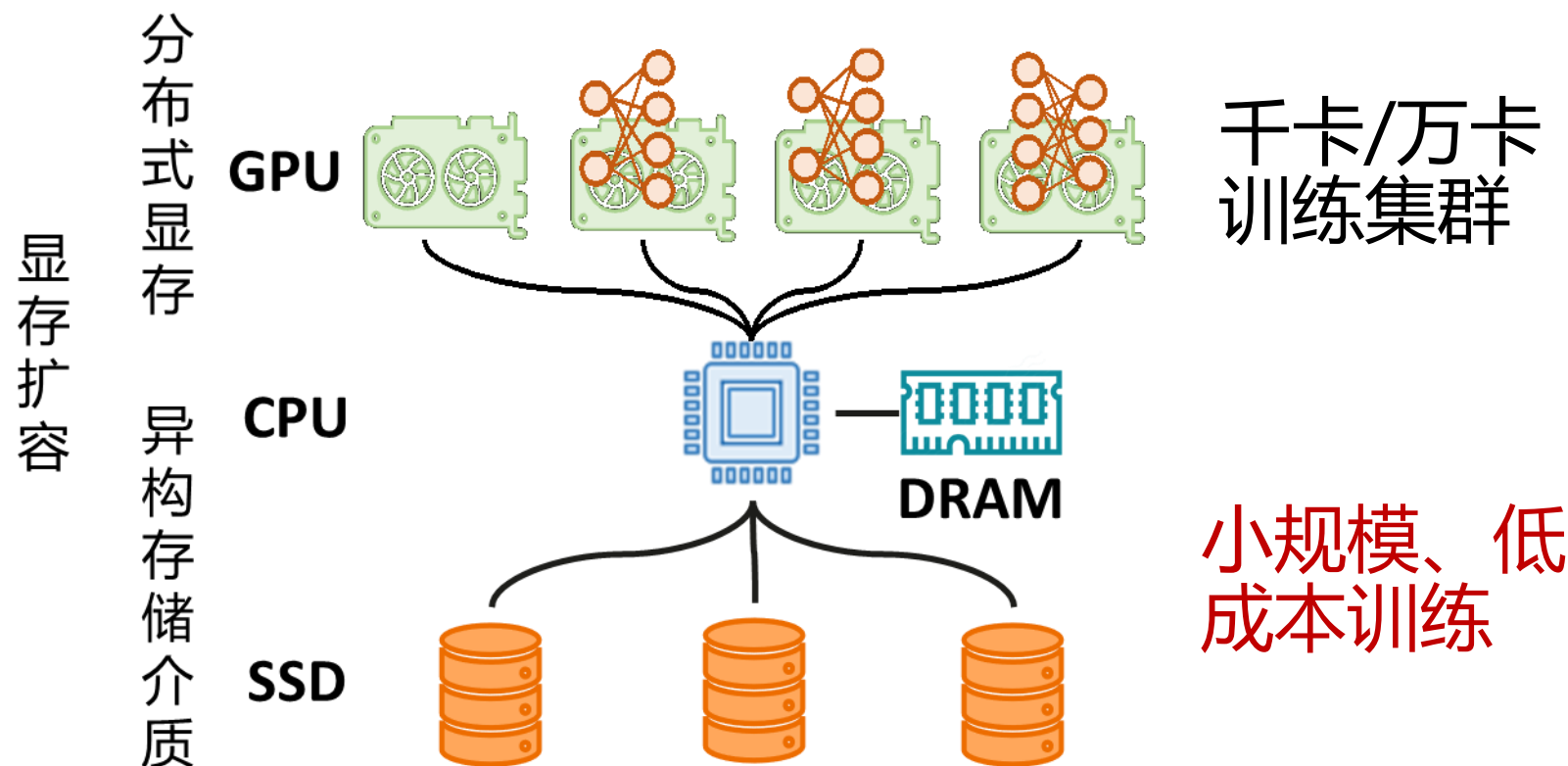
- 扩展容量需求
- GPU的存储容量难以满足大模型训练的存储需求
- GPU存储容量小，且增长缓慢
- GPU的存算资源强耦合，存算资源只能等比扩展，这导致GPU算力的浪费



模型规模每两年增长410倍；
而硬件存储容量每两年仅增长2倍

舒继武教授, AI大模型场景下的存储系统技术演进趋势

模型训练为什么需要存储

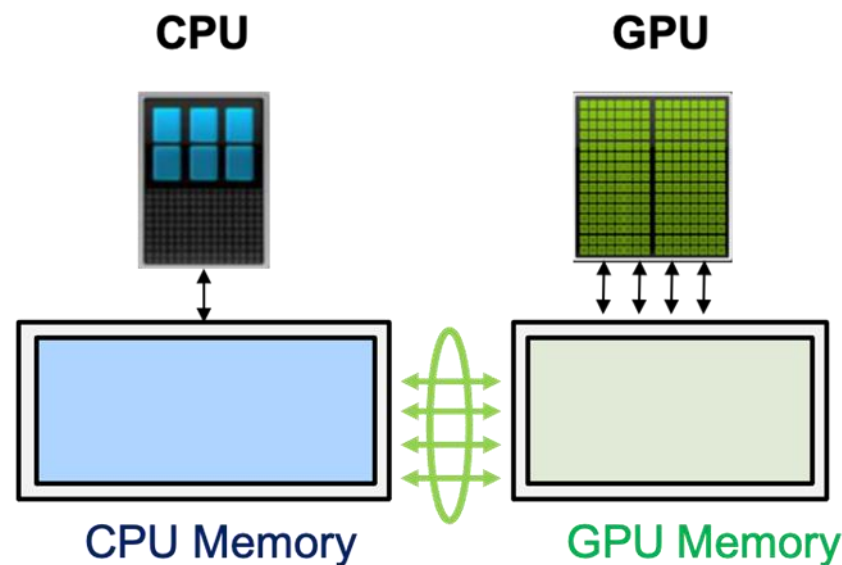


舒继武教授, AI大模型场景下的存储系统技术演进趋势

ZERO-offload

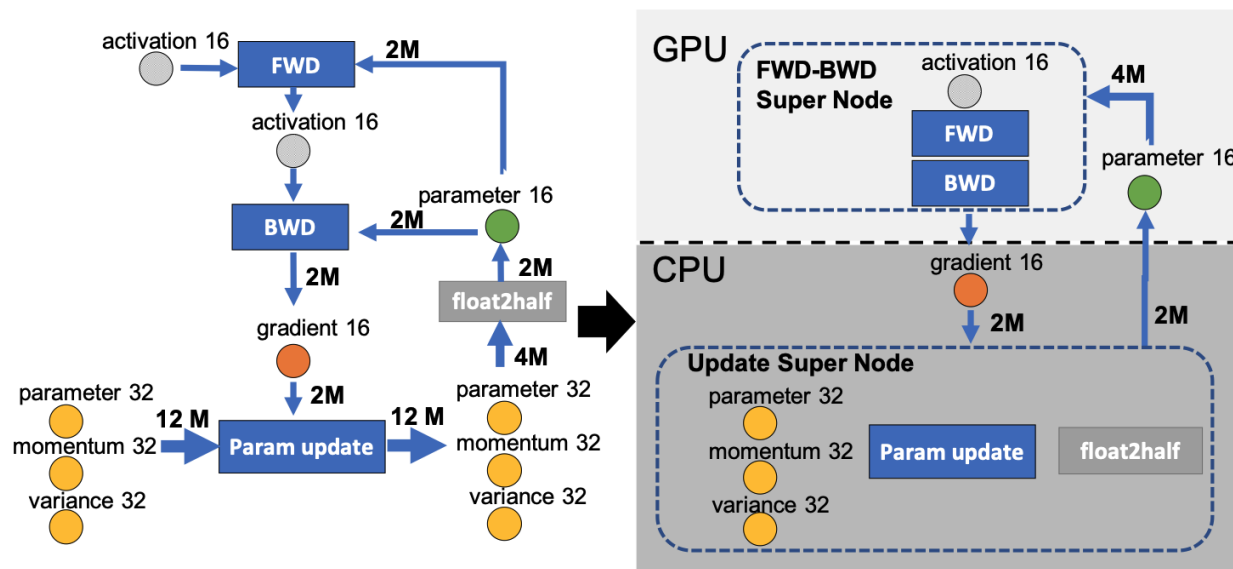


- 使用廉价的CPU内存扩展显存容量
- 在计算开始前，将张量从CPU内存预取到GPU内存。



“ZeRO-Offload: Democratizing Billion-Scale Model Training”, ATC 21

ZERO-offload



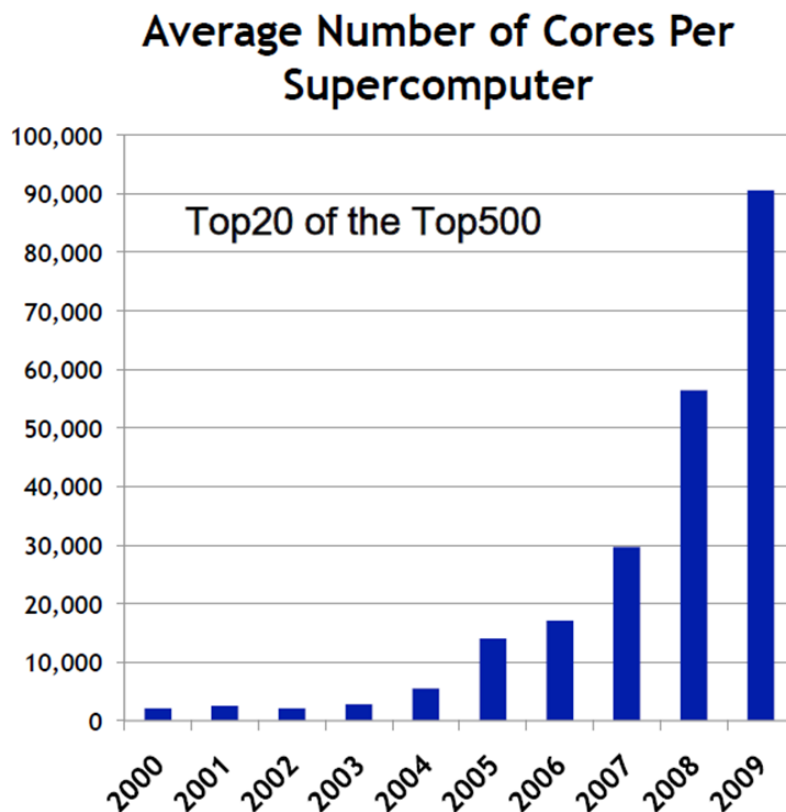
- 将优化器参数存储至DRAM中；
- 训练前将需要计算的参数以**半精度**传输至GPU中，计算后将梯度传回DRAM并使用全精度优化器进行参数更新；
- 数据传输与模型计算并行进行以掩盖数据传输的开销

“ZeRO-Offload: Democratizing Billion-Scale Model Training”, ATC 21

模型训练为什么需要存储



- 容错需求
- 系统硬件/软件
伸缩性、
应用伸缩性
- 障碍
 - 软件规模增大了
1000倍
 - 软件可靠性只提高
了100倍

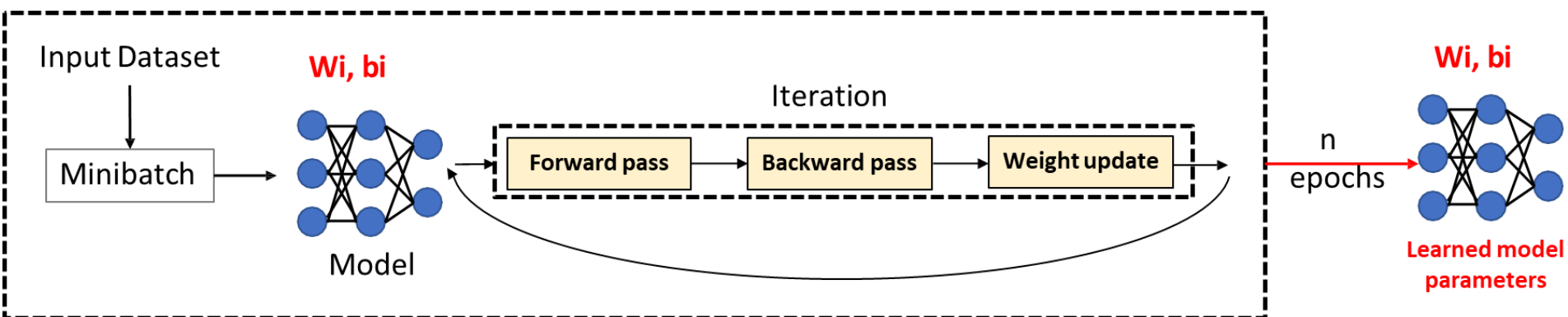


Jack Dongarra, Overview of High Performance Computing, SC13, UTK
Booth talk, November 19, 2013, Denver, CO..

模型训练为什么需要存储



- 容错需求

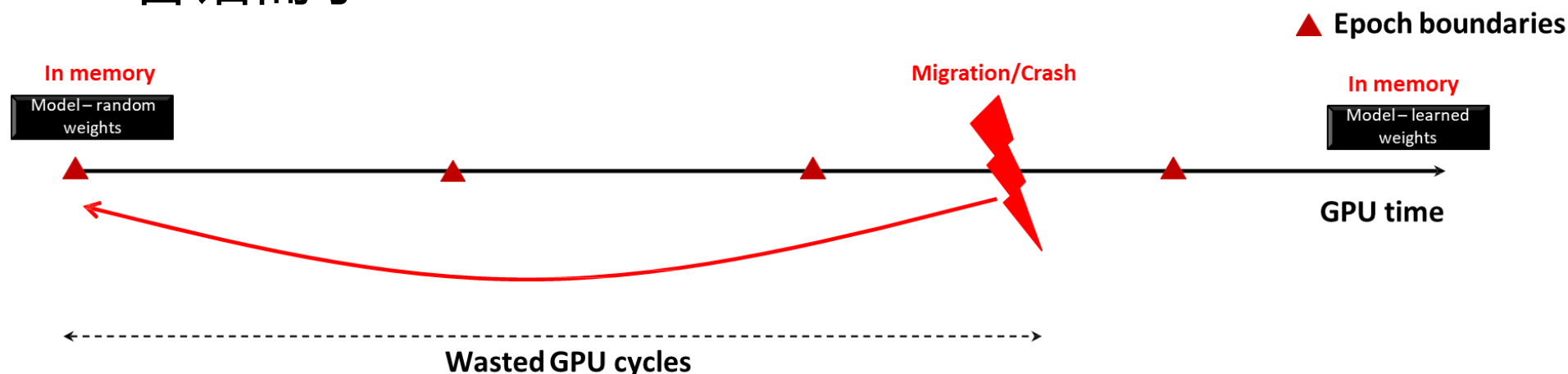


- 大模型训练耗时数天甚至数周
- 任何中断都可能导致之前的工作全部丢失
- 检查点(Checkpoint)机制是容错的关键，但时间需要尽量短，不阻塞GPU计算

模型训练为什么需要存储



- 容错需求

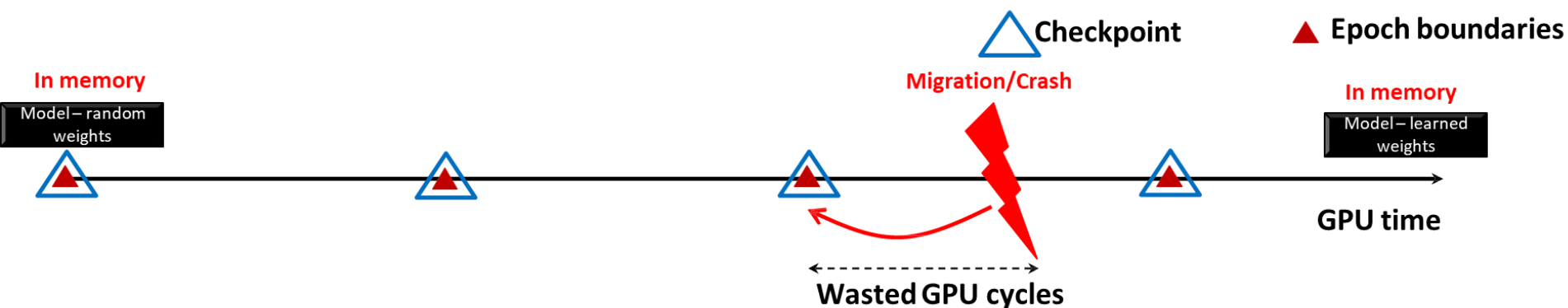


- 大模型训练耗时数天甚至数周
- 任何中断都可能导致之前的工作全部丢失
- 检查点(Checkpoint)机制是容错的关键，但时间需要尽量短，不阻塞GPU计算

模型训练为什么需要存储



- 容错需求



- 大模型训练耗时数天甚至数周
- 任何中断都可能导致之前的工作全部丢失
- 检查点(Checkpoint)机制是容错的关键，但时间需要尽量短，不阻塞GPU计算

CheckFreq: 细粒度检查点技术



- 原始的检查点机制
 - 在Epoch边界进行
 - 同步检查点引入显著的停滞 (stall) 开销
 - 随着模型规模增大, Epoch时间不断增加
 - 需要细粒度、迭代 (iteration-level) 级别的检查点机制

“CheckFreq Frequent, Fine-Grained DNN Checkpointing”, FAST 20.

CheckFreq: 细粒度检查点技术



- 细粒度检查点的挑战
- 检查点频率：多久进行一次检查点？
- 检查点停滞：如何最小化检查点开销？
- 数据不变性：如何在epoch中间正确恢复训练？

“CheckFreq Frequent, Fine-Grained DNN Checkpointing”, FAST 20.

CheckFreq: 细粒度检查点技术



- checkfreq
- 如何执行低成本检查点?
 - - 两阶段DNN感知检查点
- 何时触发检查点?
 - - 系统化在线性能分析，自适应频率调整

“CheckFreq Frequent, Fine-Grained DNN Checkpointing”, FAST 20.

CheckFreq: 细粒度检查点技术



- 两阶段检查点
- 传统同步检查点引入停滞开销
- 两阶段设计: `snapshot()` + `persist()`
- `Snapshot()`: 序列化模型状态
- `Persist()`: 异步写入持久存储

“CheckFreq Frequent, Fine-Grained DNN Checkpointing”, FAST 20.

CheckFreq: 细粒度检查点技术

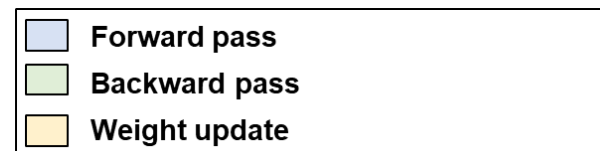


- 两阶段检查点
- 以每三次迭代进行一次检查点为例



Checkpoint (CPU)

(a) Baseline : Synchronous checkpointing



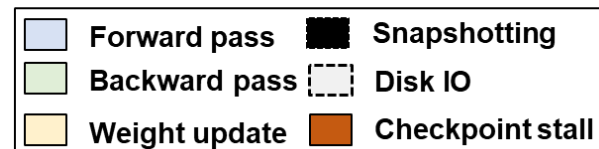
CheckFreq: 细粒度检查点技术



- 两阶段检查点
- 以每三次迭代进行一次检查点为例



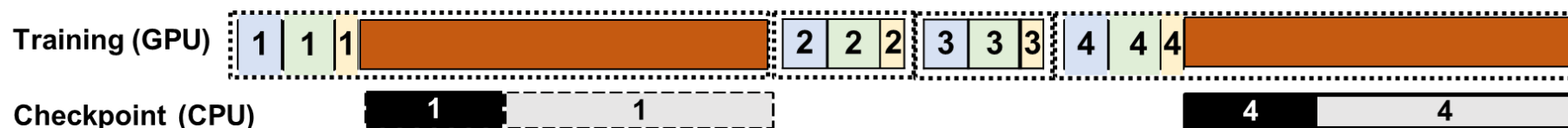
(a) Baseline : Synchronous checkpointing



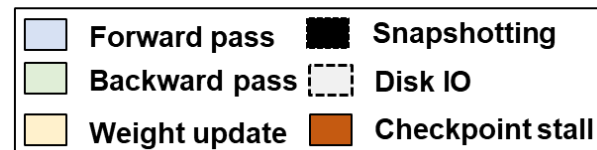
CheckFreq: 细粒度检查点技术



- 两阶段检查点
- 以每三次迭代进行一次检查点为例



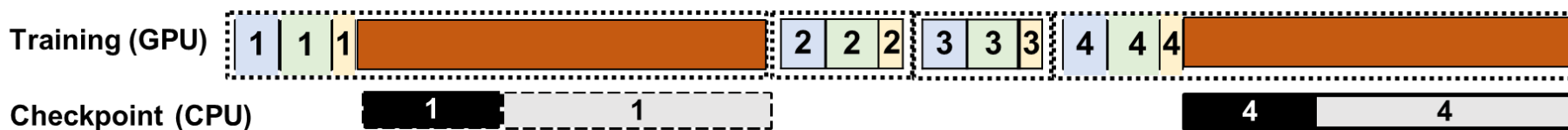
(a) Baseline : Synchronous checkpointing



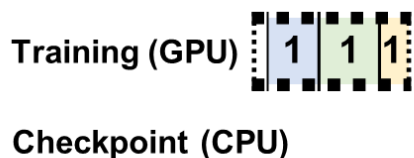
CheckFreq: 细粒度检查点技术



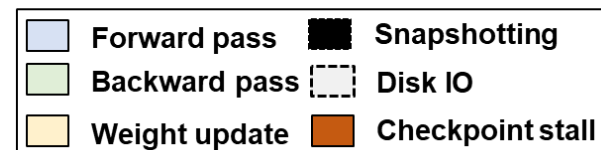
- 两阶段检查点
- 以每三次迭代进行一次检查点为例



(a) Baseline : Synchronous checkpointing



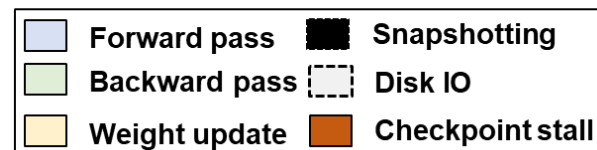
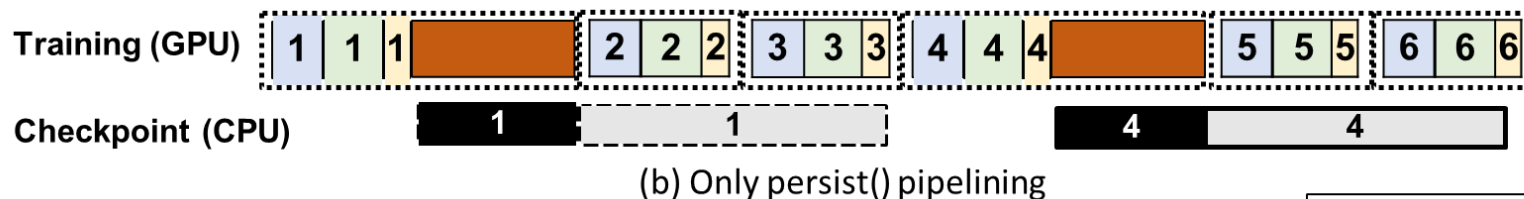
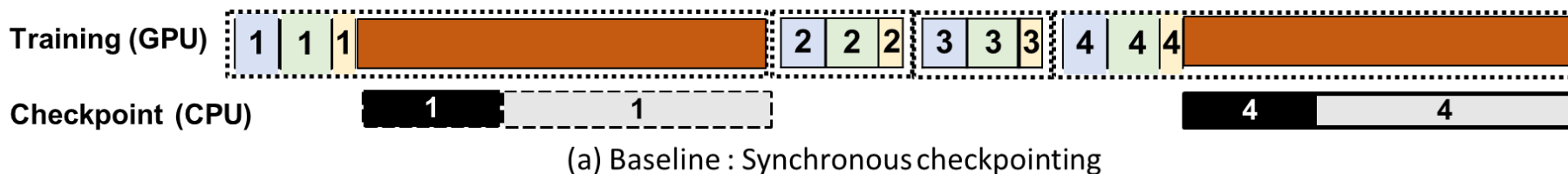
(b) Only persist() pipelining



CheckFreq: 细粒度检查点技术



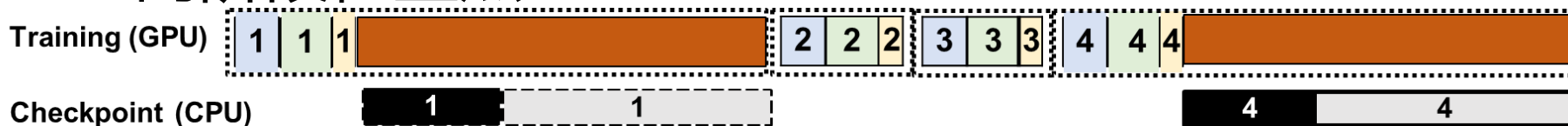
- 两阶段检查点
- 以每三次迭代进行一次检查点为例



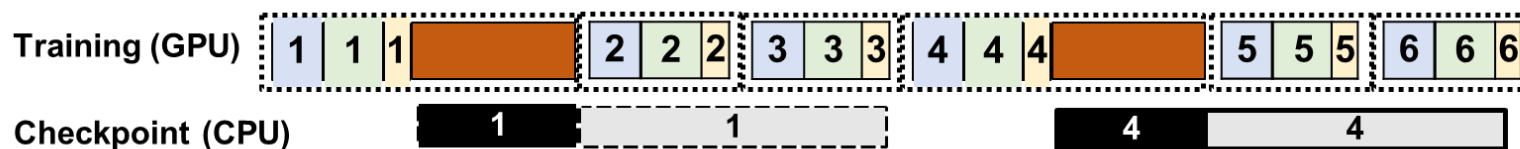
CheckFreq: 细粒度检查点技术



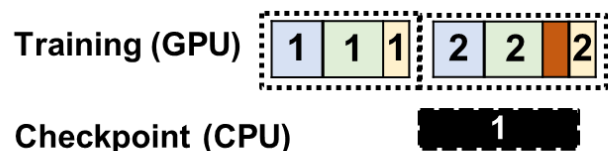
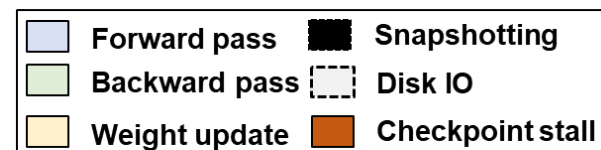
• 两阶段检查点



(a) Baseline : Synchronous checkpointing



(b) Only persist() pipelining

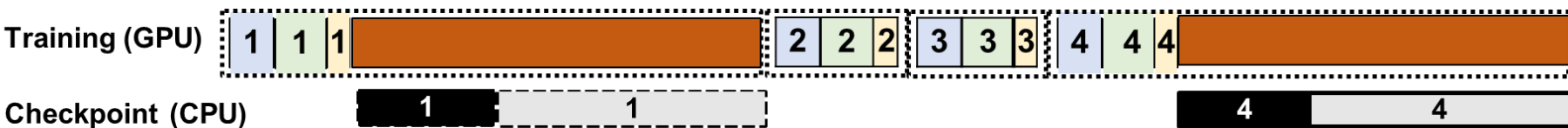


(c) Snapshot() and persist() pipelining

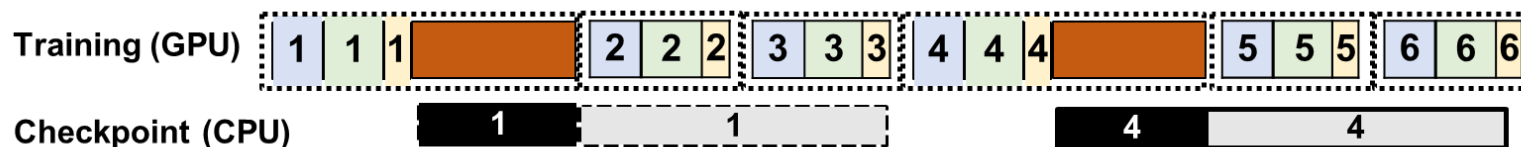
CheckFreq: 细粒度检查点技术



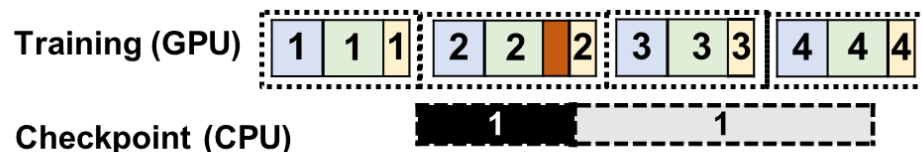
• 两阶段检查点



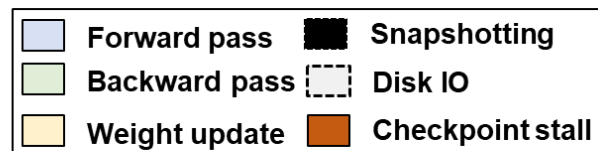
(a) Baseline : Synchronous checkpointing



(b) Only persist() pipelining



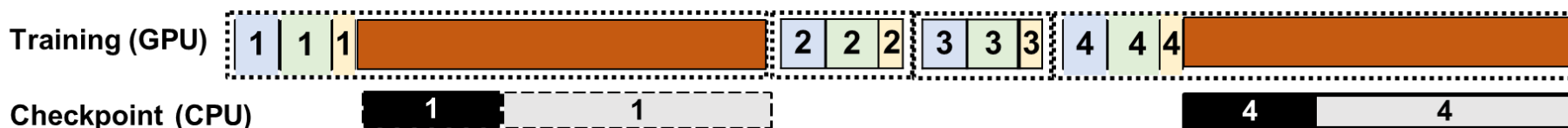
(c) Snapshot() and persist() pipelining



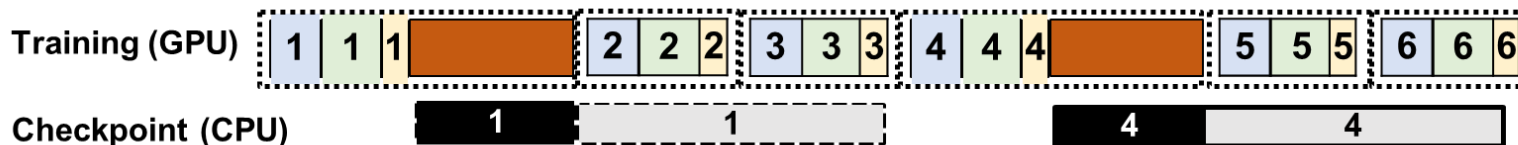
CheckFreq: 细粒度检查点技术



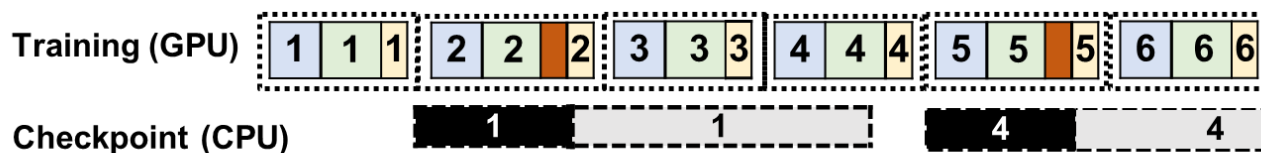
• 两阶段检查点



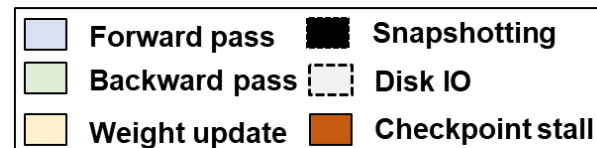
(a) Baseline : Synchronous checkpointing



(b) Only persist() pipelining



(c) Snapshot() and persist() pipelining



CheckFreq: 细粒度检查点技术



- GPU优化的snapshot
- 在GPU上进行序列化操作比CPU快10倍
- CheckFreq在GPU上执行snapshot
- 异步写入CPU内存以减少干扰
- 进一步降低检查点开销

CheckFreq: 细粒度检查点技术

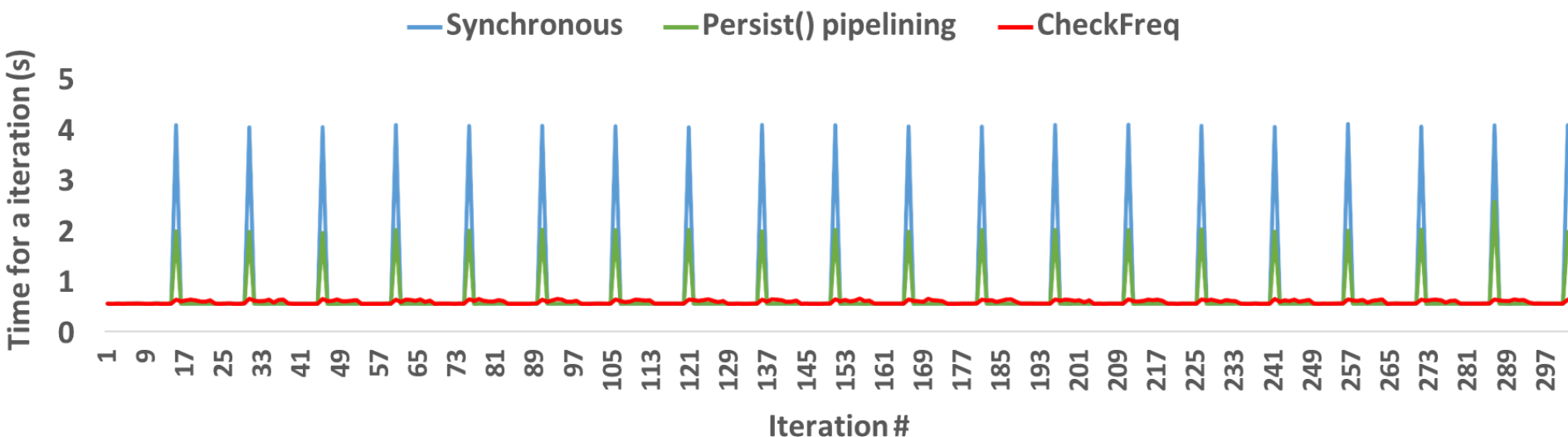


- 多少次迭代进行一次检查点
- 目标：使一次检查点的成本可在 k 次迭代内摊销
- 模型规模越大，快照和持久化的数据量增加，增加检查点的开销
- 硬件导致检查点的不同开销与恢复时间
- 动态采样模型的训练时间、snapshot的开销、persist阶段的I/O带宽等
- 自适应决定频率
- 本质上还是在恢复与训练速度之间的权衡

CheckFreq: 细粒度检查点技术



- 阻塞时间



CheckFreq: 细粒度检查点技术



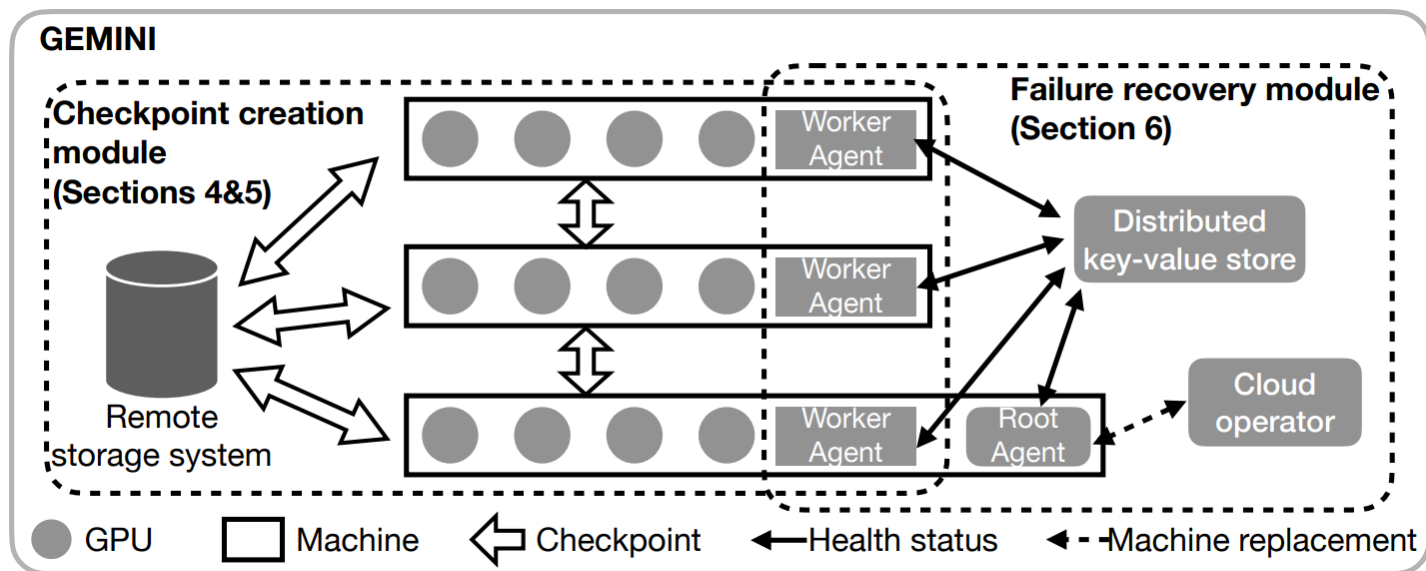
- 恢复时间

Model	Epoch-based (s)	CheckFreq (s)
Res18	840	5
Res50	2100	24
VGG16	5700	25
ResNext	7080	32
DenseNet	2340	7
Inception	3000	27
BERT	4920	85

Gemini: 利用远端内存的检查点



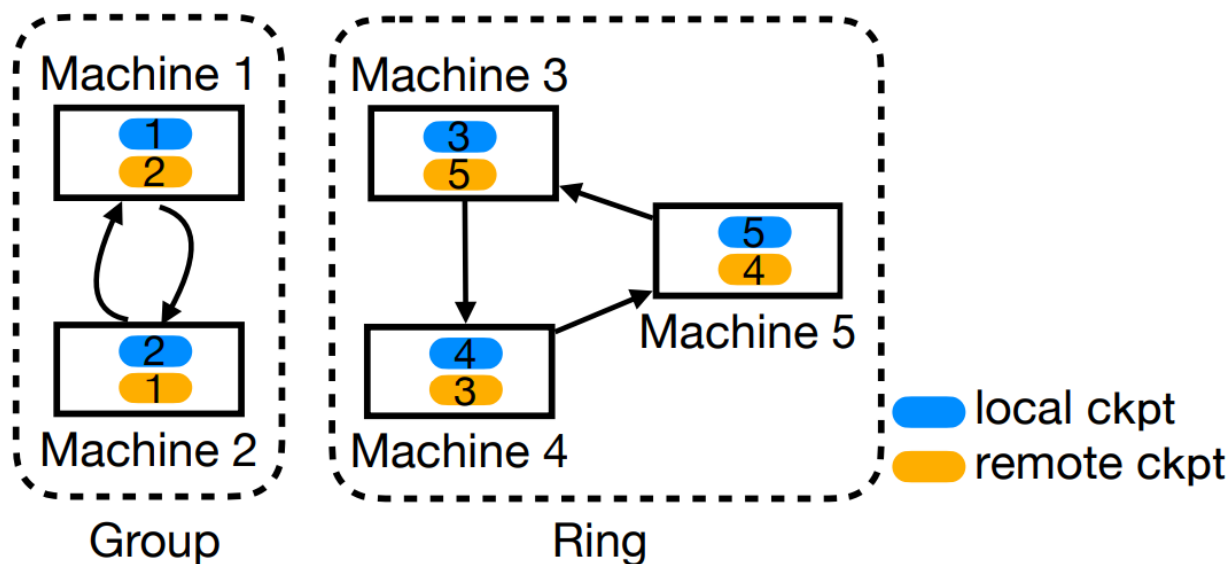
- 集群检查点如何提高效率
- 单卡检查点需要数据落盘，极大拖慢了训练时间
- LLM使用大规模集群训练，能否利用不同节点的主存优化检查点



Gemini: 利用远端内存的检查点

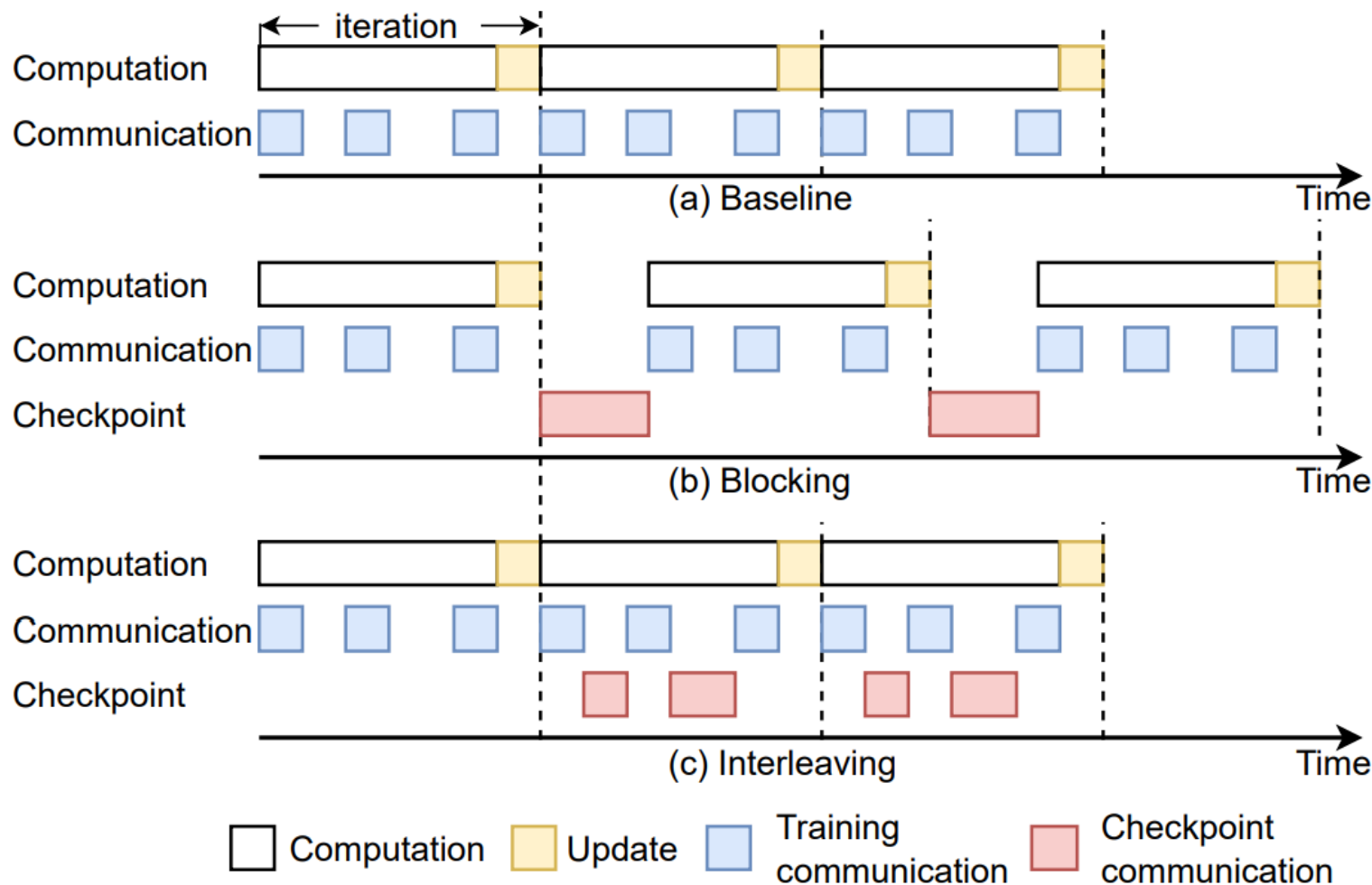


- 如何提高单次检查点的速度?
- 1) Checkpoint只写入集群中其他节点的内存, 并不落盘



会有什么问题?

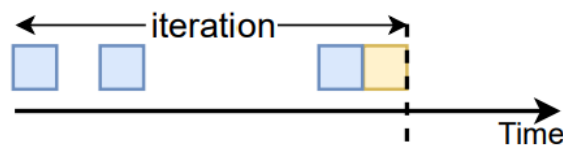
Gemini: 利用远端内存的检查点



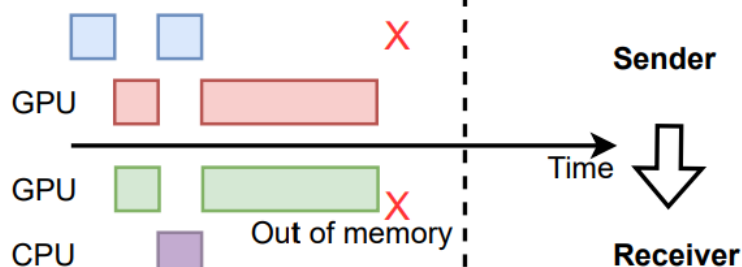
Gemini: 利用远端内存的检查点



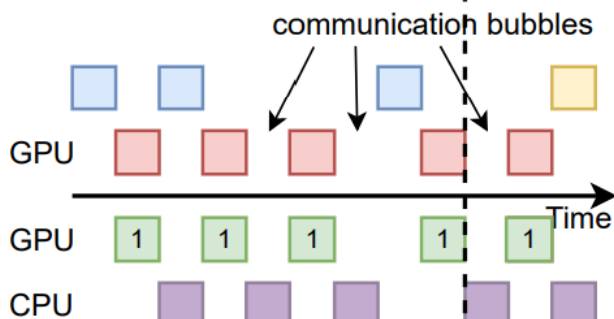
(a) Baseline without checkpointing. Only training communications and Update are displayed.



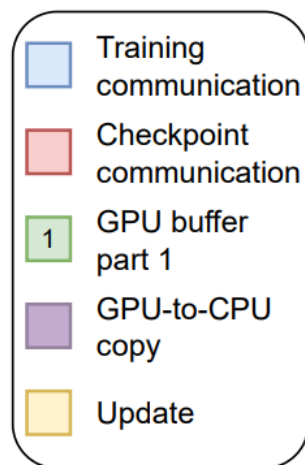
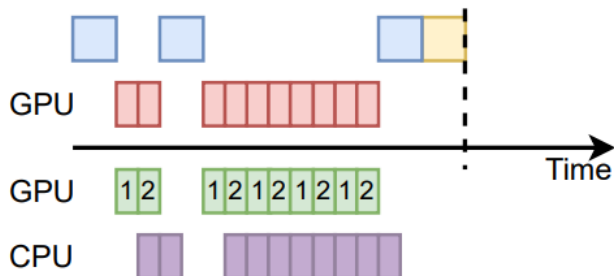
(b) The whole checkpoint is stored in GPU, causing OOM errors.



(c) Checkpoint is partitioned, but GPU-to-CPU copies block checkpoint communications.



(d) The GPU buffer is splitted into two parts. Checkpointing to CPU memory is pipelined.



Gemini: 利用远端内存的检查点

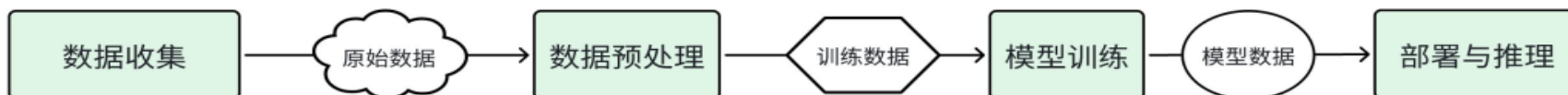


- 检查点的频率?
- 频率尽可能提高, 每个iteration一次
- 如何恢复?
 - 1) GPU错误, 本机未crash → 从本地主存恢复
 - 2) 本机crash → 使用集群中其他节点的主存恢复
 - 3) 集群全部crash时 → 拉起后使用磁盘数据恢复

模型训练为什么需要存储



- 大模型带来新的I/O访问模式



- AI 训练和推理有独特的I/O 模式
 - 数据收集和处理-海量数据
 - 模型训练-检查点
 - 模型部署-参数加载

模型训练为什么需要存储



- 传统存储系统没有针对AI负载设计
 - 数据收集和处理-海量数据
 - 海量数据存储、频繁的元数据操作、带宽需求高
- 模型训练-检查点
 - 巨量大文件顺序写，要求极致吞吐，以最大限度缩短训练停顿时间
- 模型部署-参数加载
 - 大模型文件并发高速读取，要求极高读吞吐需求，缩短推理服务冷启动时长

Tectonic-Shift Meta的训练数据存储



- 基于同构存储介质的分布式文件无法高效地同时满足ML训练负载对**存储容量与IOPS需求**

	存储 bound	IOPS bound	
	存储	IOPS	存储与IOPS
HDD 集群	1.00	9.92	9.92
Flash 集群	6.53	1.88	6.53
HDD与Flash 集群	1.00	1.88	2.69

利用HDD满足存储需求 利用Flash满足IOPS需求

Tectonic-Shift: A Composite Storage Fabric for Large-Scale ML Training, ATC 23

Tectonic-Shift Meta的训练数据存储



- 使用Flash集群满足IOPS需求，使用HDD满足容量需求

	存储 bound	IOPS bound	
	存储	IOPS	存储与IOPS
HDD 集群	1.00	9.92	9.92
Flash 集群	6.53	1.88	6.53
HDD与Flash 集群	1.00	1.88	2.69

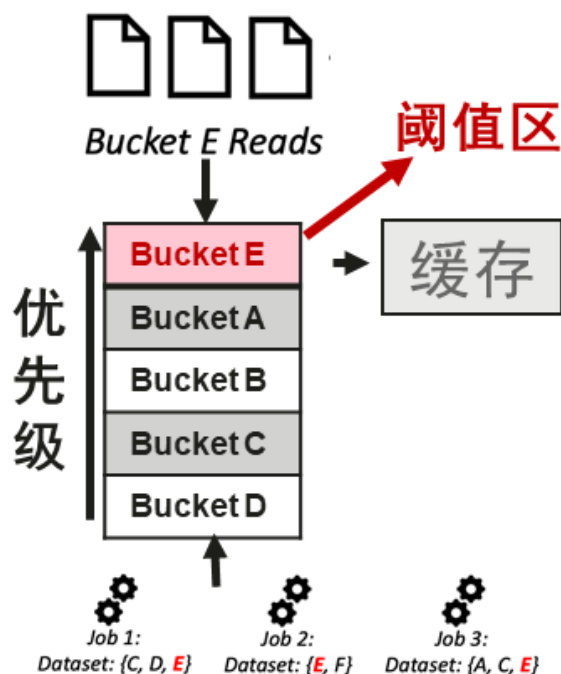
利用HDD满足存储需求 利用Flash满足IOPS需求

Tectonic-Shift: A Composite Storage Fabric for Large-Scale ML Training, ATC 23

Tectonic-Shift Meta的训练数据存储



- 利用ML任务间数据重用性对数据进行缓存，设置阈值区避免缓存抖动。



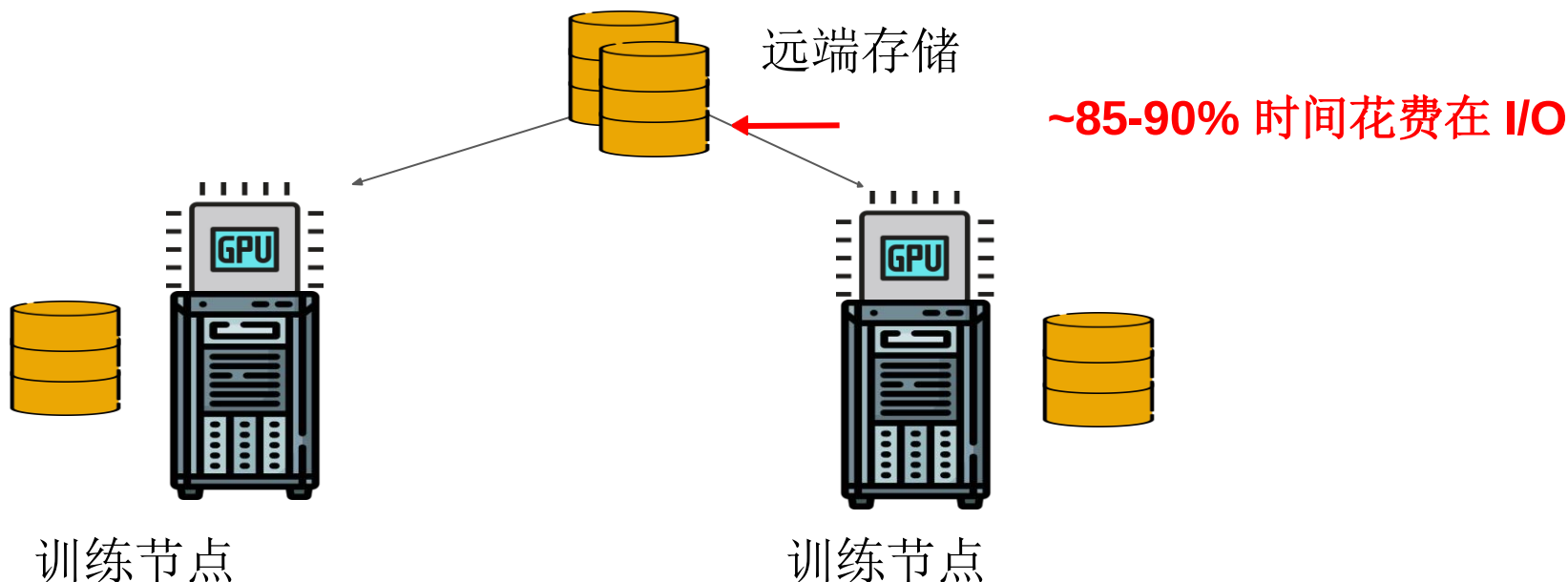
基于任务间数据重用性确定存储优先级和缓存

Tectonic-Shift: A Composite Storage Fabric for Large-Scale ML Training, ATC 23



SHADE: 数据采样预处理优化

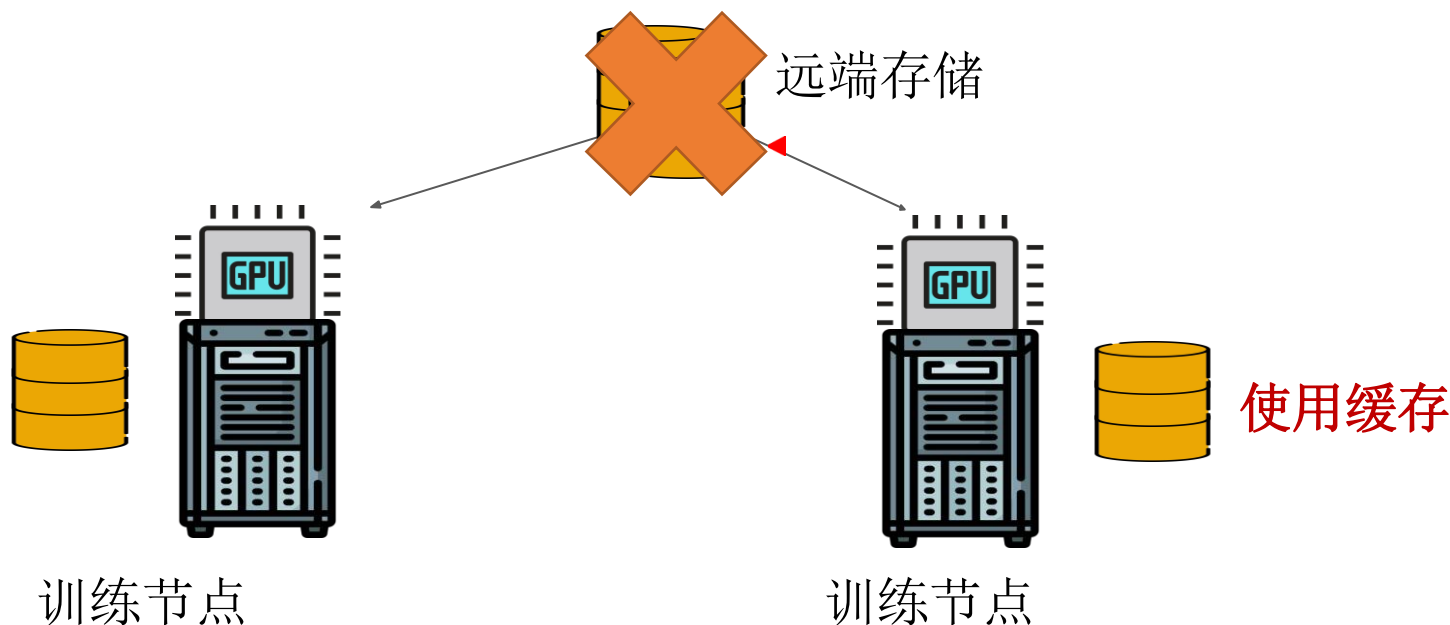
- 机器学习训练任务需要大量的数据
- 在训练时访问远端存储的I/O开销巨大



SHADE: 数据采样预处理优化



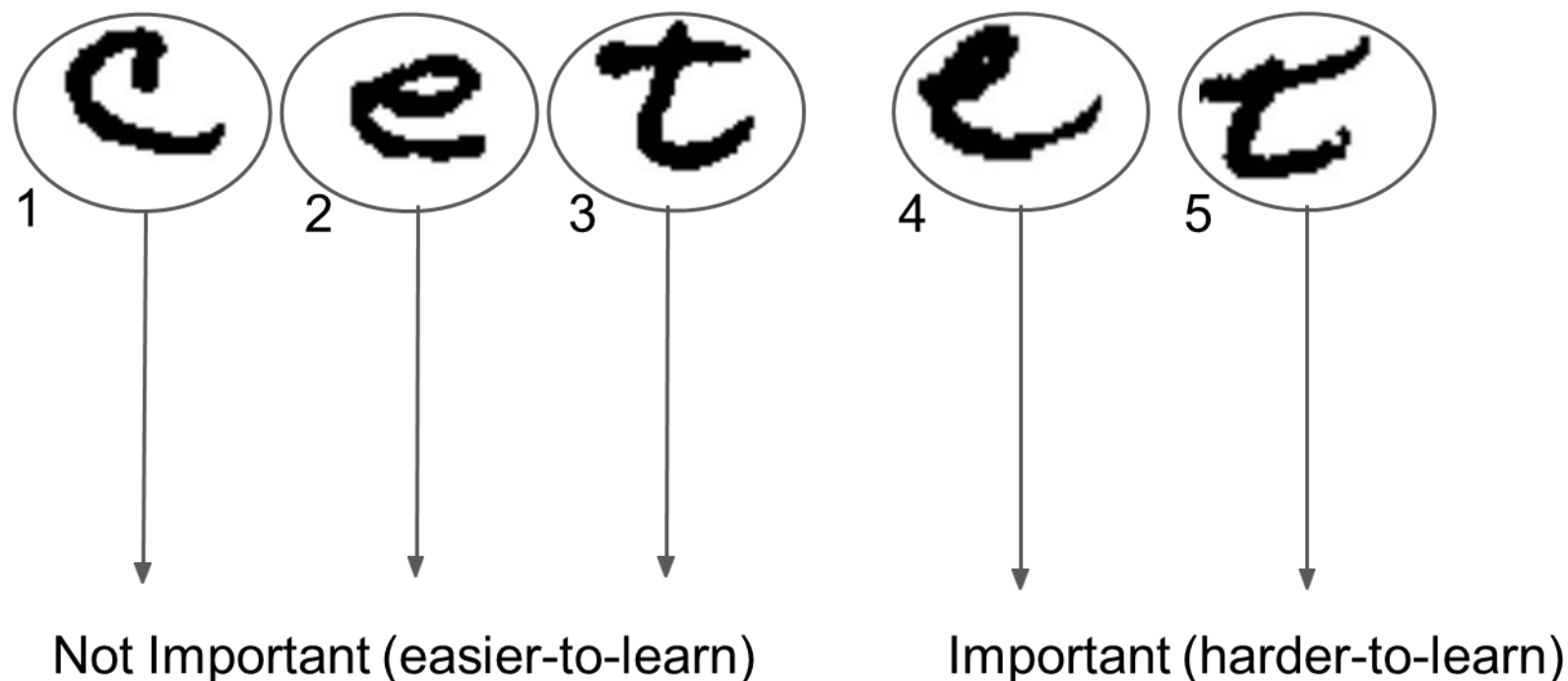
- 机器学习任务中的数据访问模式对现有的缓存策略不友好。
- 机器学习训练任务采用随机采样策略进行训练，导致数据集访问模式随机，数据的局部性差。



SHADE: 数据采样预处理优化



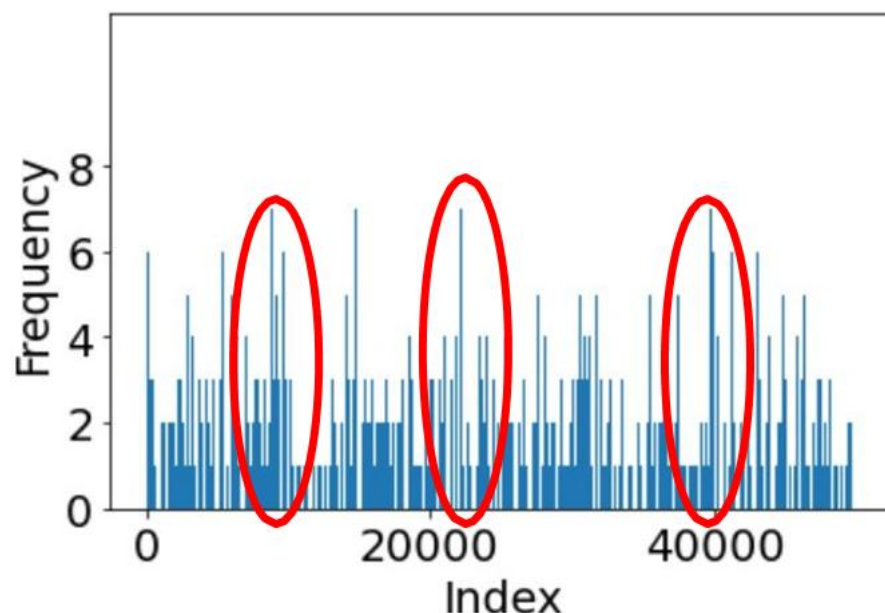
- 应该缓存哪些数据?
- 所有数据并不是都一样重要
- Femnist数据集



SHADE: 数据采样预处理优化



- 传统重要性采样方案
- 26%的数据使用超过2次
- 10%的数据使用超过3次

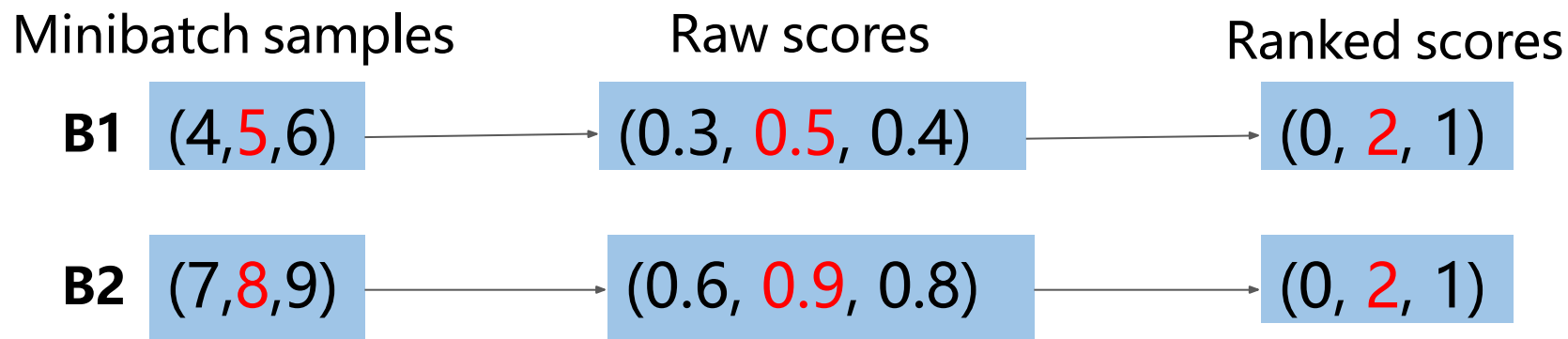


深度学习训练可以区别对待不同的样本。如果我们能够预测 I/O 访问，就可以缓存样本，从而从根本上提高 I/O 效率。



SHADE: 数据采样预处理优化

为样本设置重要性评分以适应优先级缓存





SHADE: 数据采样预处理优化

- 自适应优先级感知预测
- 把最重要的样本保存在缓存中
-
- 优先队列 (PQ) → 当前缓存样本的重要性
- 幽灵缓存 → 所有训练样本的重要性。

比较(importance of cached sample, importance of incoming sample)

↑
优先队列

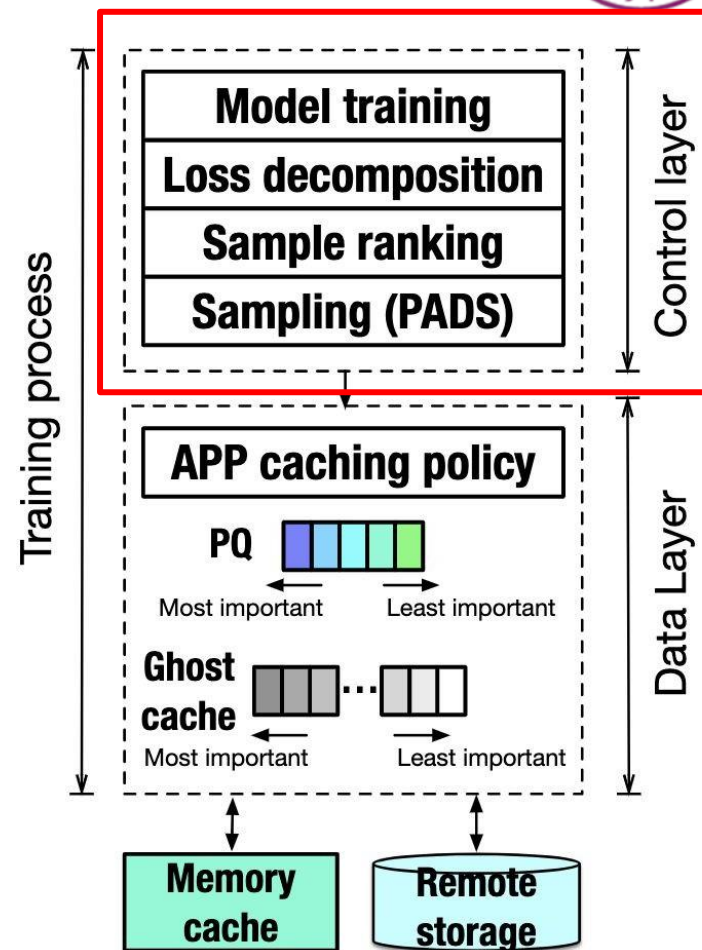
↑
幽灵缓存

比较重要性评分以保留最重要的样本缓存

SHADE: 数据采样预处理优化



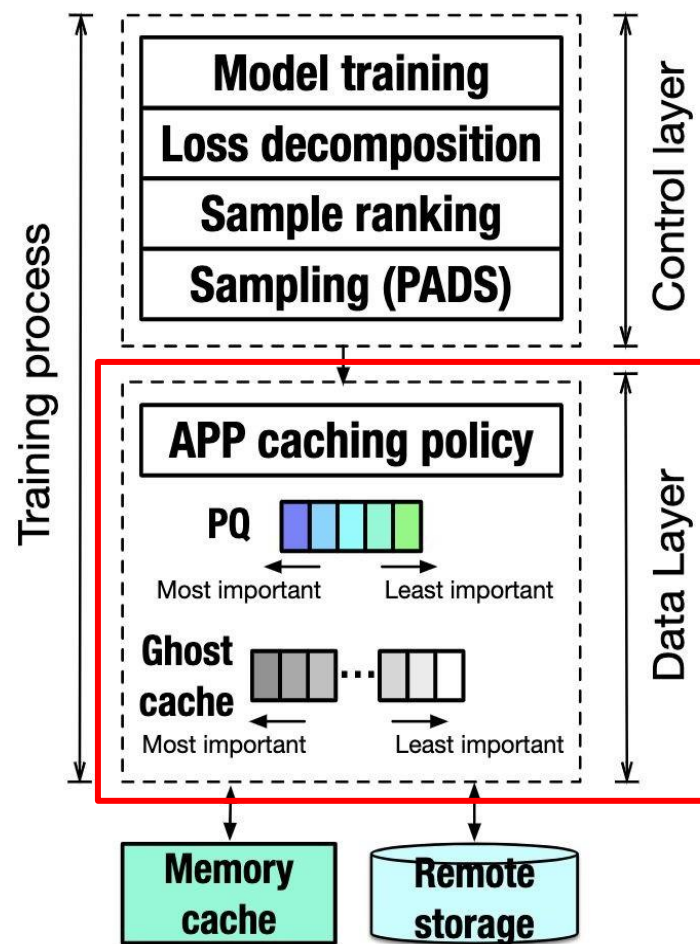
- SHADE 控制层
- 记录细粒度的采样重要性，并对数据采样进行排名



SHADE: 数据采样预处理优化



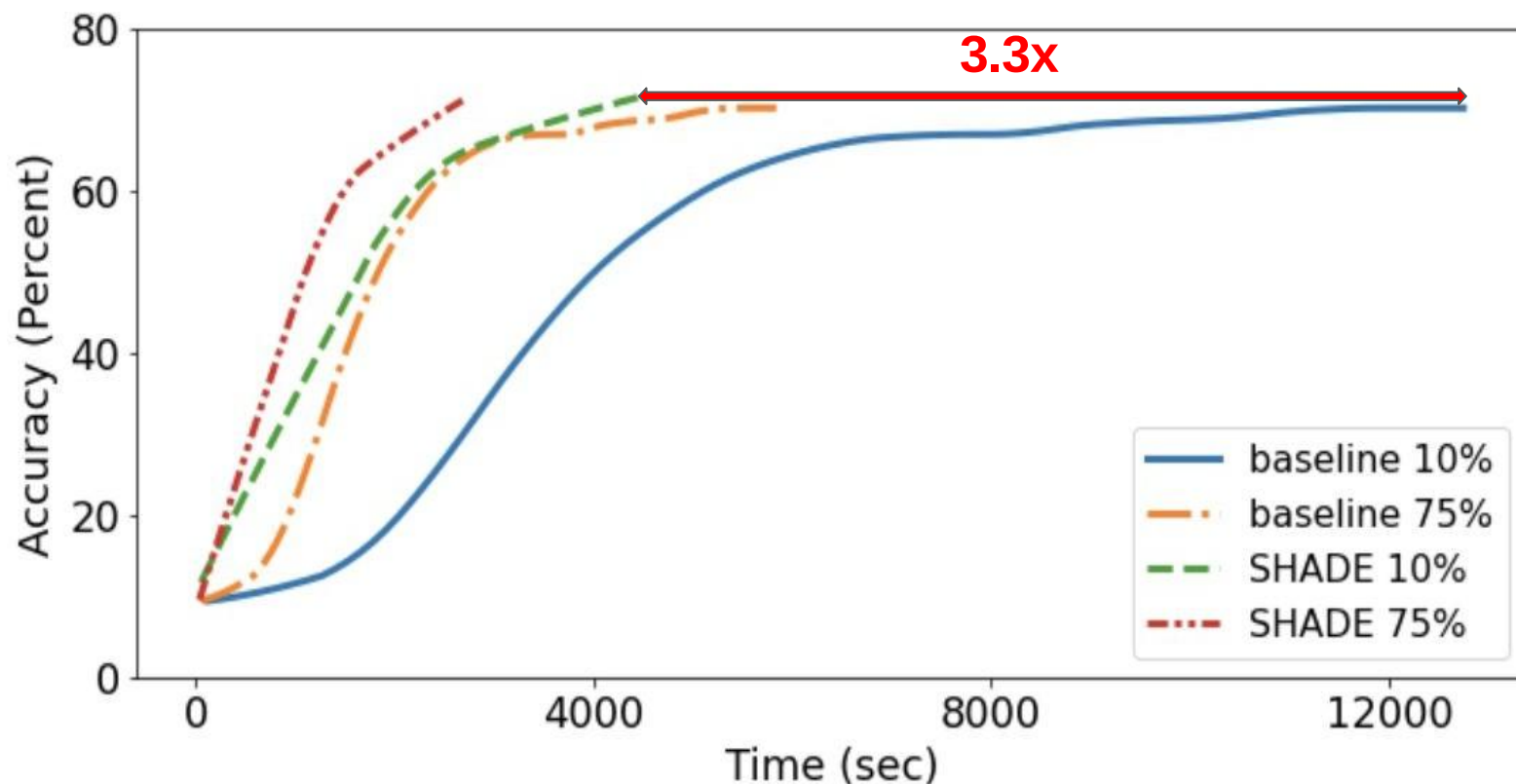
- SHADE 数据层
- 缓存的存储与检索
- 更新缓存



SHADE: 数据采样预处理优化



- 更快达到相同的模型精度



模型训练为什么需要存储

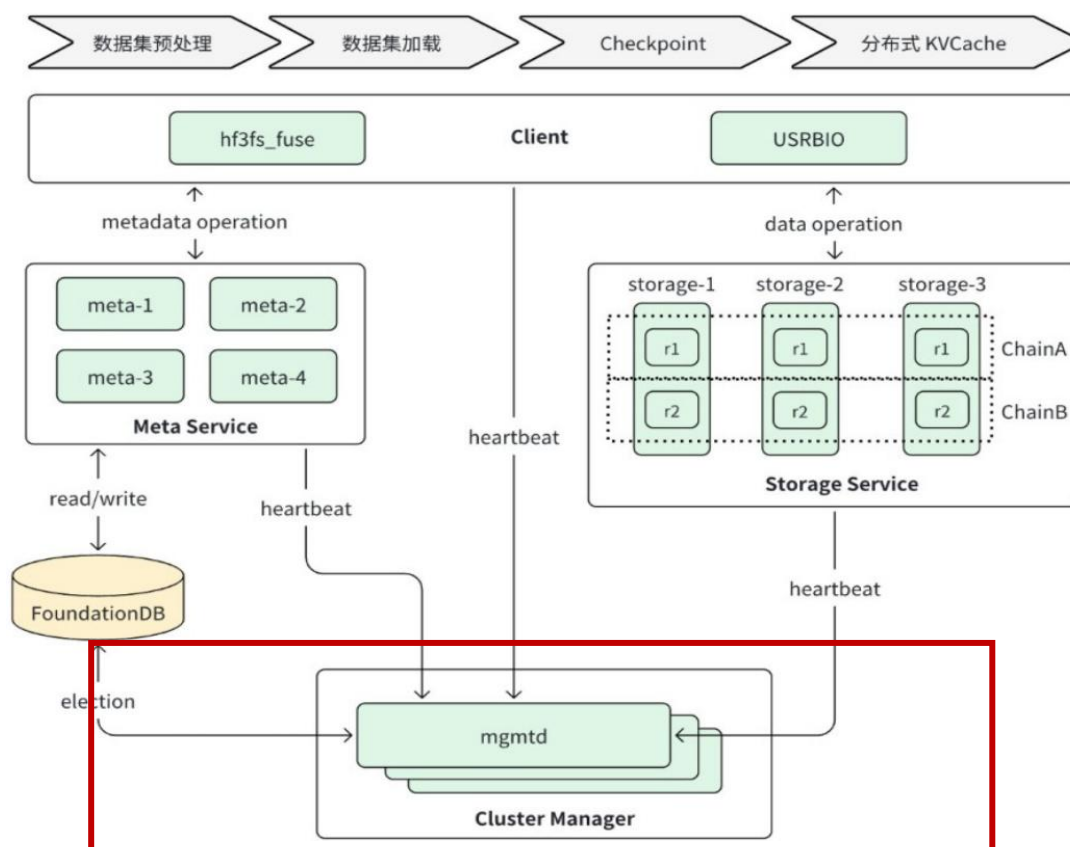


- 传统存储系统没有针对AI负载设计
 - 数据收集和处理-海量数据
 - 海量数据存储、频繁的元数据操作、带宽需求高
- 模型训练-检查点
 - 巨量大文件顺序写，要求极致吞吐，以最大限度缩短训练停顿时间
- 模型部署-参数加载
 - 大模型文件并发高速读取，要求极高读吞吐需求，缩短推理服务冷启动时长

Deepseek 3FS: AI特化的分布式文件系统



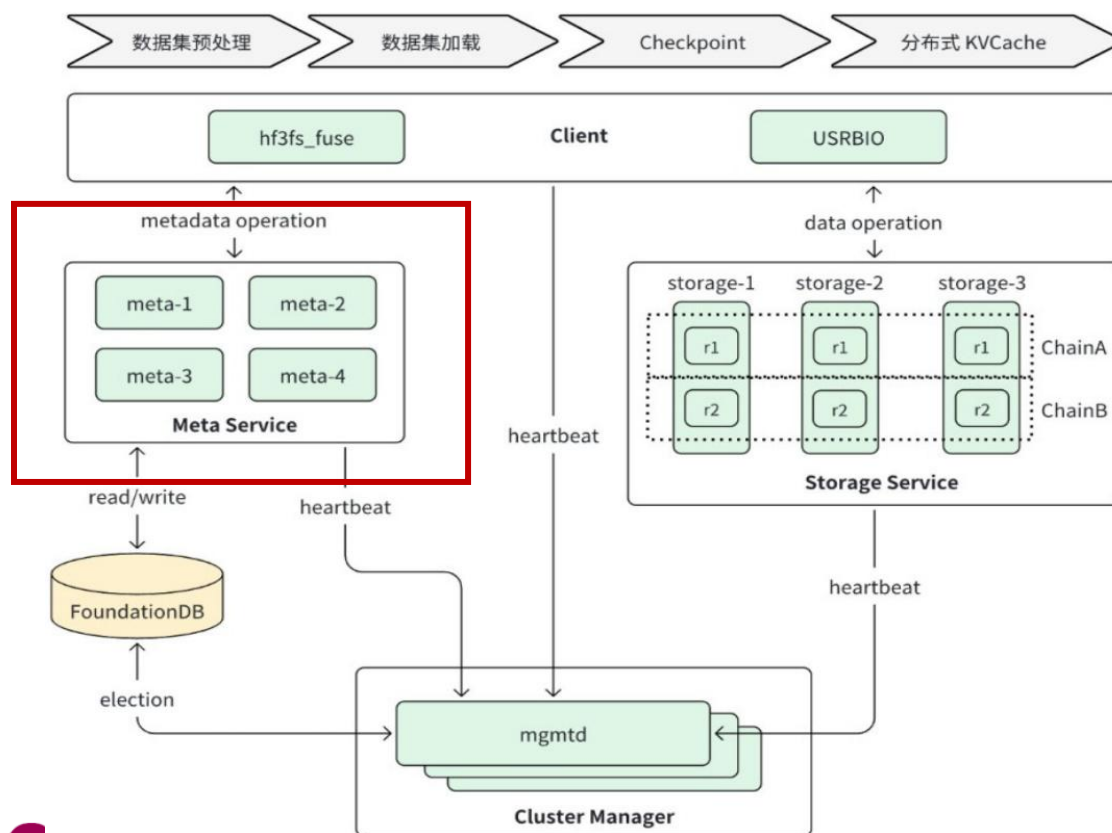
- Cluster Manager (mgmtd)
- 集群大脑：负责节点状态、心跳维护



Deepseek 3FS: AI特化的分布式文件系统



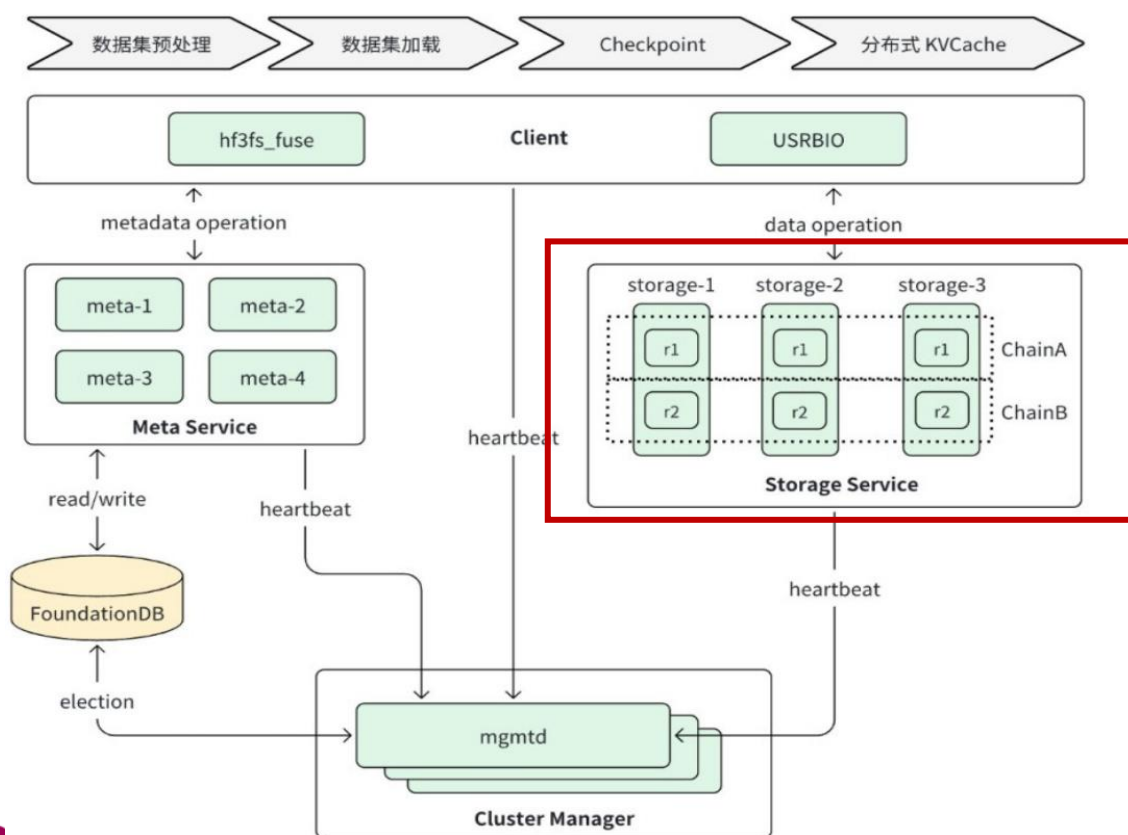
- Meta Service (Metadata)
- 无状态 POSIX 翻译层，基于 FoundationDB（键值对存储） 保证强一致性事务。



Deepseek 3FS: AI特化的分布式文件系统



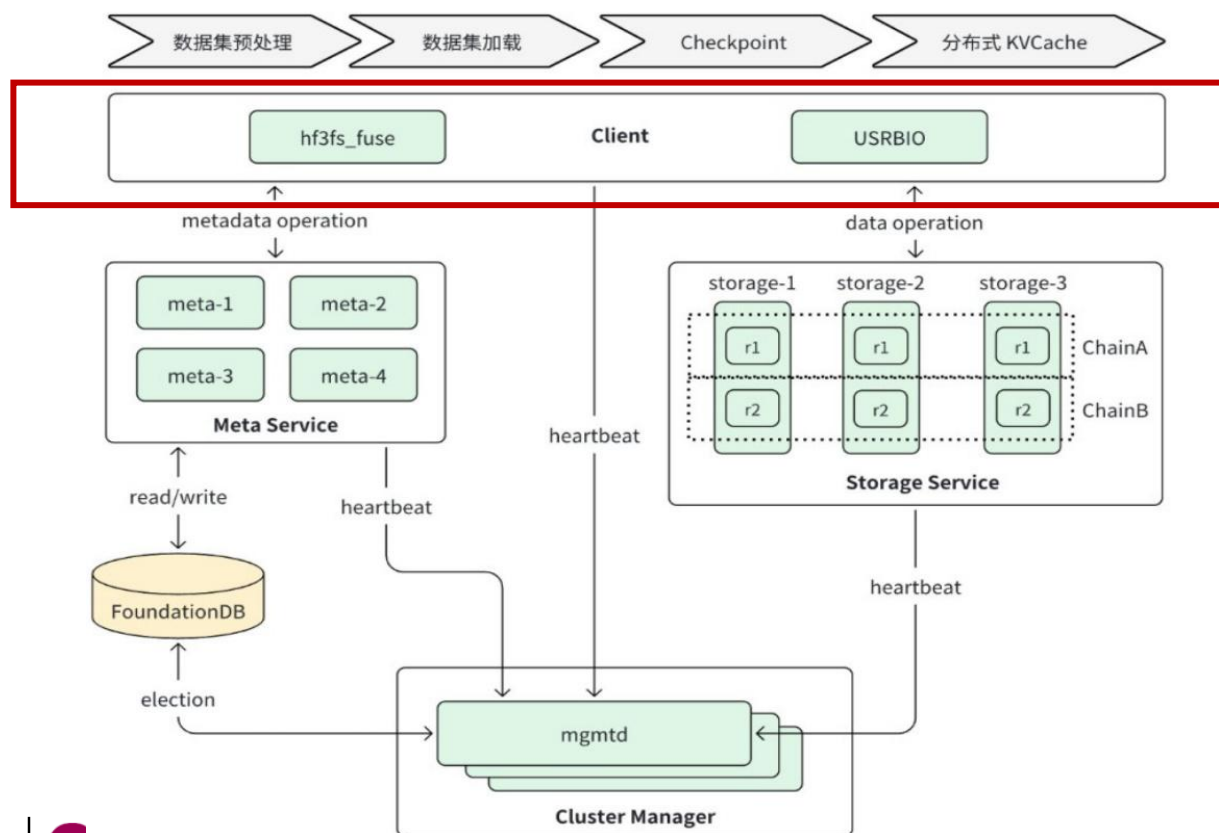
- Storage Service
- 数据节点：管理数据存储，负责物理数据的持久化与可靠性。



Deepseek 3FS: AI特化的分布式文件系统



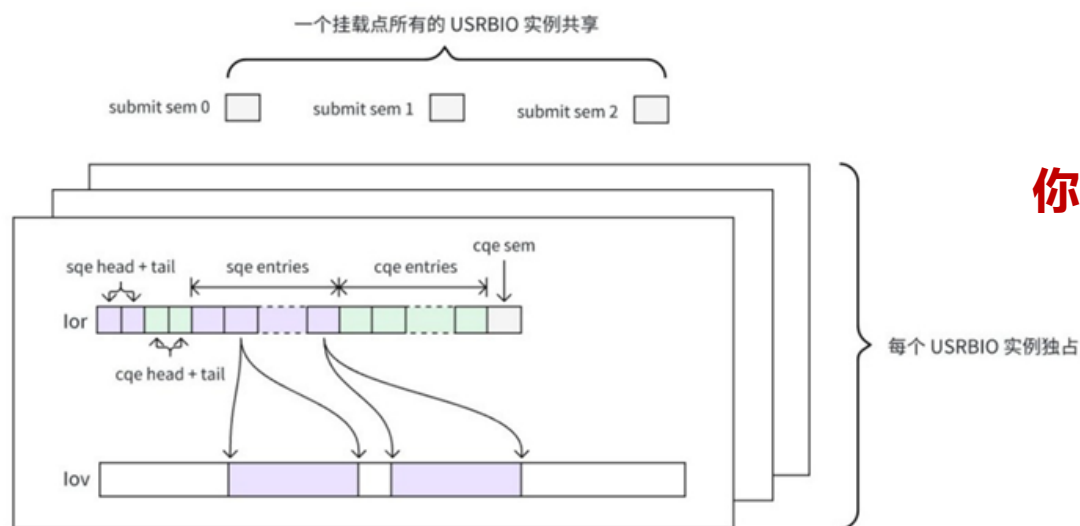
- Client
- 接入枢纽：提供两种灵活方式（SDK/Mount）连接高性能计算应用。



Deepseek 3FS: AI特化的分布式文件系统



- 客户端优化: FUSE 的易用性 vs. USRBIO 的极致性能
- 标准的 FUSE 因内核态/用户态频繁切换及多次数据拷贝, 在高吞吐 AI 训练场景下成为性能瓶颈。
- USRBIO
- 核心思想: 绕过内核, 实现端到端的用户态零拷贝 I/O。

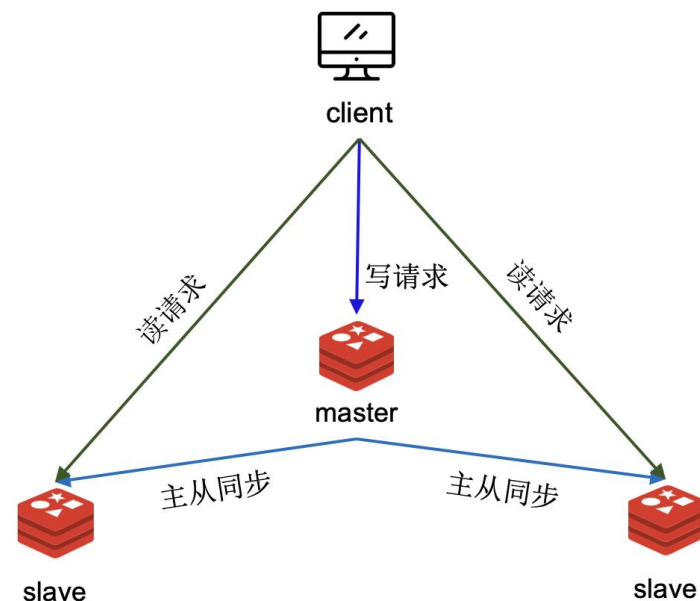


你还在哪里见到过类似的设计?

Deepseek 3FS: AI特化的分布式文件系统



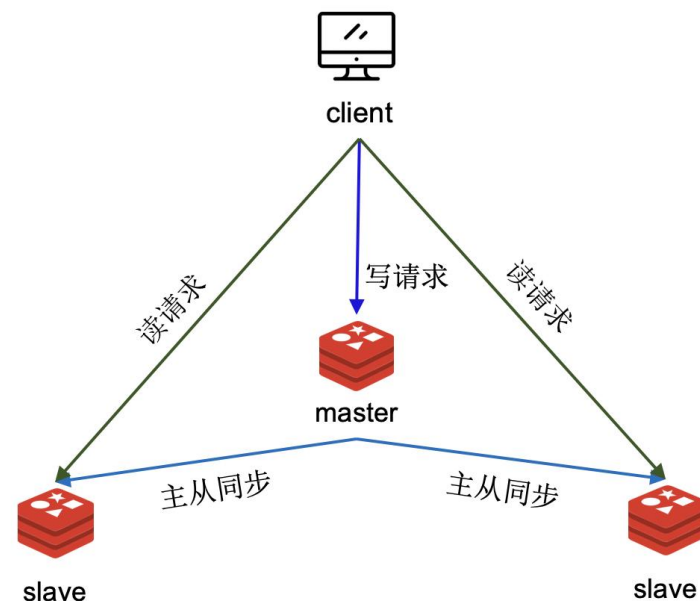
- 为“读”优化的链式复制(CRAQ)
- 传统分布式系统中的主从复制
- 主节点接收写入，异步地将数据推送给所有从节点。
- 通常有两种读取模式：
 1. 强一致读：必须从主节点读取，确保读到最新数据。从节点只提供最终一致或旧数据的读。
 2. 最终一致读：可以从任意从节点读，性能高，但可能读到旧数据。



Deepseek 3FS: AI特化的分布式文件系统



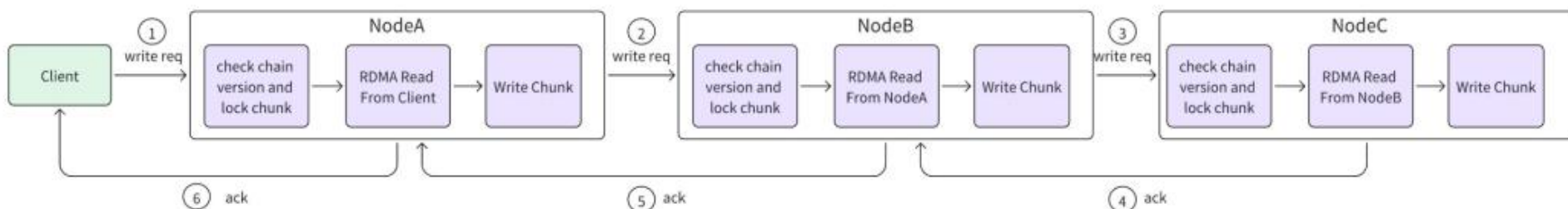
- 为“读”优化的链式复制(CRAQ)
- 传统分布式系统中的主从复制
- 如果要求强一致读，所有读压力都集中在主节点，成为瓶颈。从节点的计算资源无法用于强一致读。
- 写入延迟较低。主节点并行推送数据，只需等待部分节点确认即可返回。
- 故障恢复相对简单。主节点故障后，需要选举一个新主



Deepseek 3FS: AI特化的分布式文件系统



- 为“读”优化的链式复制(CRAQ)



- 写入遵循一条链：客户端 -> 头节点 -> 中间节点 -> ... -> 尾节点。数据按链顺序同步。
- 写入延迟较高**。写入需要依次经过链上的每个节点，网络跳数多，尾节点确认后才返回。
- 所有节点（包括头、中、尾）都可以提供强一致性的读取。
- 读扩展性极强**。读请求可以均匀分散到链上的所有节点，读取吞吐量随节点数增加而线性增长。
- 契合AI任务时需要从存储中海量并发读取文件的极高I/O需求。

Deepseek 3FS: AI特化的分布式文件系统



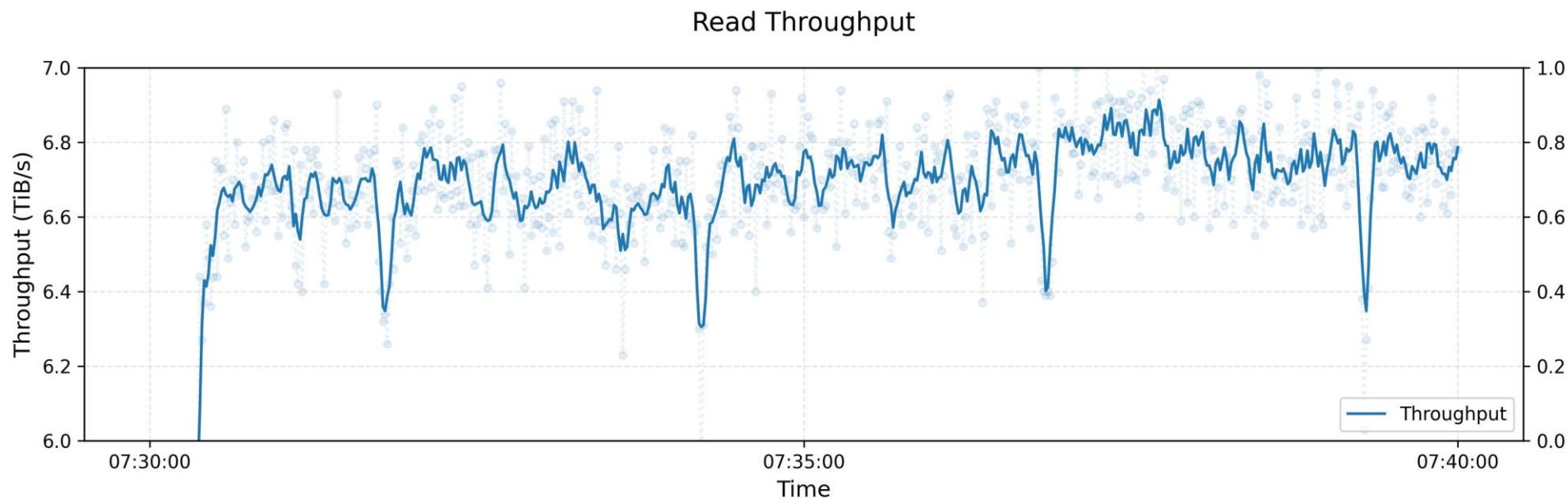
- 合并多个小文件为大文件
- 直接存取海量小文件会产生大量元数据请求（如 open , stat）, 拖垮后端元数据服务磁盘与IO。
- 打包策略 (Packing), 将大量小文件打包一个大文件
- 设计FFRecord文件格式, 减少了训练时打开大量小文件的开销; 支持随机批量读取, 提升读取速度
- 小文件问题交由上层应用在数据预处理阶段解决

checksum		N	
checksums		offsets	
sample1	sample2		sample N

Deepseek 3FS: AI特化的分布式文件系统



- 峰值带宽接近存储介质物理带宽



<https://github.com/deepseek-ai>

Deepseek 3FS: AI特化的分布式文件系统



- 设计启发
- 系统设计的本质是trade-off
- 在有限资源下，不追求“全能”，而是聚焦核心场景。
- 3FS专注优化高吞吐读写负载（大文件、大IO），不深度优化小文件。
- 打破系统分层的壁垒，让应用层与存储层协同工作，达成全局最优。
- FFSRecord: 将小文件问题交由上层应用在数据预处理阶段解决。

<https://github.com/deepseek-ai>

小结：模型训练中的存储系统



- 模型参数的不断增长推动着模型算法性能的不不断提升
- 模型参数数据量大、训练数据集规模大
- 模型参数的增长在AI全生命周期给存储系统提出新的诉求
 - 数据集存储：海量小文件管理难
 - 采样预处理：数据采样随机性高
 - 模型训练：大模型训练的存储容量与容错的需求高、开销大
- 面向AI的数据存储、采样的优化 (3FS, SHADE, shift)
- 利用异构存储卸载模型参数 (ZERO-offload)
- 训练中的检查点机制 (Checkfreq/Gemini)



目录

- 大模型训练中的存储
- 大模型推理中的存储
- 大模型指导存储系统设计

模型推理为什么需要存储



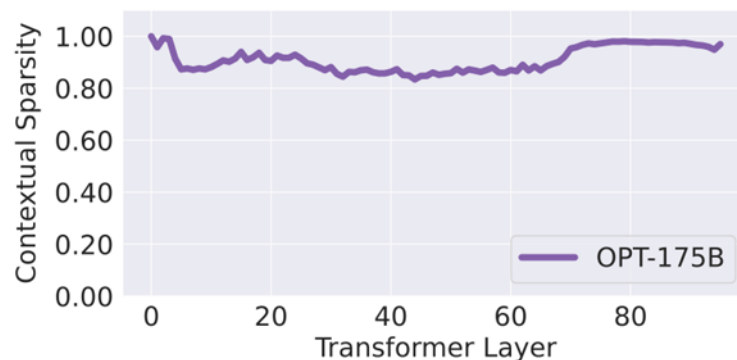
- 模型参数巨大
- 内存/显存容量有限的设备无法承载全部参数
- 使用异构存储设备卸载

什么样的设备适合此种场景？

LLM in flash[Apple]



- 在手机上本地推理大模型 为什么我们需要本地推理？
- 端侧设备（手机）的SSD存储容量大，但IO带宽远小于内存。
- 使用 SSD 存储模型参数会影响推理延迟，而模型推理对于延迟敏感。
- 模型参数也有“冷热”

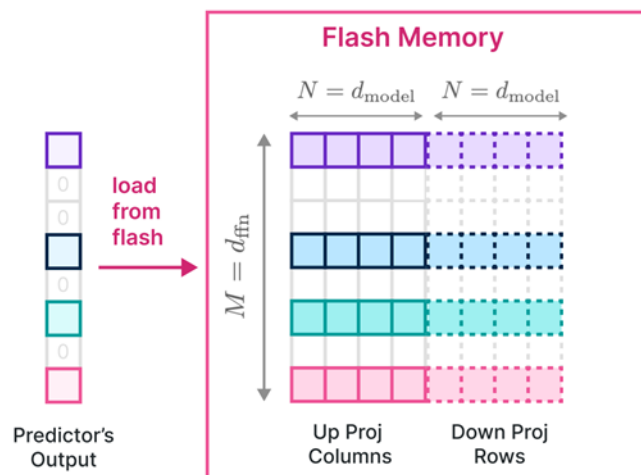


MLP层计算的结果中，
90%位置的值为0

LLM in flash[Apple]



- 基于MLP层输出的激活量结果稀疏的特点
- 利用小模型推测会被激活的非零结果对应的参数
- 在推理时仅上传这些参数至内存，从而降低数据传输量

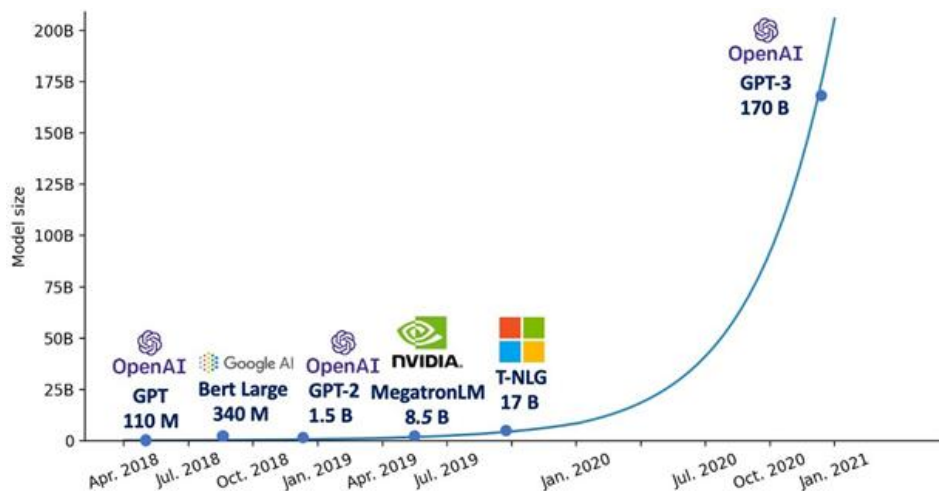


仅上传非0结果对应的
模型参数至内存

Powerinfer: 消费级GPU推理大模型



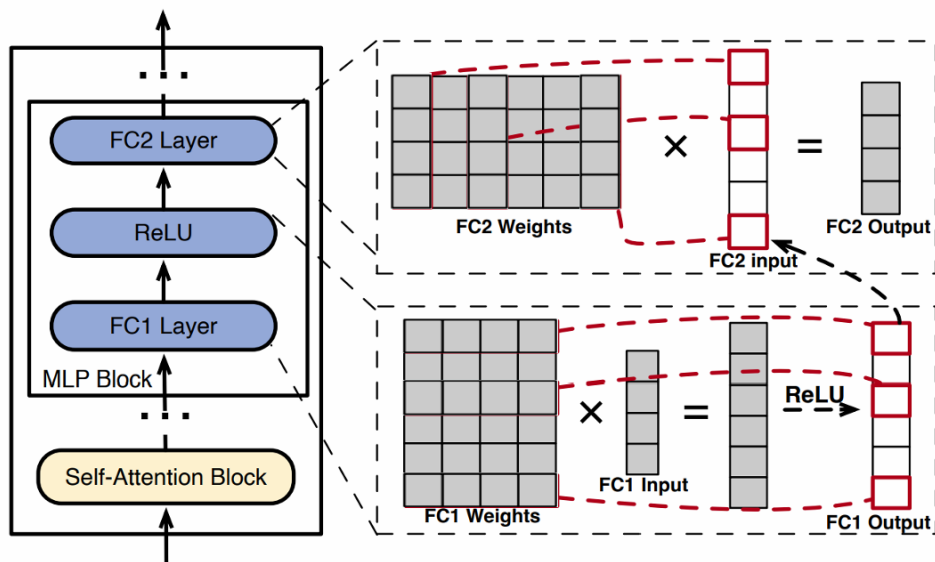
- 消费级GPU显存容量非常有限
- 大模型参数量不断增长
- 即使使用量化，也超过GPU容量
- 使用GPU+CPU memory



Powerinfer: 消费级GPU推理大模型



- 模型参数激活具有明显长尾效应
- 少量的神经元承载了大量计算

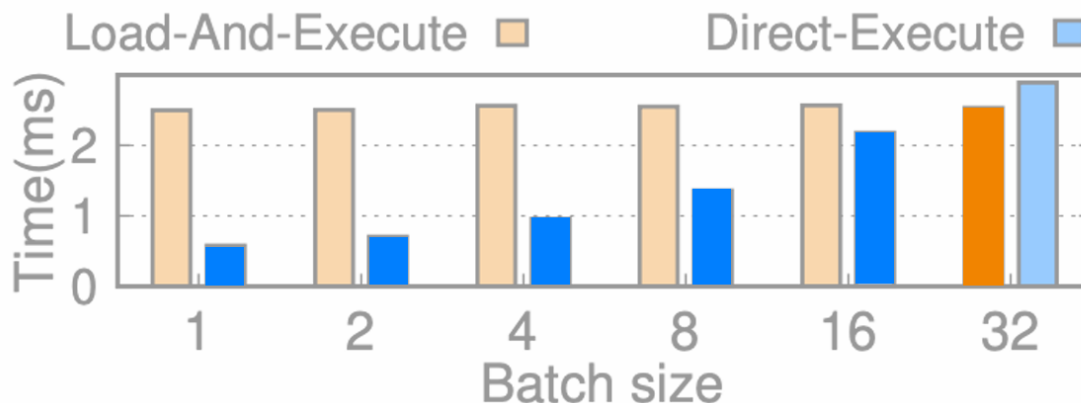


- 对冷热参数分别处理？

Powerinfer: 消费级GPU推理大模型



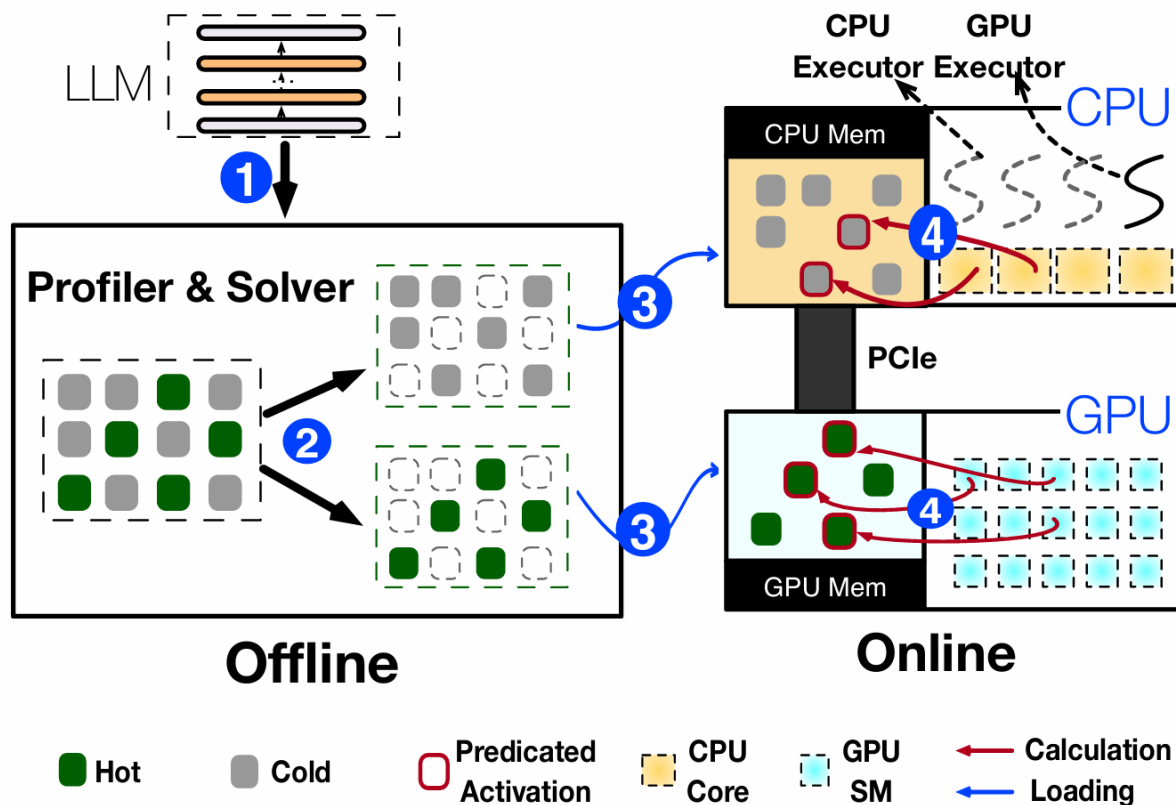
- CPU也具有计算能力，能让它处理冷参数的计算吗？
- 有时候直接在CPU计算表现更好





Powerinfer: 消费级GPU推理大模型

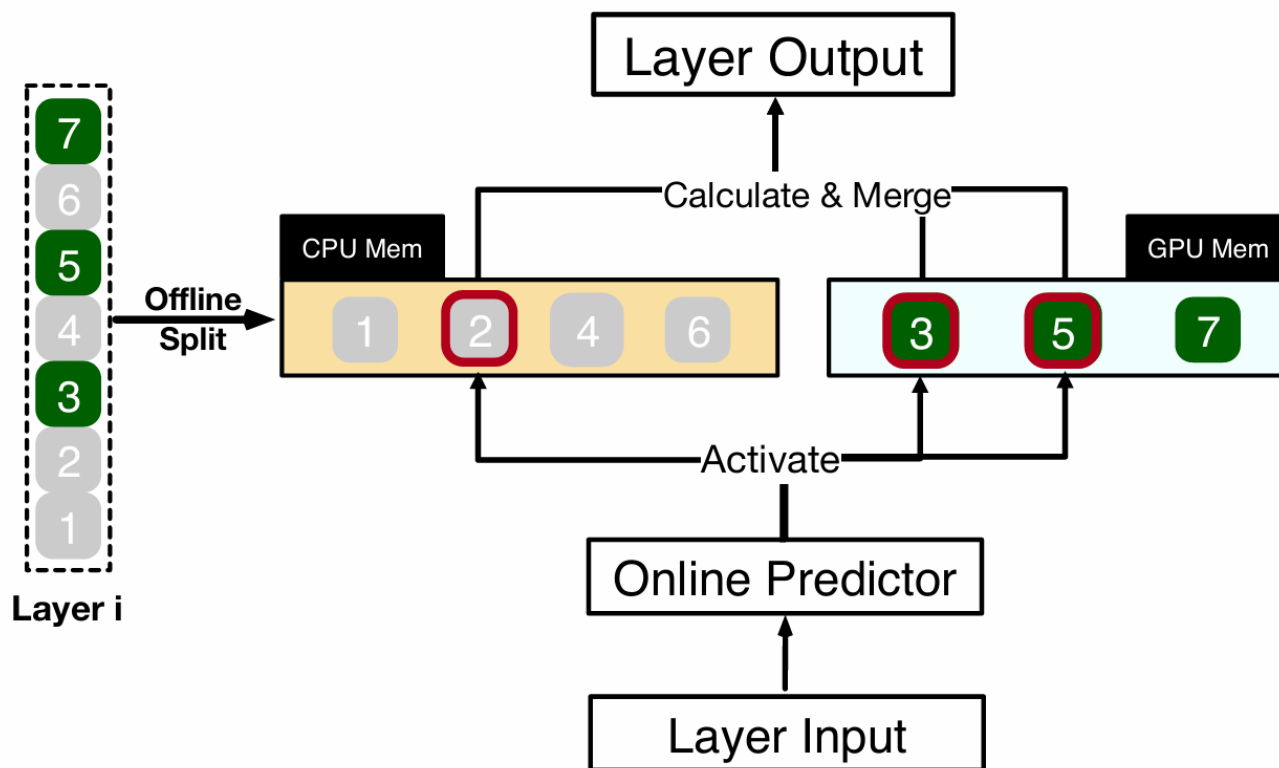
- GPU: 热神经元的计算, 空间小
- CPU: 冷神经元的计算, 空间大



Powerinfer: 消费级GPU推理大模型



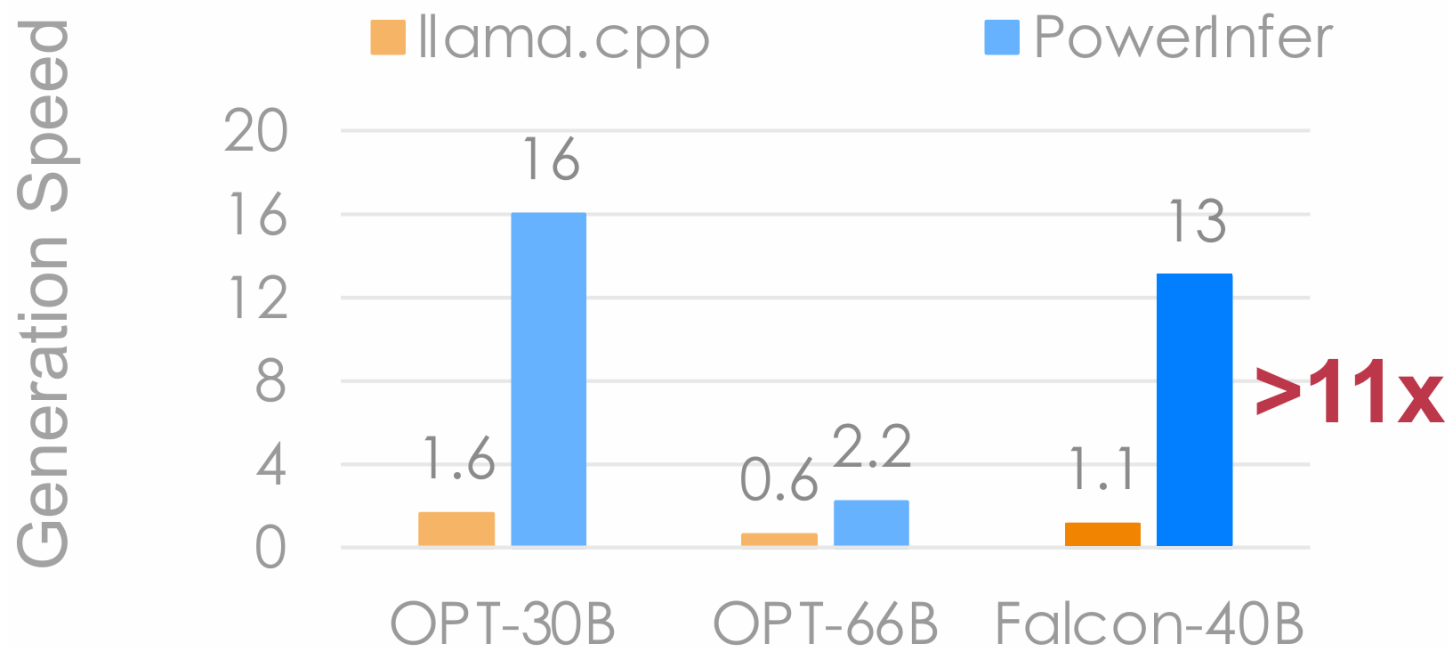
- 离线分析，判定参数的冷热



Powerinfer: 消费级GPU推理大模型



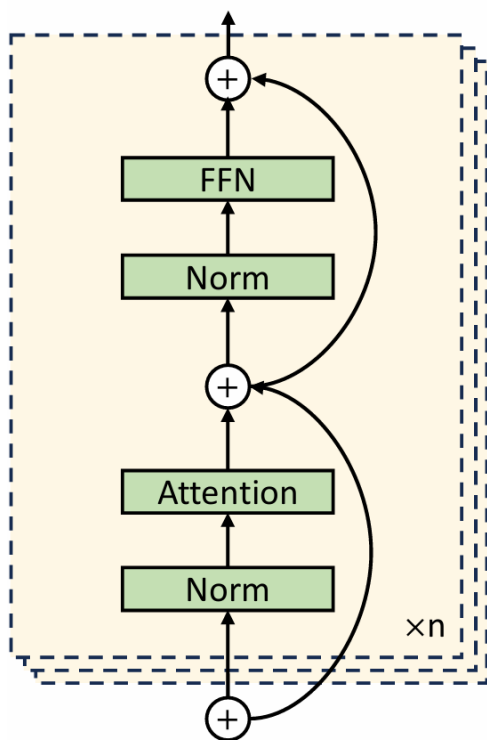
- 单张RTX 4090 (24GB) 实现高速推理



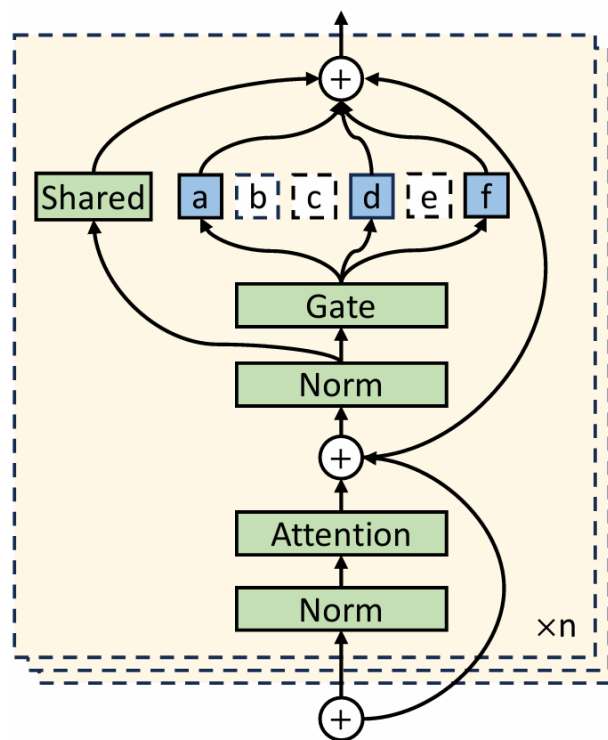
Ktransformer: 本地推理Deepseek?



- MOE大模型的兴起



(a) Dense model.

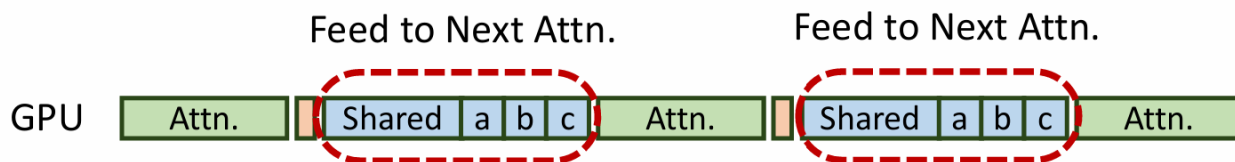


(b) Sparse MoE model.

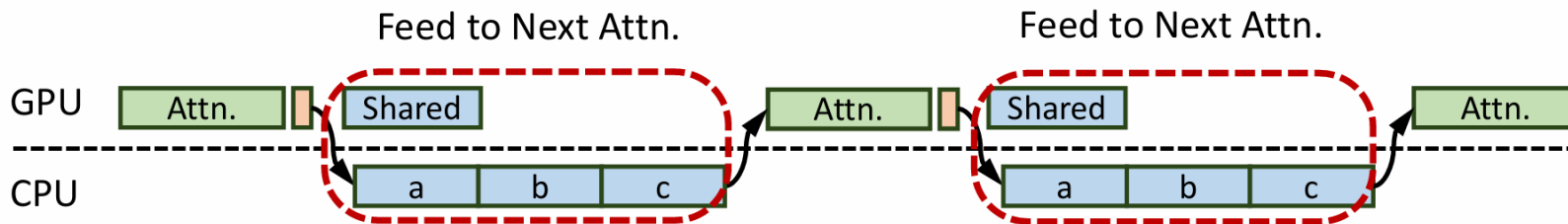
Ktransformer: 本地推理Deepseek?



- 将参数量巨大的专家参数卸载到CPU
- GPU和CPU协同推理



(a) GPU only.

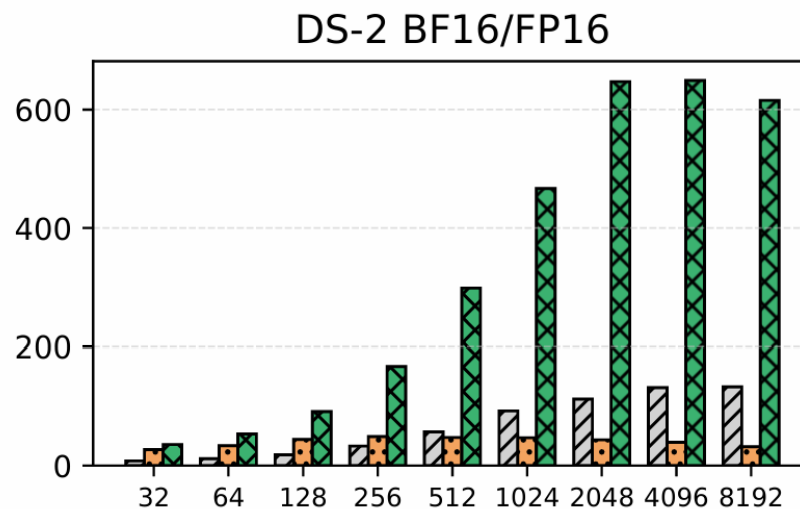
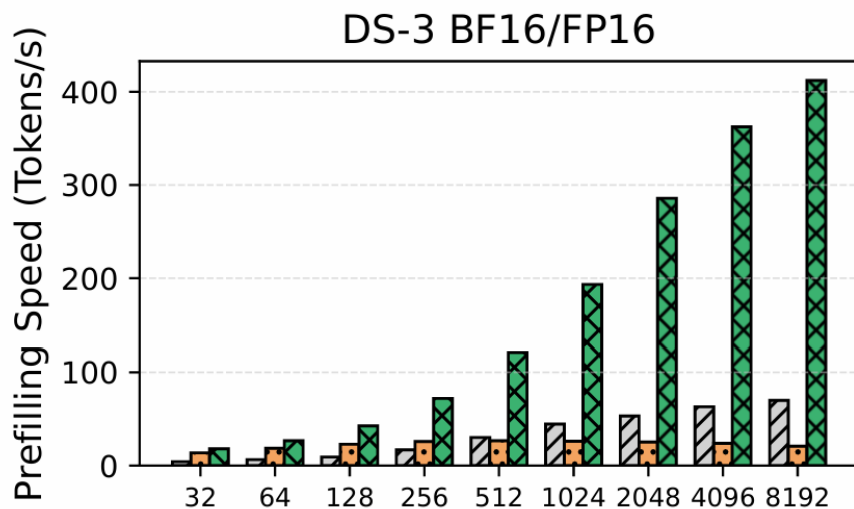


(b) CPU-GPU orchestration.

Ktransformer: 本地推理Deepseek?



- 40GB A100 GPU + **1TB memory**



仍然需要巨大容量的主机内存

KTransformers: Unleashing the Full Potential of
CPU/GPU Hybrid Inference for MoE Models

模型推理为什么需要存储

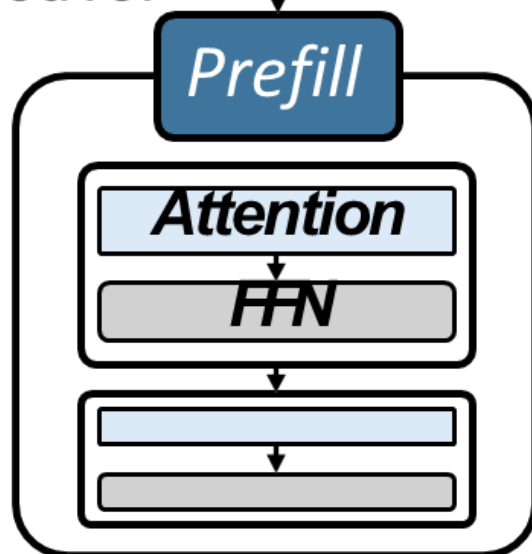


- 模型推理流程
 - Prefill阶段：处理输入提示，生成首个Token
 - Decode阶段：自回归生成后续Token
 - 每个Token的生成都需要访问完整的KV Cache
 - • TTFT(首Token延迟) vs TBT(Token间延迟)

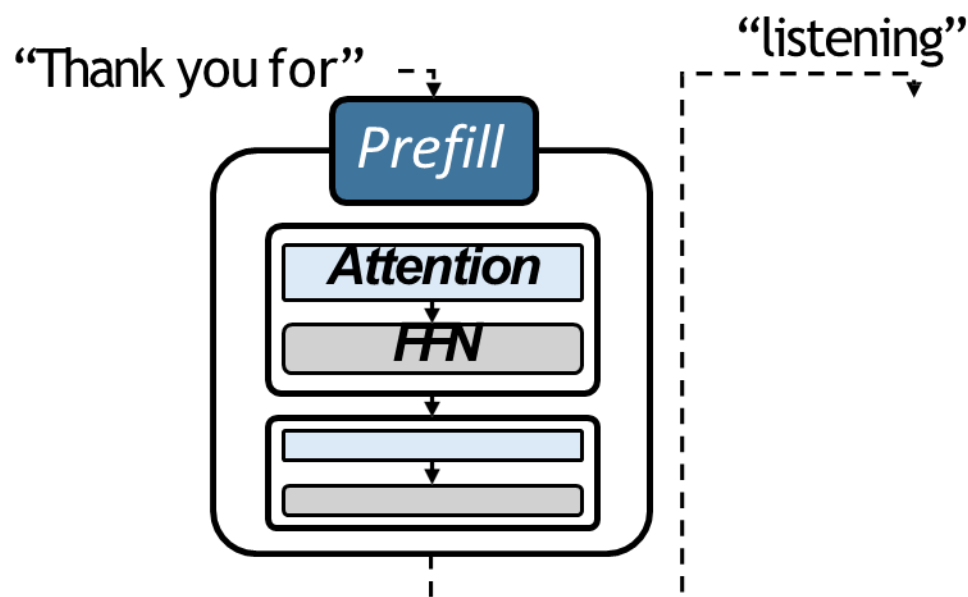
模型推理为什么需要存储



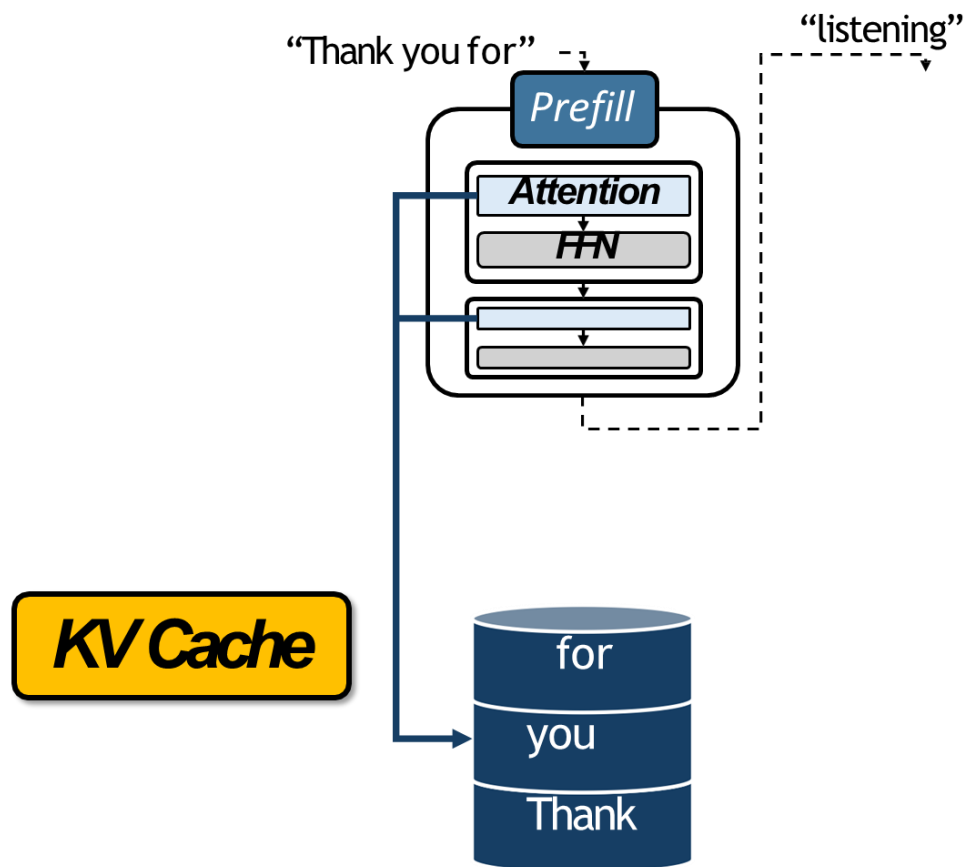
“Thank you for” - $\downarrow$



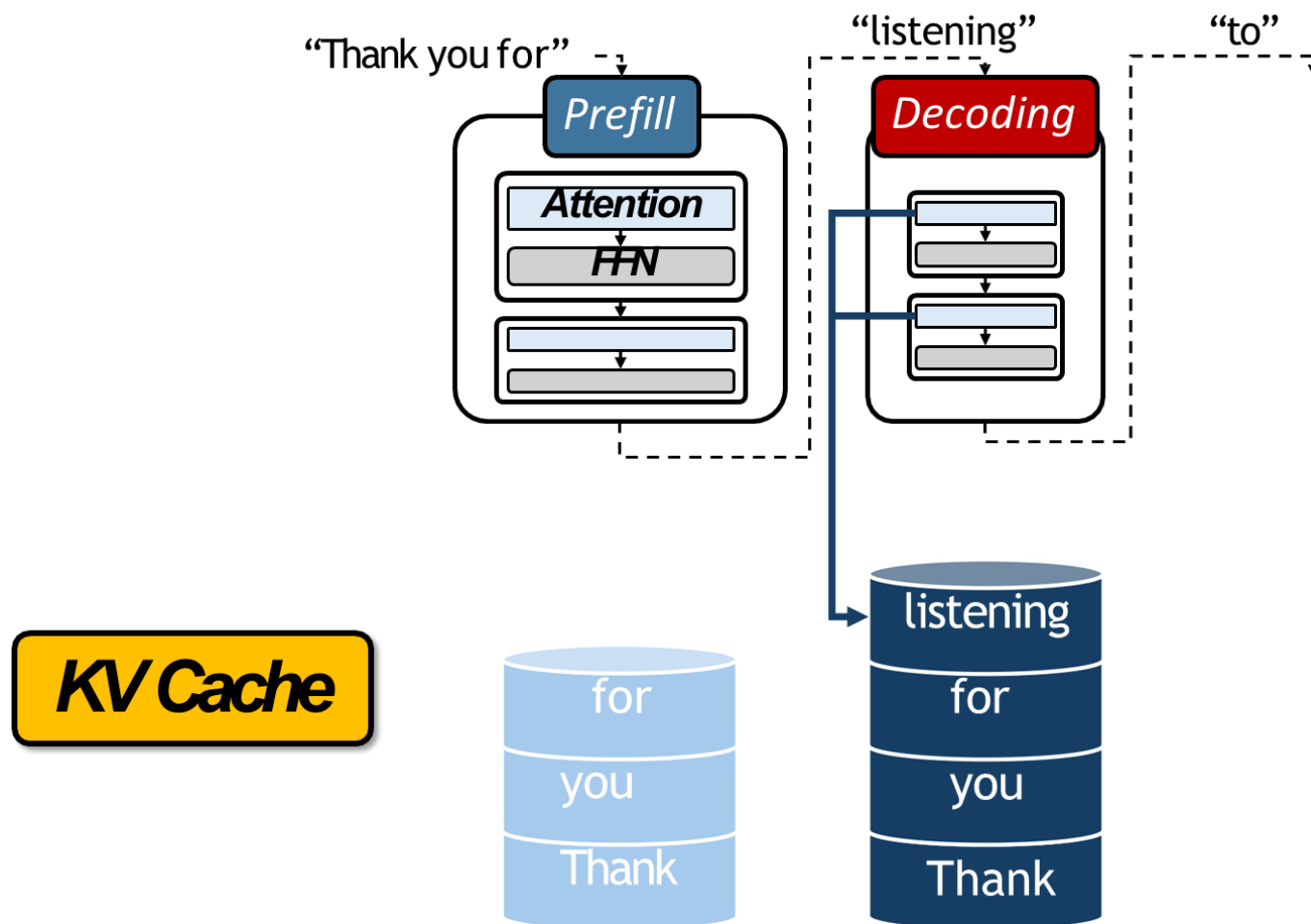
模型推理为什么需要存储



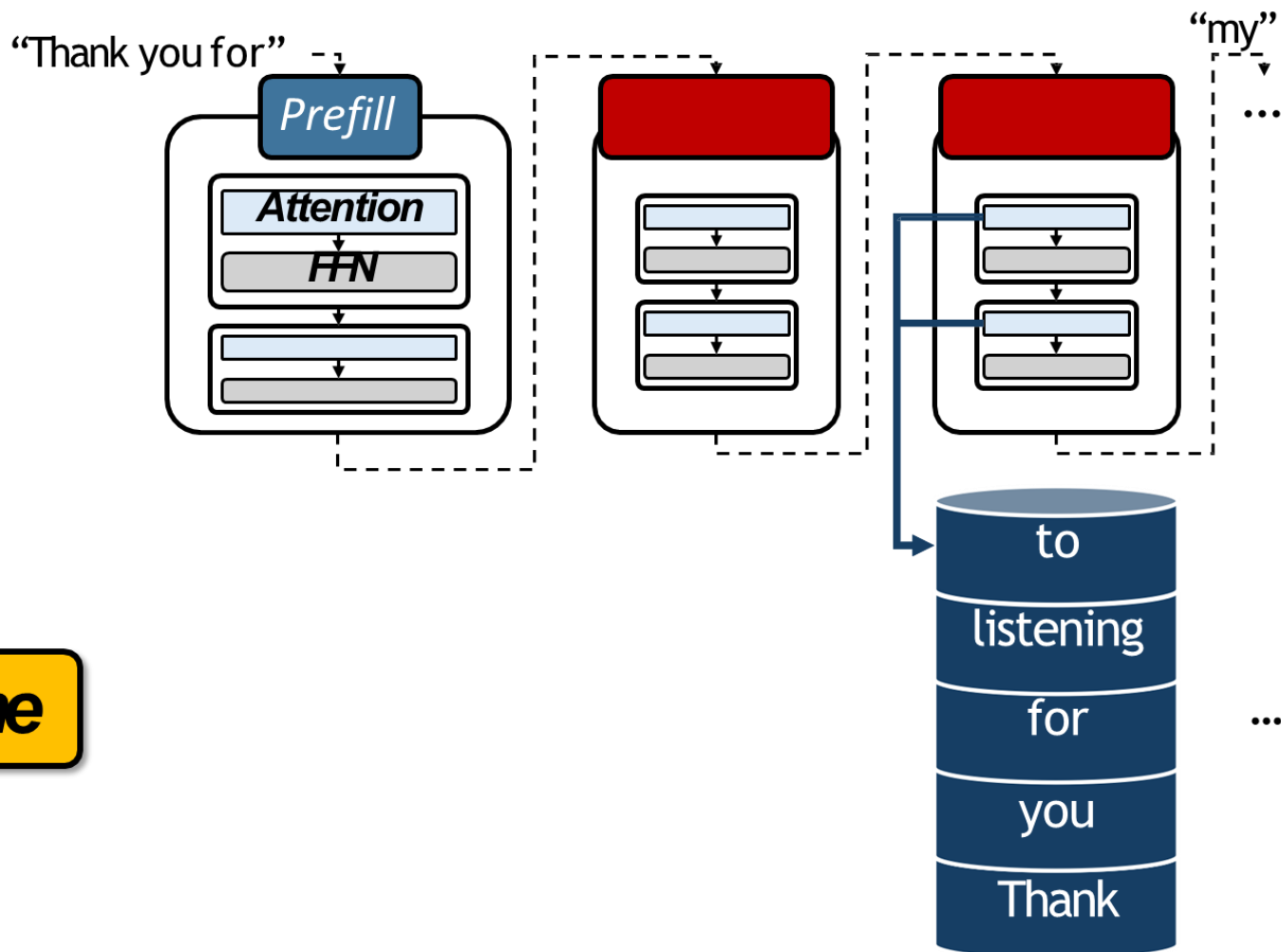
模型推理为什么需要存储



模型推理为什么需要存储



模型推理为什么需要存储

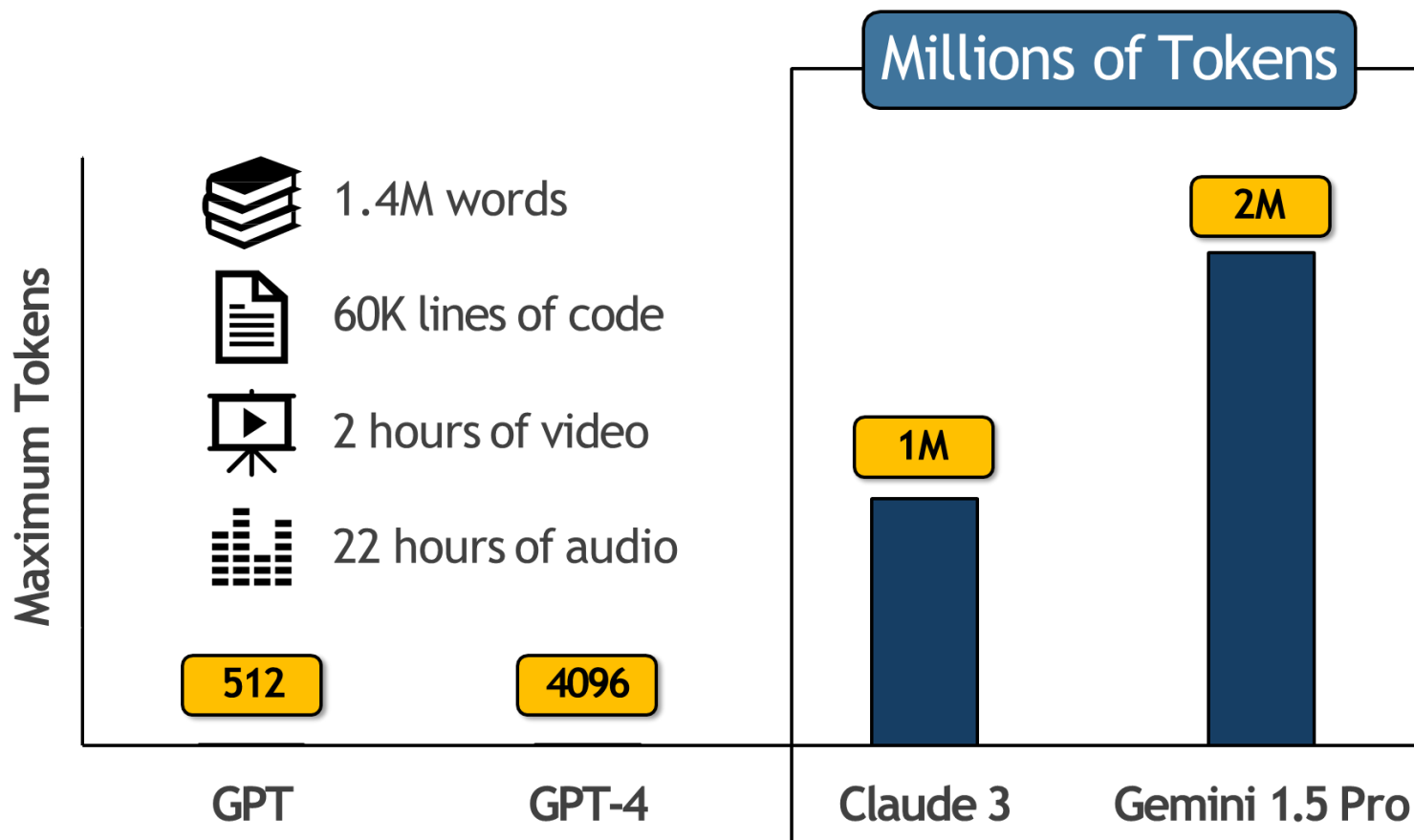


KV Cache

模型推理为什么需要存储



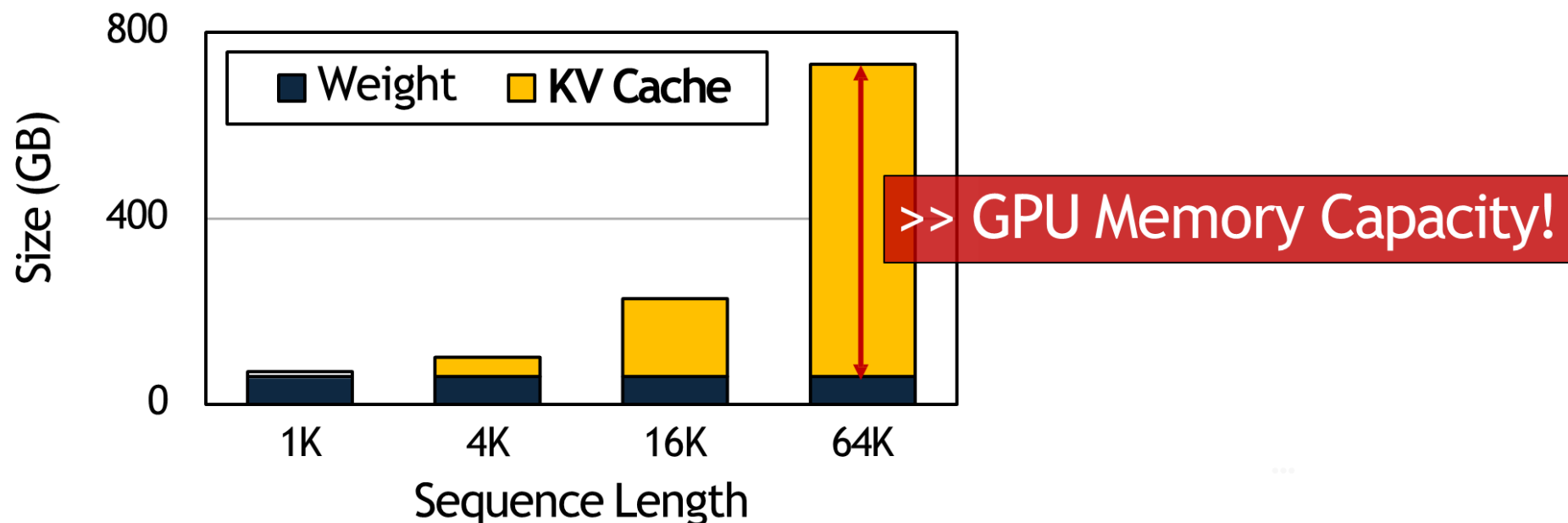
- 模型可处理的token数量不断增长



模型推理为什么需要存储



- KV cache需要的存储空间巨大
- 在agent时代，这一趋势会更加明显



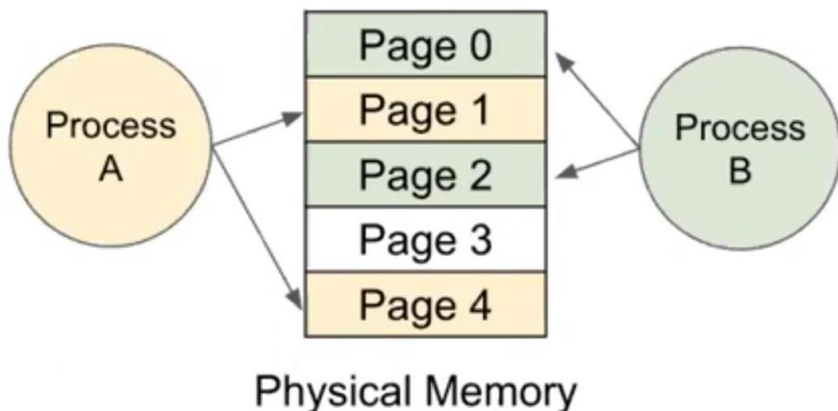
模型推理为什么需要存储



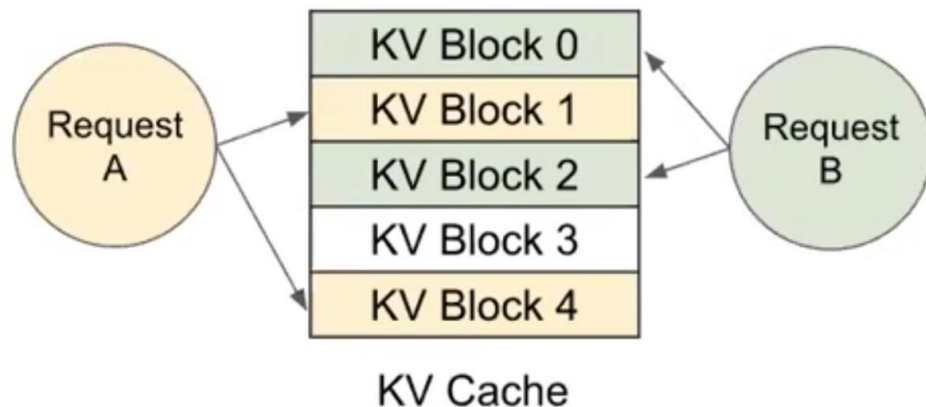
PagedAttention (vLLM)

仿照操作系统的内存页，管理LLM的内存，避免内存碎片

Memory management in OS



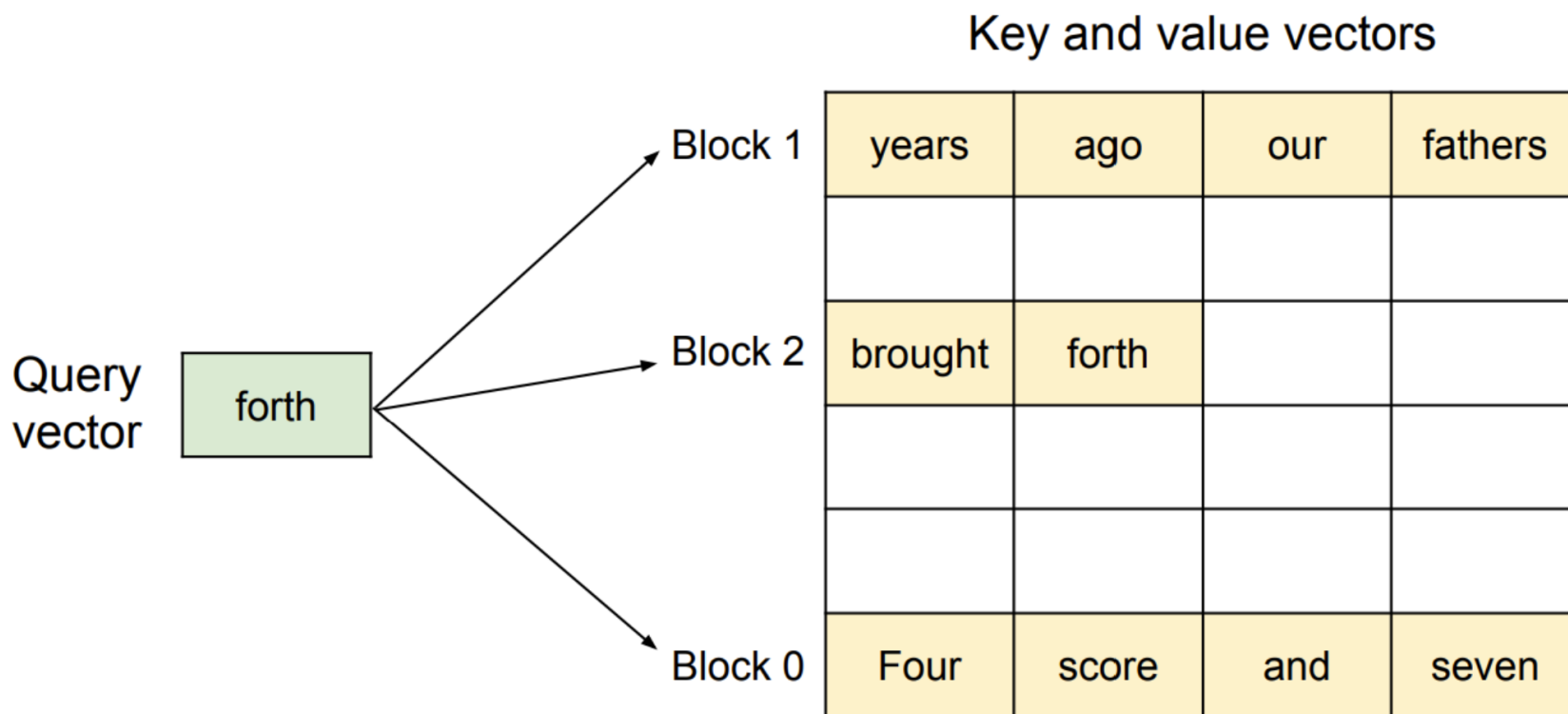
Memory management in vLLM



模型推理为什么需要存储



PagedAttention (vLLM) 仿照操作系统的内存页，管理LLM的内存



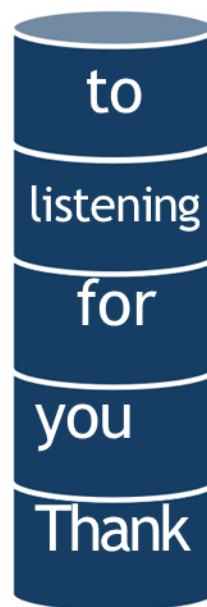
Example sequence: “Four score and seven years ago our | fathers brought forth”



缩减KV cache存储压力的方法



1. 量化 压缩KV cache, 转化成低bit的数据格式



有什么问题?

缩减KV cache存储压力的方法



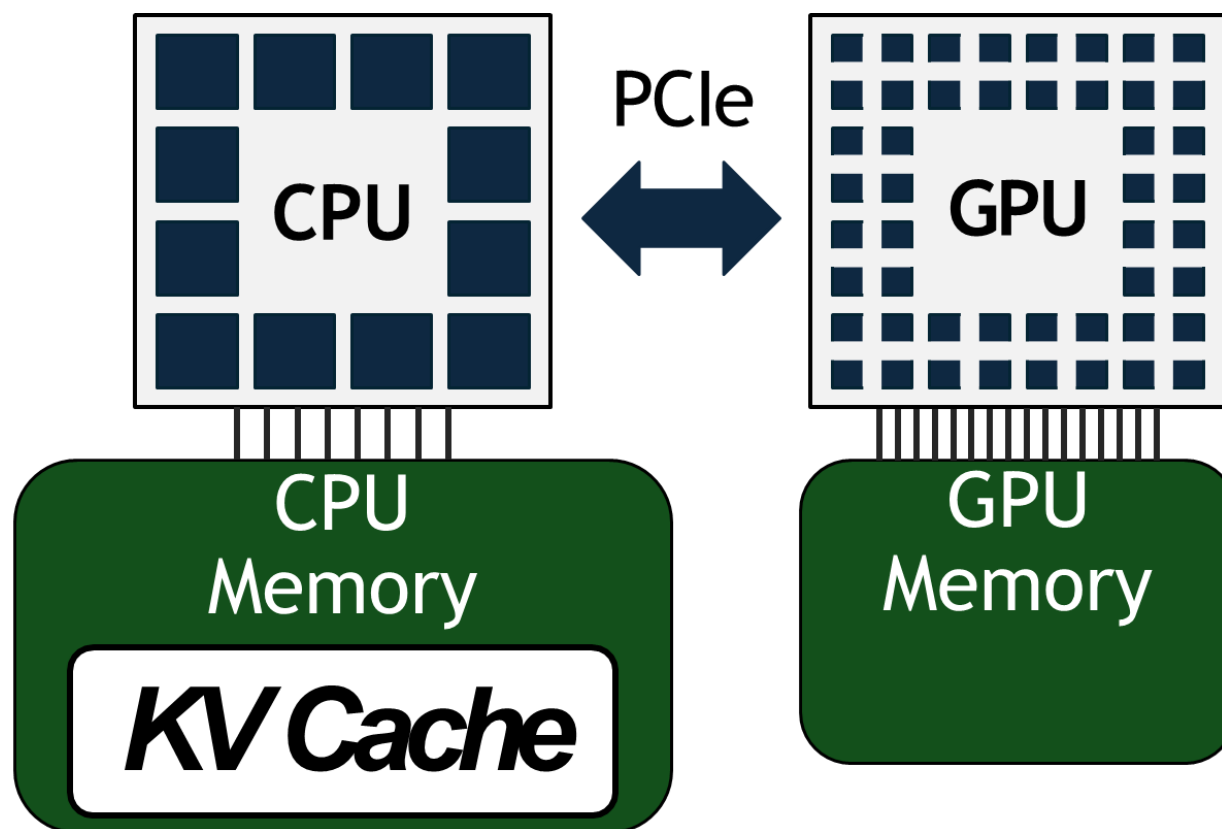
2. 驱逐

永久删除一些token的KV，保证KV cache可以放入显存

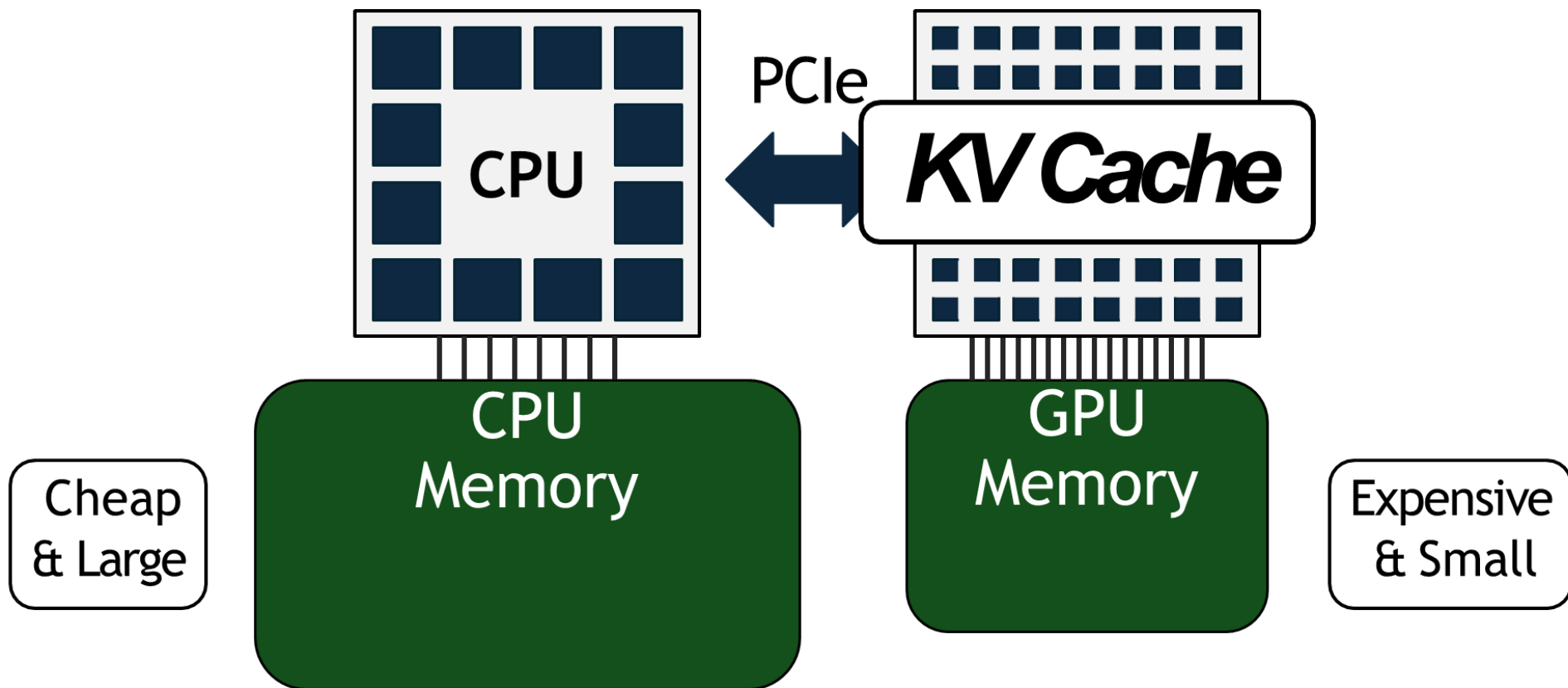


有什么问题？

缩减KV cache存储压力的方法



缩减KV cache存储压力的方法



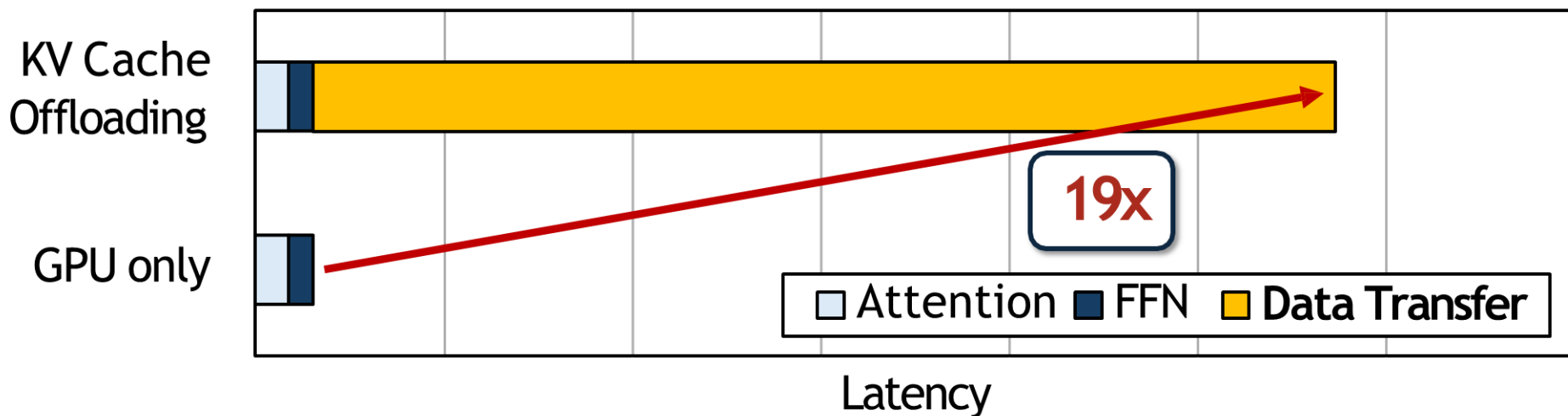
infiniGen: KV cache卸载到CPU内存



受限于PCIe带宽，卸载KV cache会带来严重的推理延迟增长

怎么办？

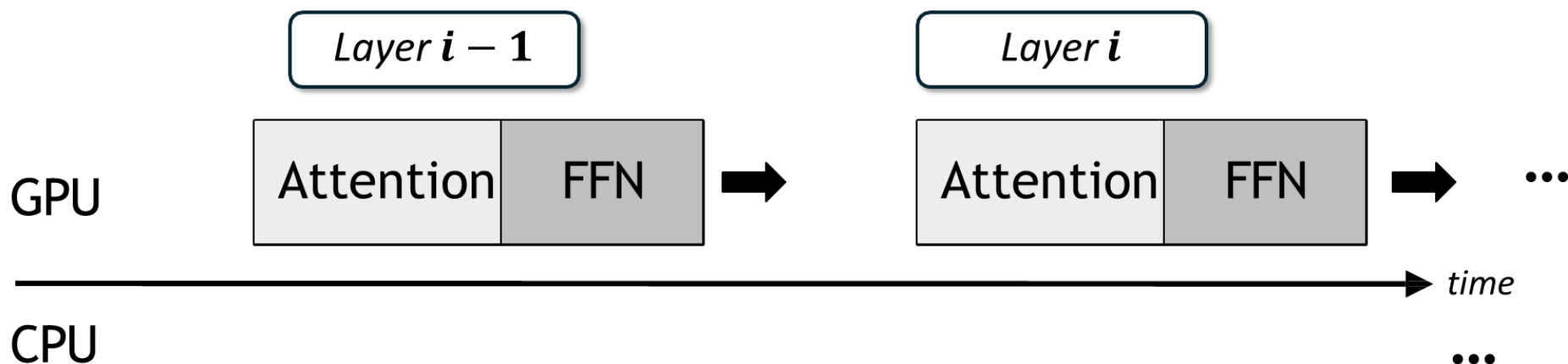
- 传输量减少
- 提前传输



infiniGen: KV cache卸载到CPU内存



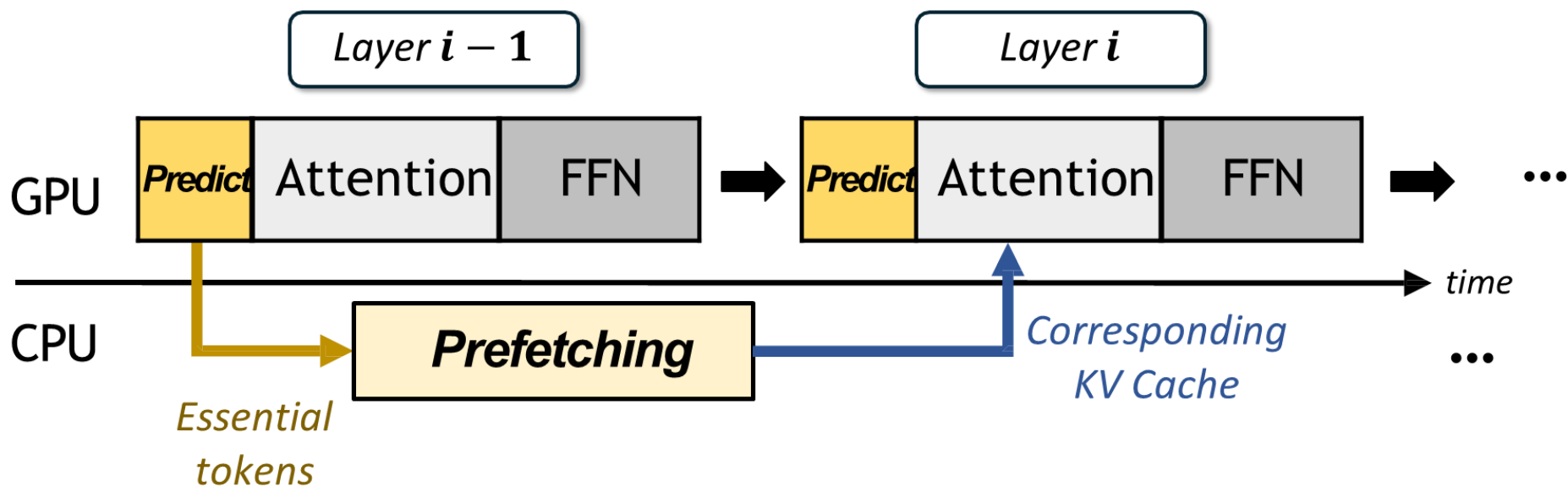
提前传输



infiniGen: KV cache卸载到CPU内存



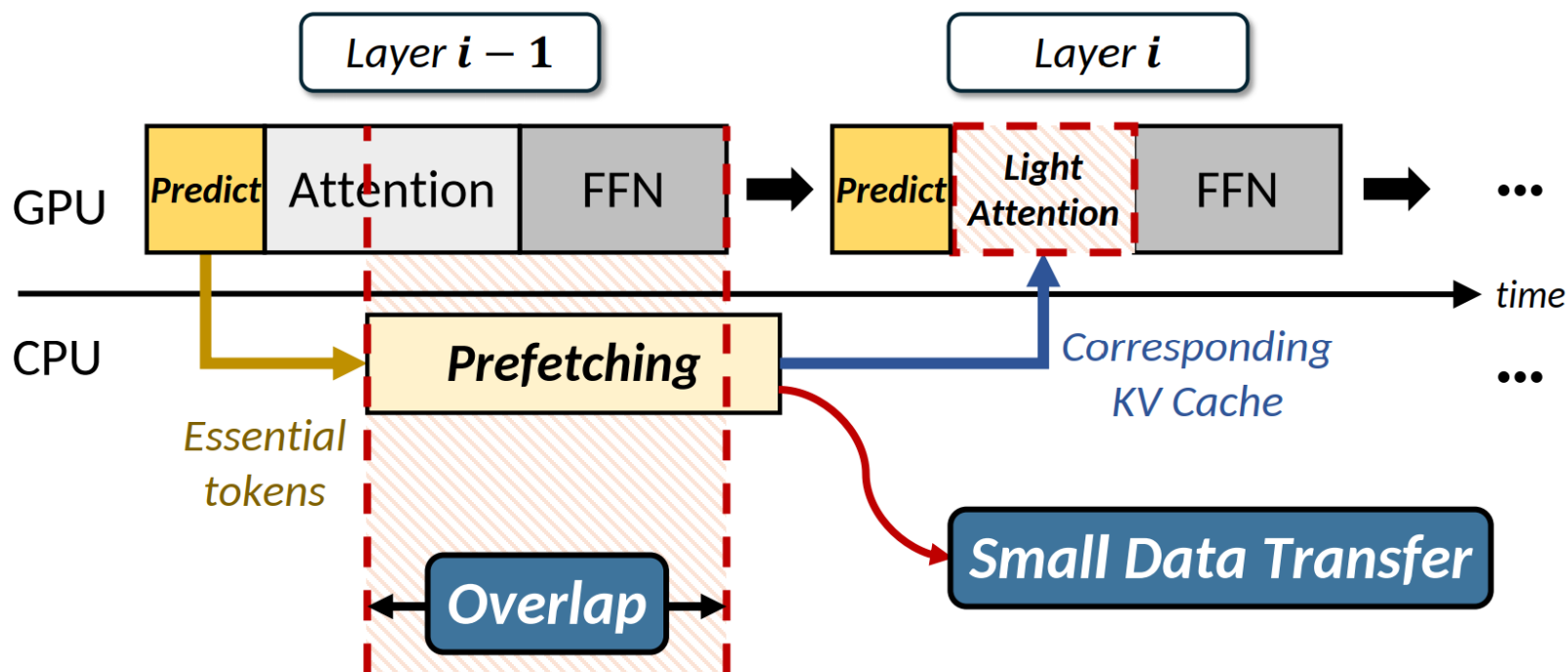
提前传输



infiniGen: KV cache卸载到CPU内存



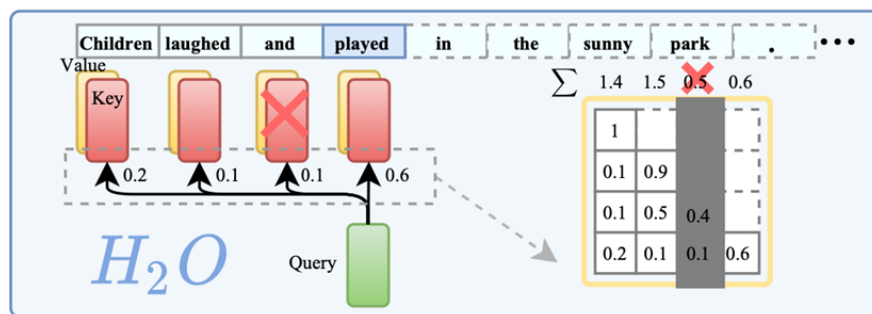
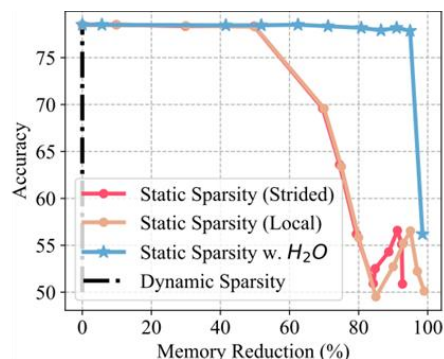
- 传的少
- 用奇异值分解 (SVD) 分析KV矩阵, 使少数列权重更大, 让模型计算注意力分数时聚焦关键列信息, 增强对重要token的预测能力。



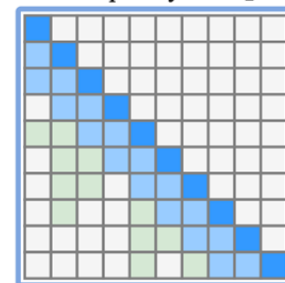
H2O: KV cache是稀疏化的



- 少数的重要KV就可以维持预测精度
- 在GPU中保留重要的KV



Static Sparsity w. H_2O



仅保存20% 的KV cache 时
仍然可以保持很高的精度

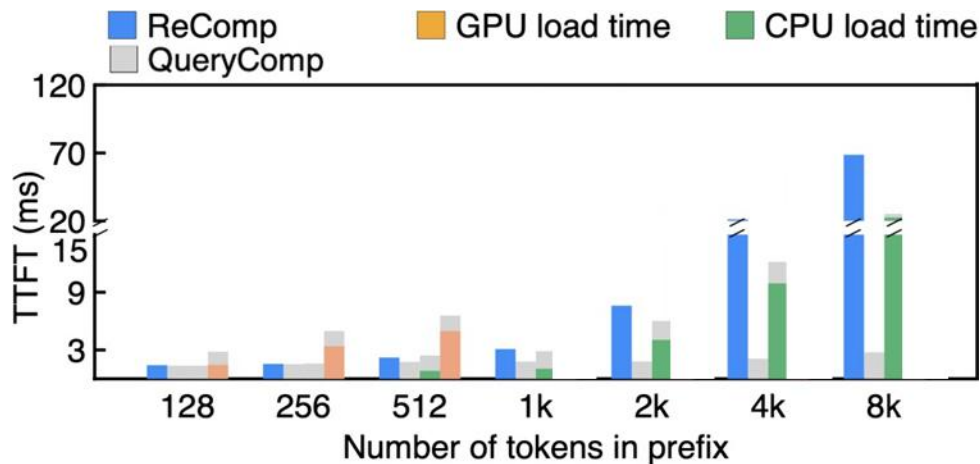
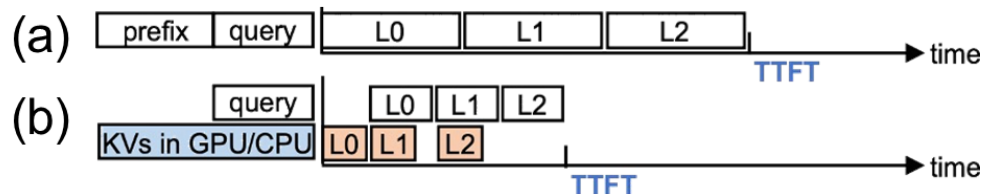
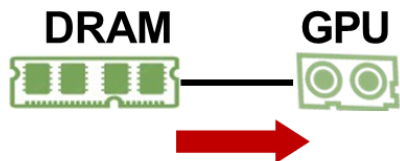
动态选择对结果影响微
弱的KV cache

保存部分
KV cache

IMPress: KV cache卸载到SSD



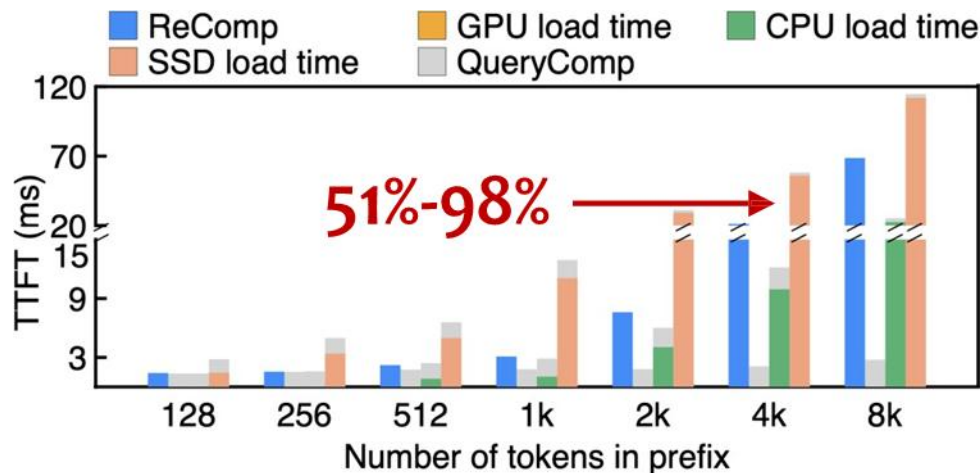
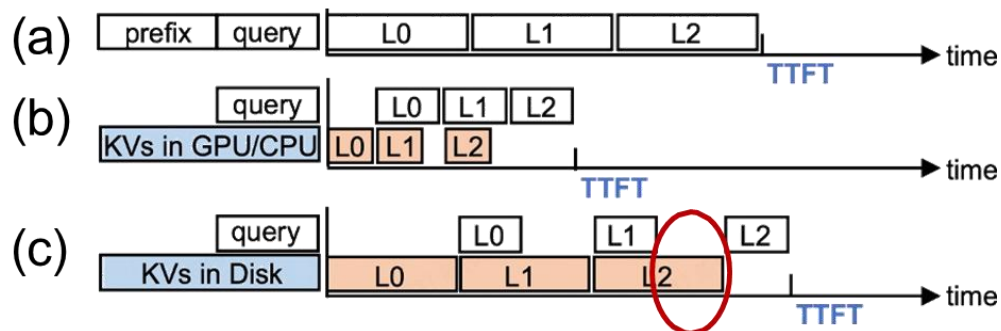
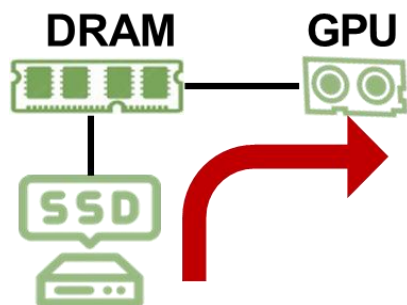
- 使用内存卸载KV cache是有效的
- 进一步增大KV cache的容量
- 使用廉价、容量巨大的SSD



IMPress: KV cache卸载到SSD



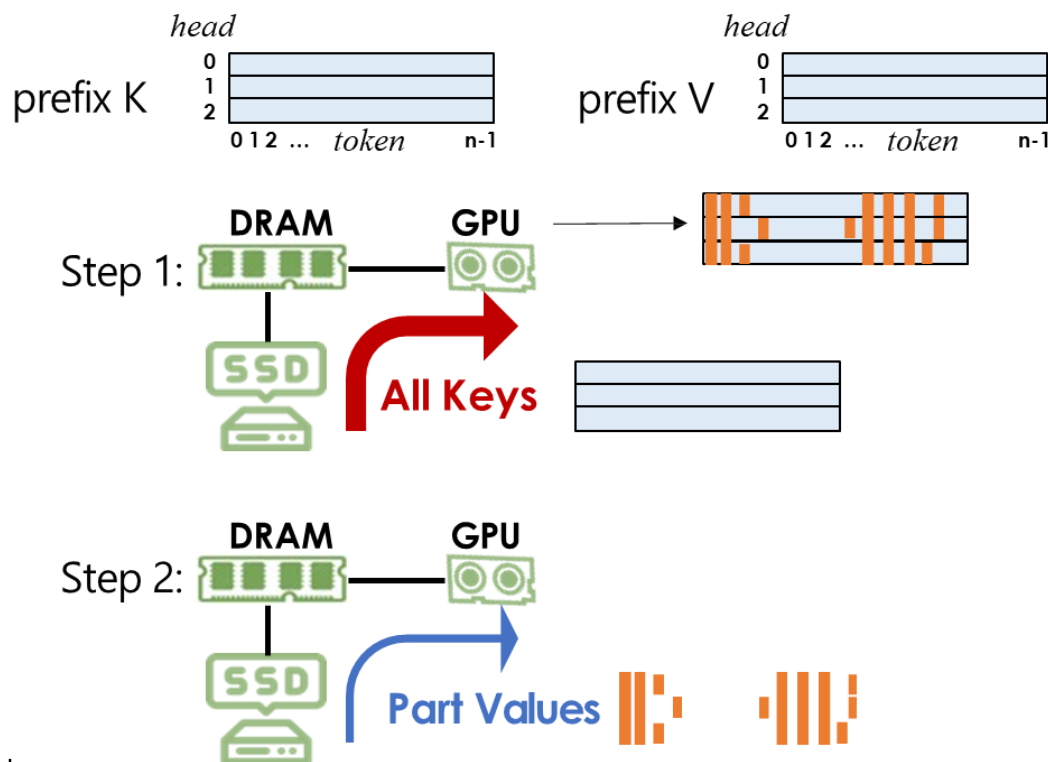
- SSD太慢，直接使用会严重降低性能



IMPress: KV cache卸载到SSD



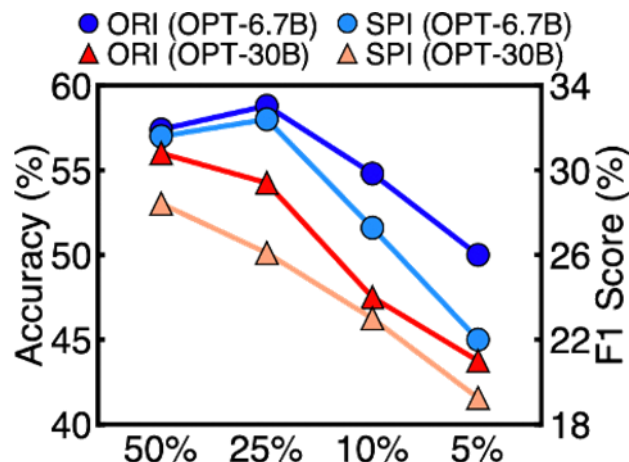
- 减少传输数据量，就可以缓解I/O瓶颈
- 机会：KV cache具有稀疏性
- 使用key计算注意力分数，识别出重要的token



IMPress: KV cache卸载到SSD



- 将key传输到GPU还是需要大量的I/O
- 离线静态的预定义一些KV是重要的?
- 精度严重下降

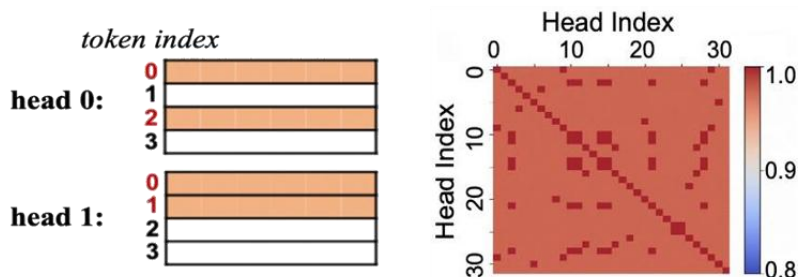


- SPI: statically pre-determine importance
- ORI: original dynamically determine importance

IMPress: KV cache卸载到SSD

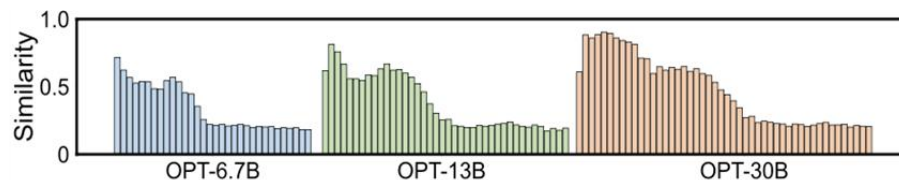


- 多头注意力中，不同head之间的重要token是相似的
- 只计算head0的重要KV，就可以猜测head1的重要KV

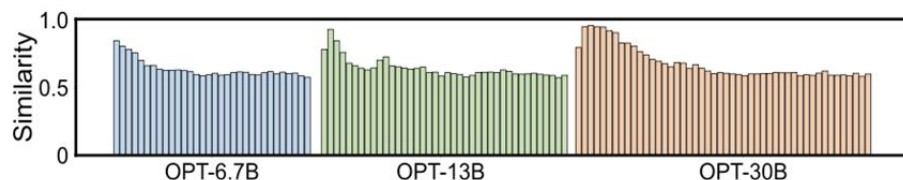


Similarity measurement:

$$h_0=\{0, 2\} \quad h_1=\{0, 1\} \quad J(h_0, h_1) = \frac{|h_0 \cap h_1|}{|h_0 \cup h_1|} = \frac{1}{3}$$



(a) select the top 10% most important tokens

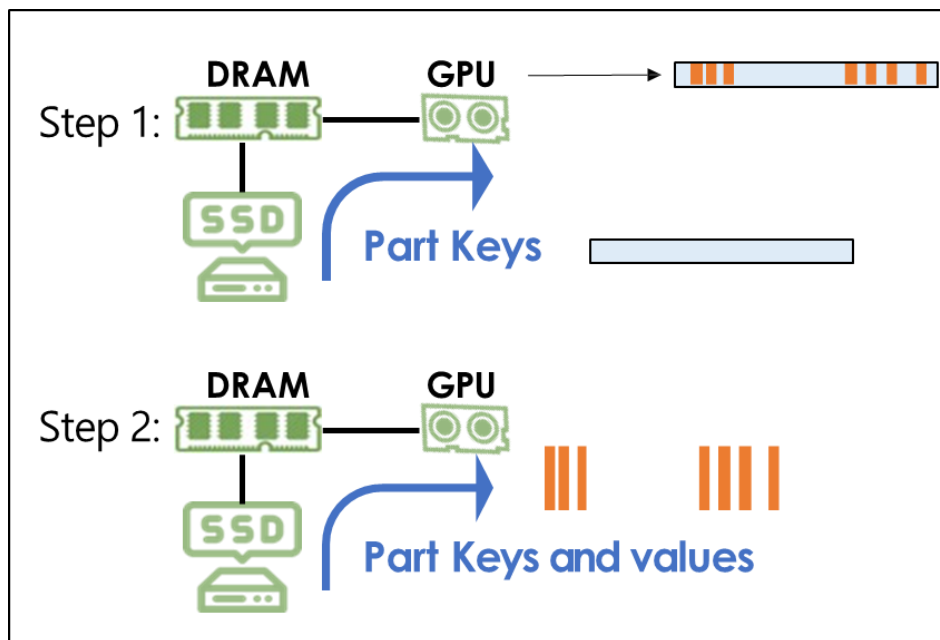


(b) select the top 40% most important tokens

IMPress: KV cache卸载到SSD



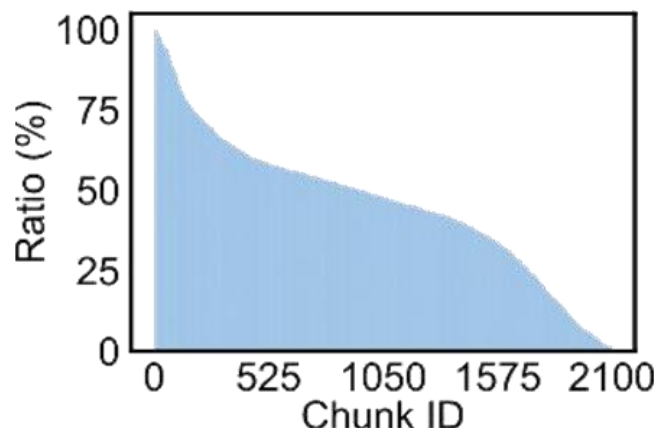
- 传输少数head的key用于重要性计算
- 根据计算结果，筛选出其他head的重要KV
- 传输量降低



IMPress: KV cache卸载到SSD



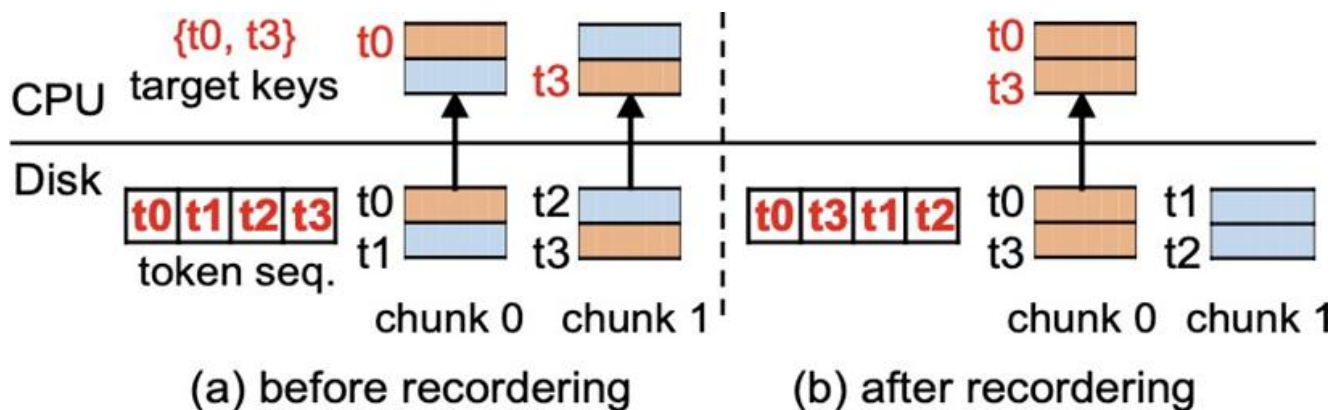
- 存储的读放大问题，按照chunk读取，但是chunk中包含的重要KV数量各不相同
- 额外I/O一些不重要的KV



IMPress: KV cache卸载到SSD



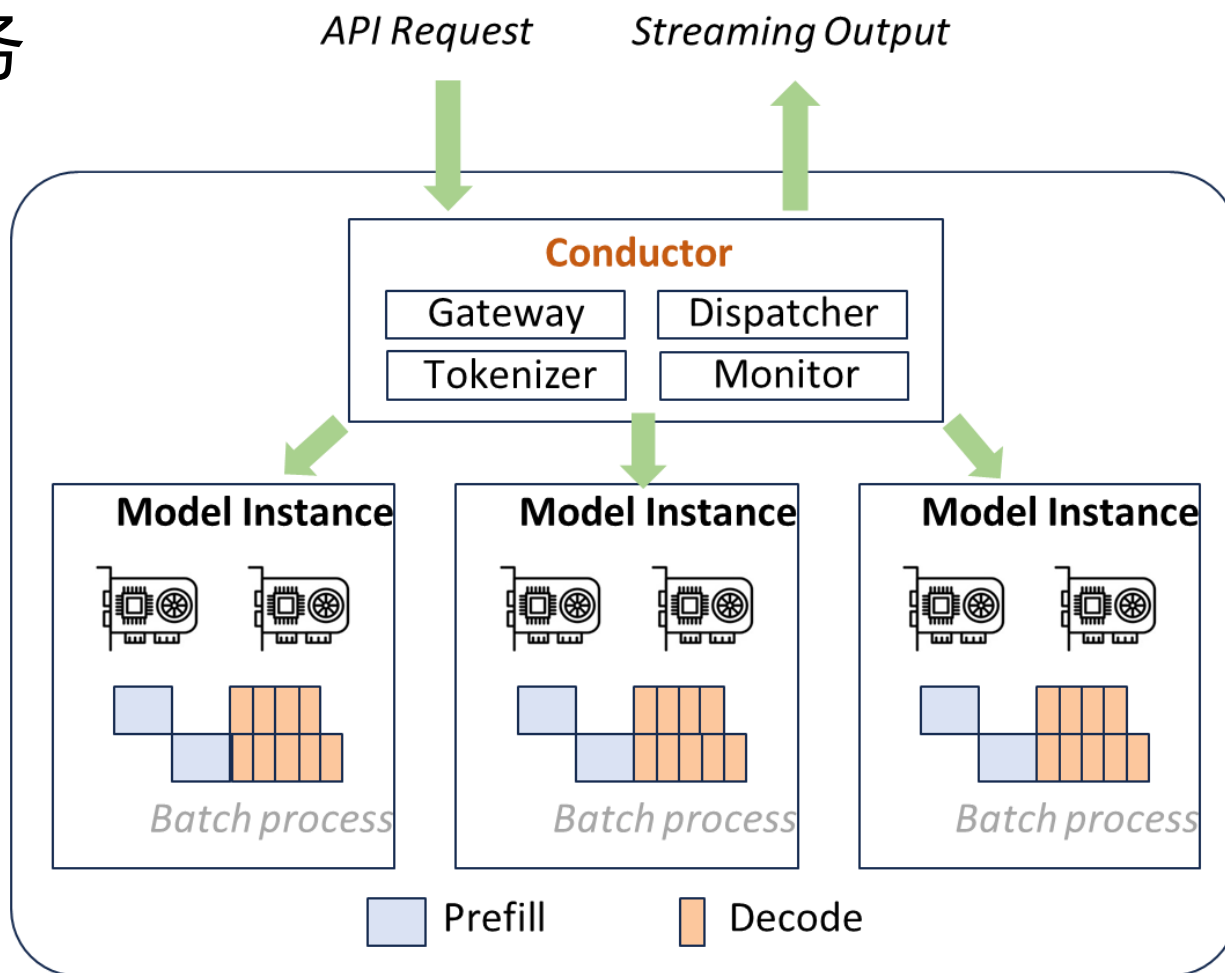
- 把重要的KV尽量放在相同的chunk中
- KV重排



Mooncake: KV cache为中心的推理系统



在线大模型服务



Mooncake: KV cache为中心的推理系统

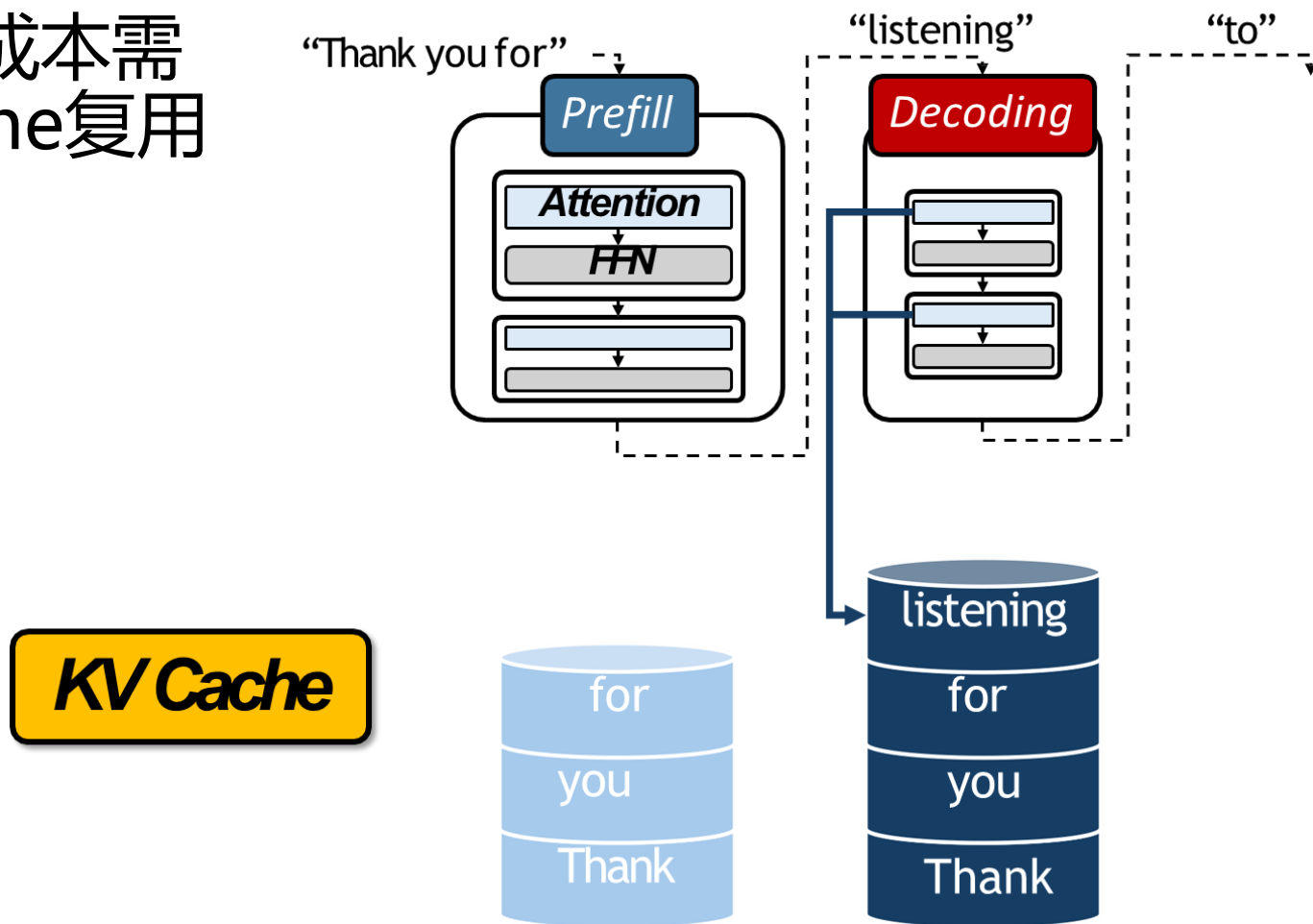


- Kimi 
 - 每天处理1000亿的token
 - 需要实时相应
 - 最长可以支持1百万长度的上下文
-
- 智能=更多的数据+更大的模型+更长上下文

Mooncake: KV cache为中心的推理系统



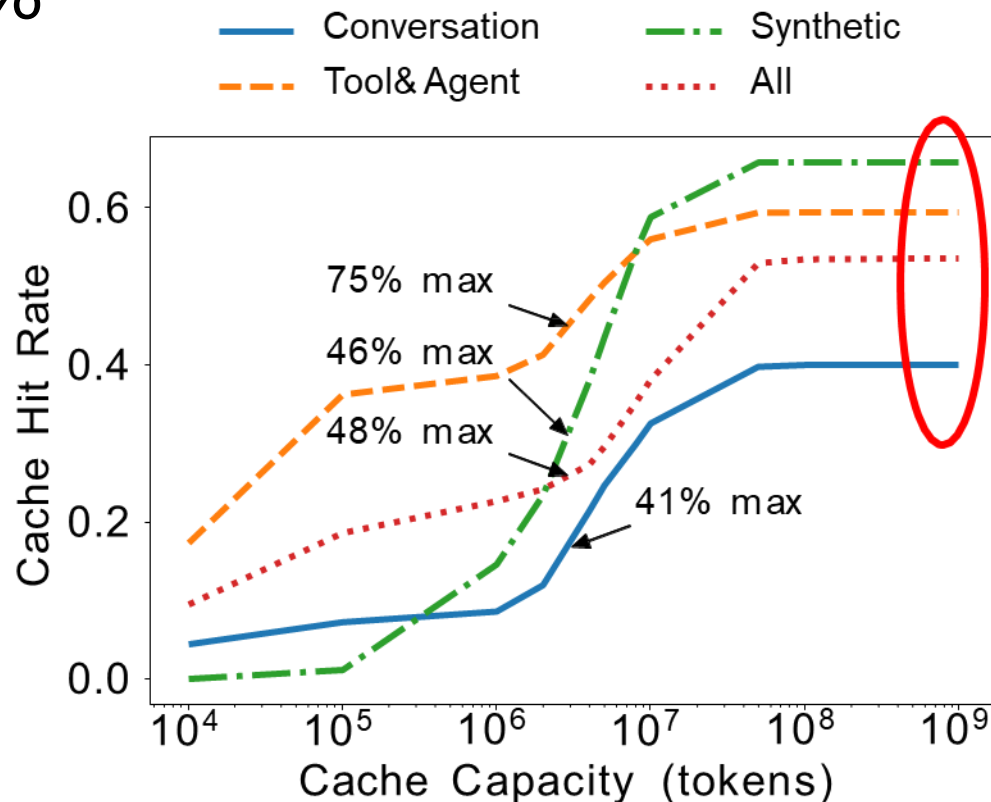
降低服务成本需要KV cache复用



Mooncake: KV cache为中心的推理系统



实际负载中，超过50%
的KV是会被复用的



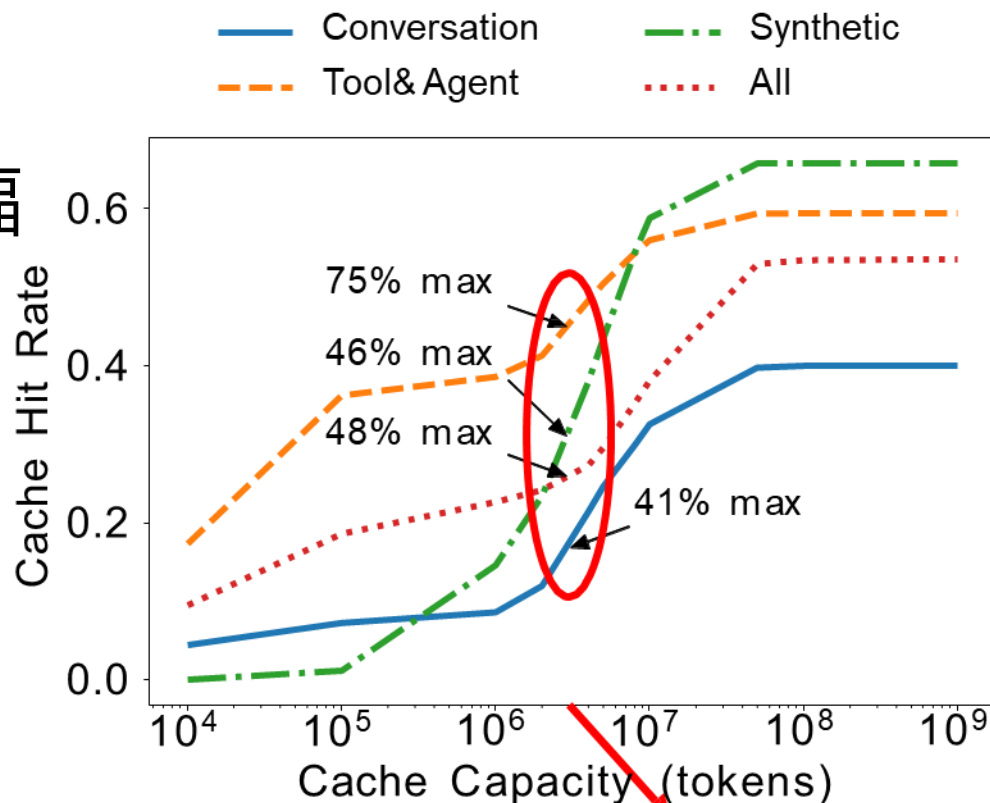
Traces in Mooncake paper

(open-sourced in <https://github.com/kvcache-ai/Mooncake>)

Mooncake: KV cache为中心的推理系统



如果仅使用服务器本地内存，
cache命中率大幅下降

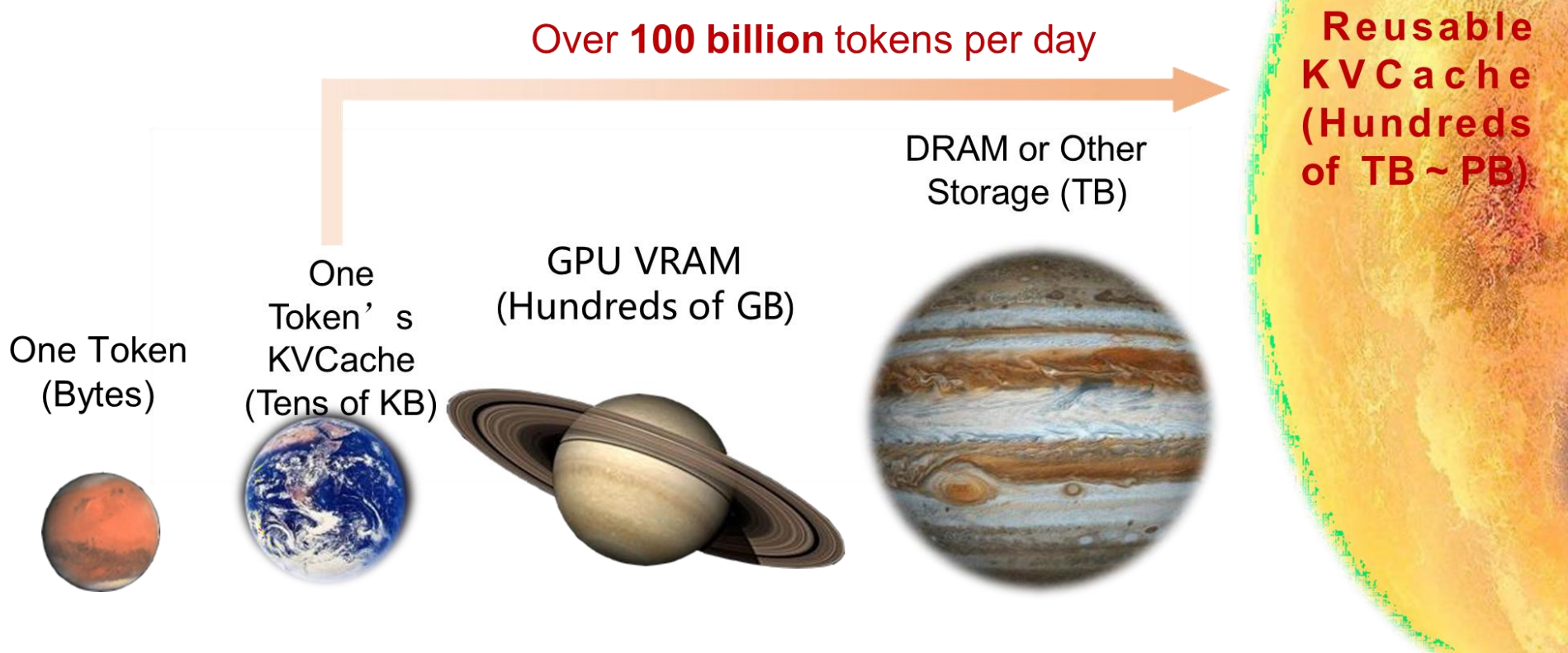


Traces in Mooncake paper

(open-sourced in <https://github.com/kvcache-ai/Mooncake>)

local capacity (~3M tokens)

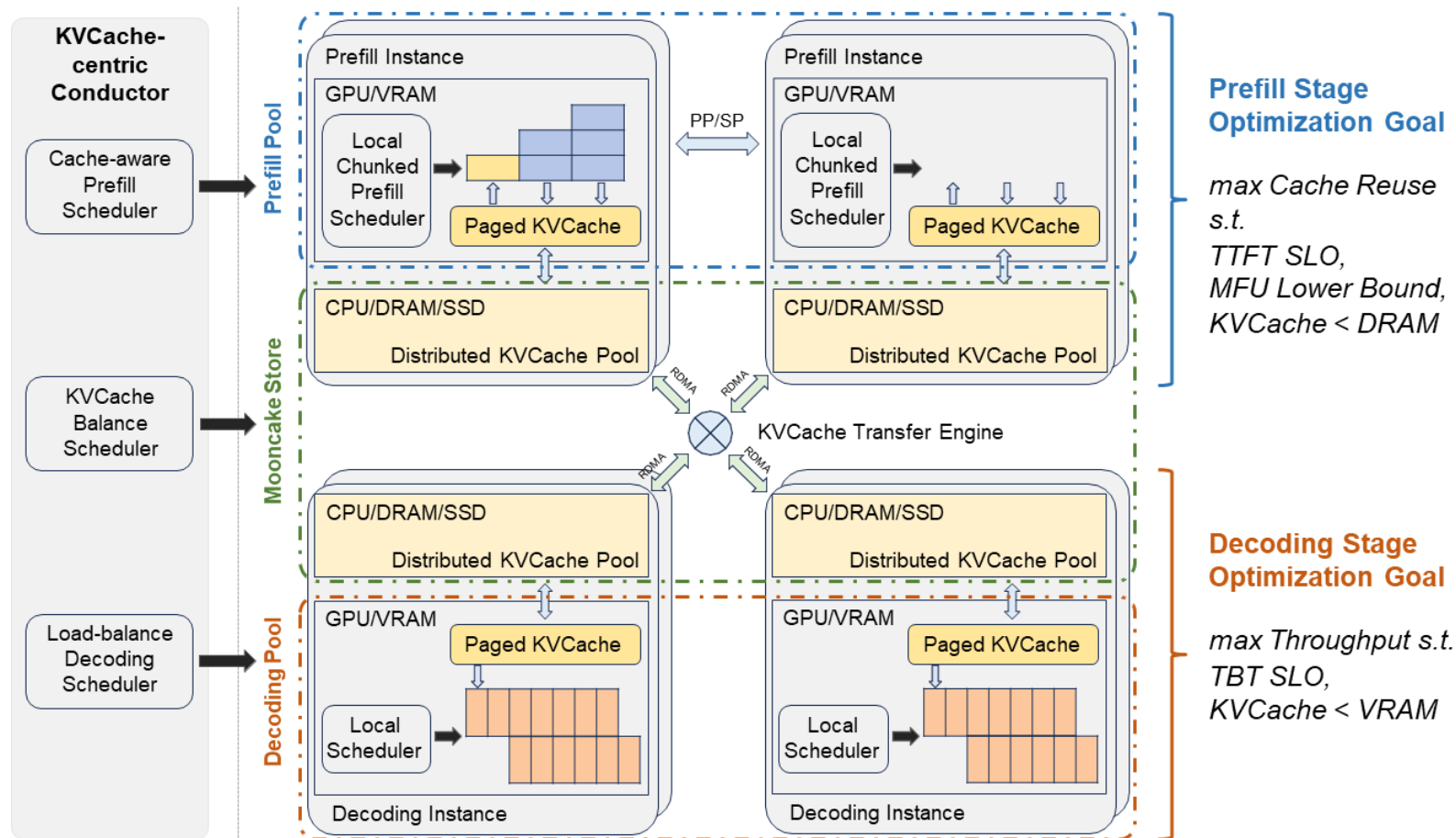
Mooncake: KV cache为中心的推理系统



Mooncake: KV cache为中心的推理系统



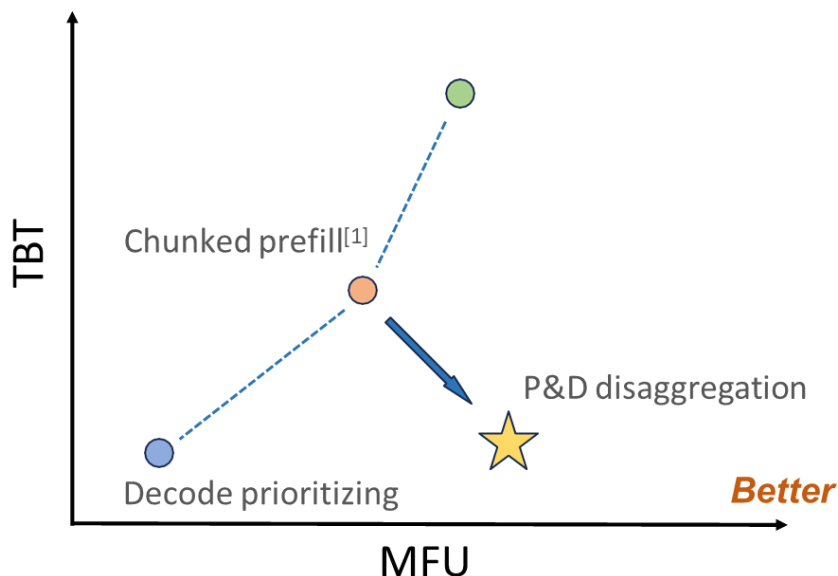
Mooncake的整体架构



Mooncake: KV cache为中心的推理系统



- PD分离的推理引擎
- Prefill: 计算密集型 Decode: 访存密集型
- 避免混合批处理中预填充和解码之间的干扰
- 解耦二者以提高 MFU (模型浮点运算利用率)

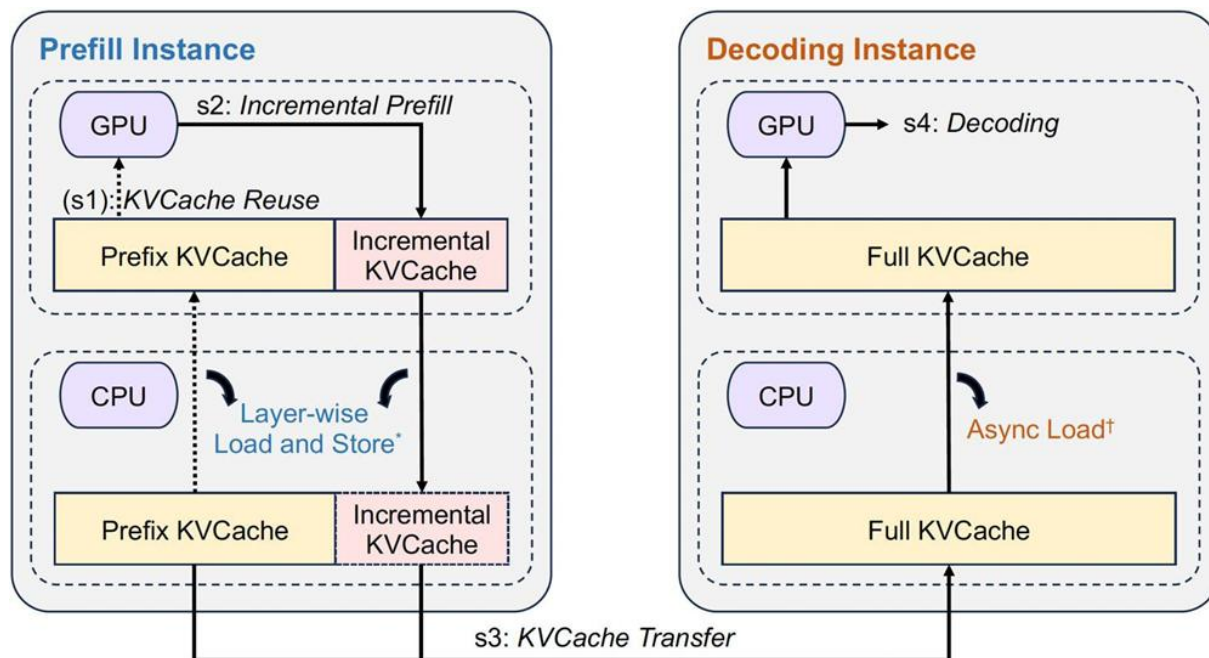


[1]Agrawal et al. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve (OSDI'24)

Mooncake: KV cache为中心的推理系统



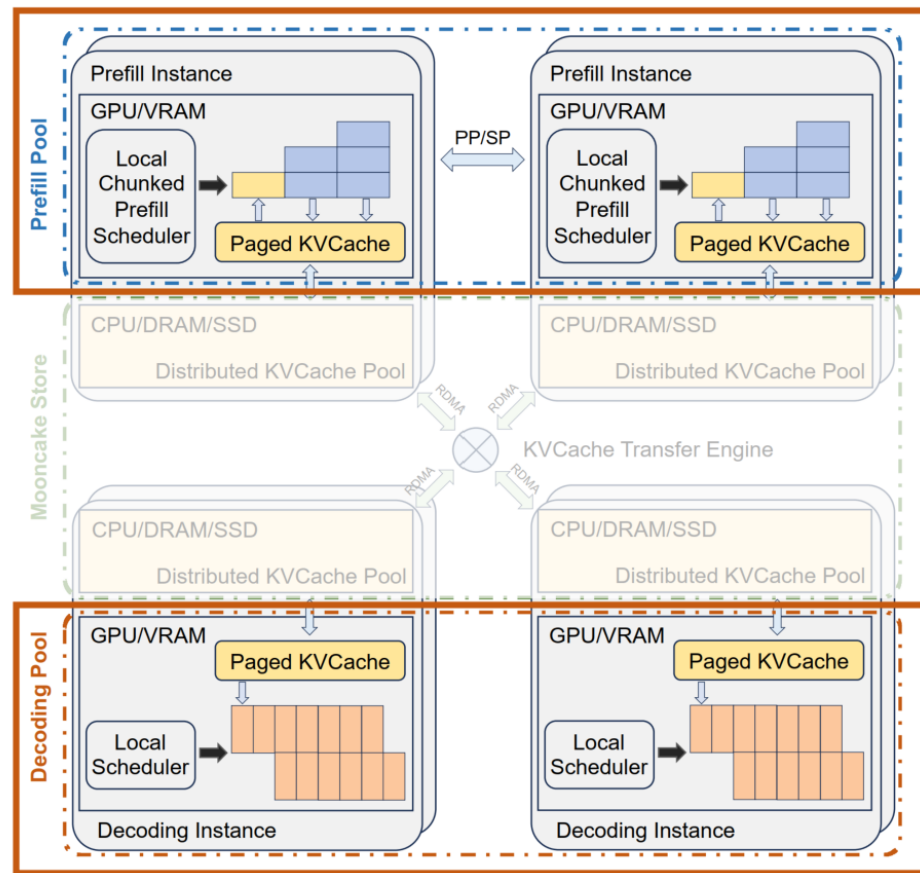
- PD分离的推理引擎
- Prefill: 计算密集型 Decode: 访存密集型
- 避免混合批处理中预填充和解码之间的干扰
- 解耦二者以提高 MFU (模型浮点运算利用率)



Mooncake: KV cache为中心的推理系统



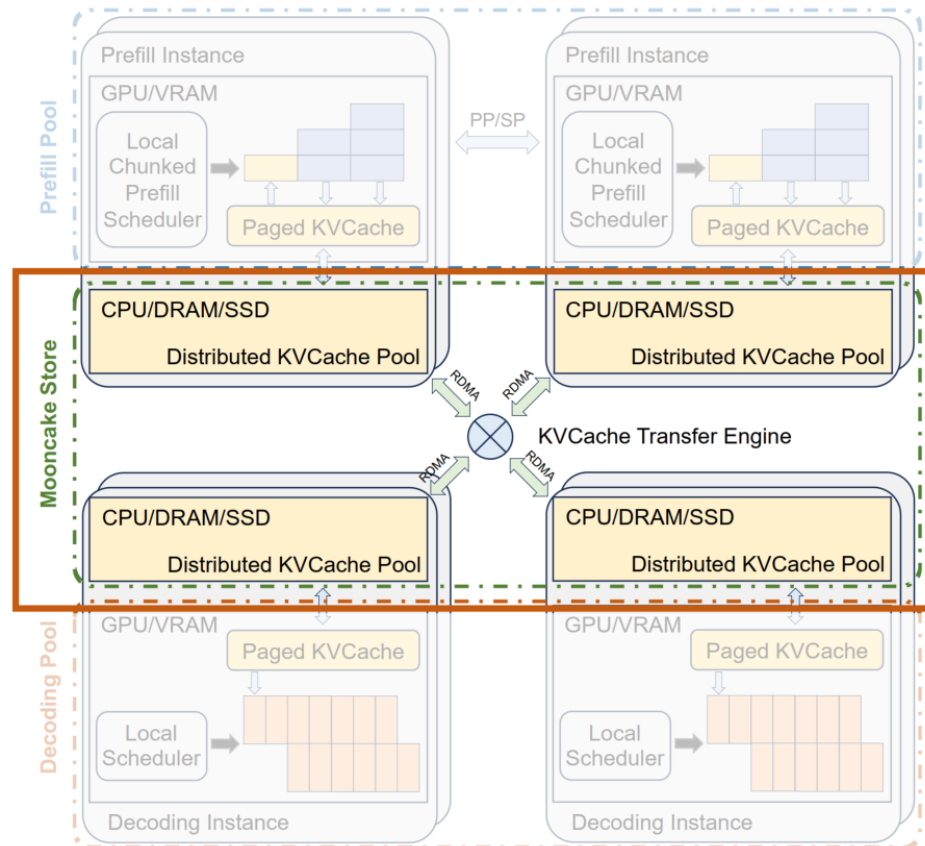
- 分布式多层KV cache pool
- 将GPU 节点重组为多个“解耦资源池”。
- “计算密集型资源池”
(侧重 GPU 算力, 专门适配 prefill 阶段)
- “内存密集型资源池”
(侧重 VRAM/DRAM 容量与访问速度, 专门适配 decoding 阶段)



Mooncake: KV cache为中心的推理系统



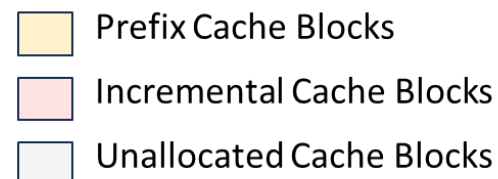
- 分布式多层KV cache pool
- 利用闲置CPU/DRAM/SSD, 专门存储可复用的 KVCache, 构成KVCache 存储池
- 使用高速RDMA网络互联



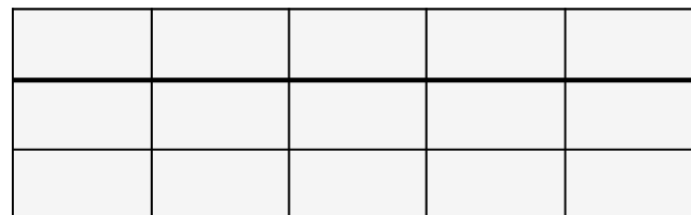
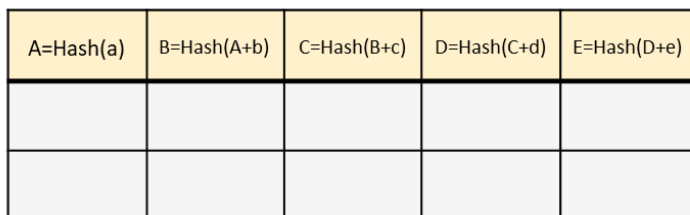
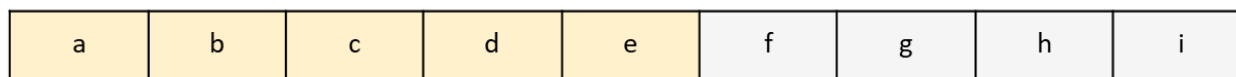
Mooncake: KV cache为中心的推理系统



- 推理流程



Token Blocks
(16~512 tokens)



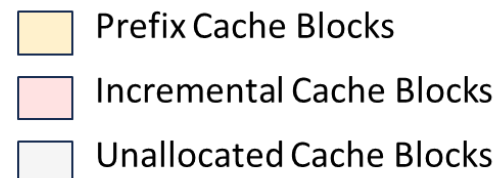
Prefill
Instance

Decoding
Instance

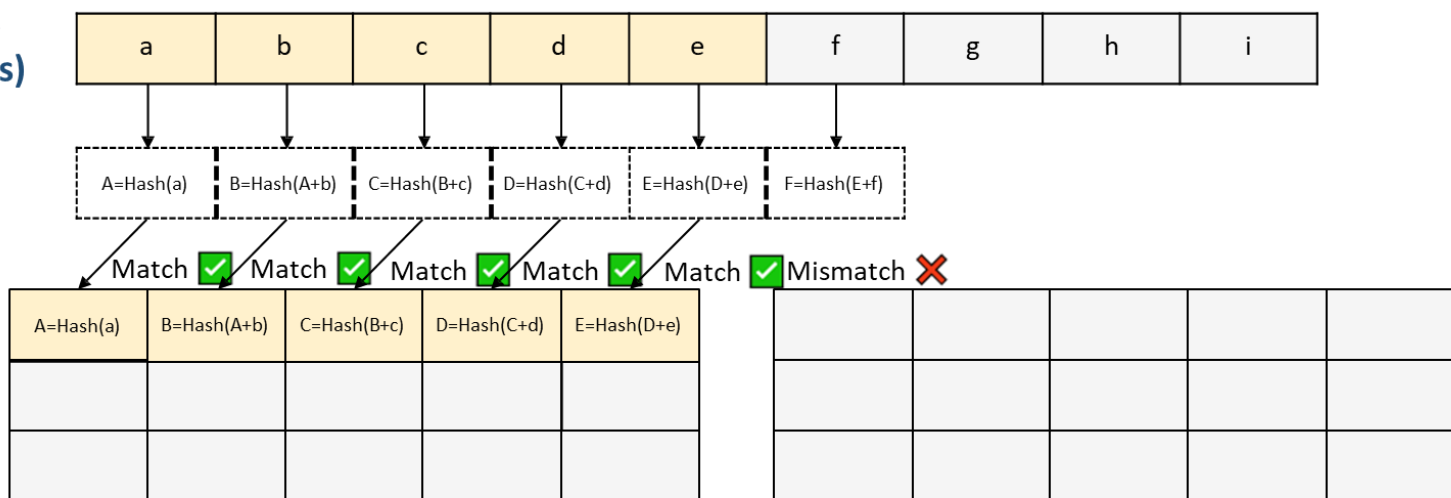
Mooncake: KV cache为中心的推理系统



- 推理流程



Token Blocks
(16~512 tokens)



Prefill
Instance

Decoding
Instance

- Prefix Cache Blocks
- Incremental Cache Blocks
- Unallocated Cache Blocks

The diagram illustrates the interaction between a Prefill Instance and a Decoding Instance. The Prefill Instance takes input tokens (a-i) and outputs hashes (A-F). The Decoding Instance then processes these hashes and subsequent tokens (g-i) to produce further hashes (G-I).

Input Tokens: a, b, c, d, e, f, g, h, i

Hashes (A-F): A=Hash(a), B=Hash(A+b), C=Hash(B+c), D=Hash(C+d), E=Hash(D+e), F=Hash(E+f)

Match Results: Match ✓, Match ✓, Match ✓, Match ✓, Match ✓, Mismatch ✗

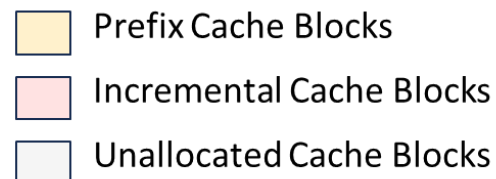
Decoding Instance Output: G=Hash(F+g), H=Hash(G+h), I=Hash(H+i)

Operations: Put(), Get()

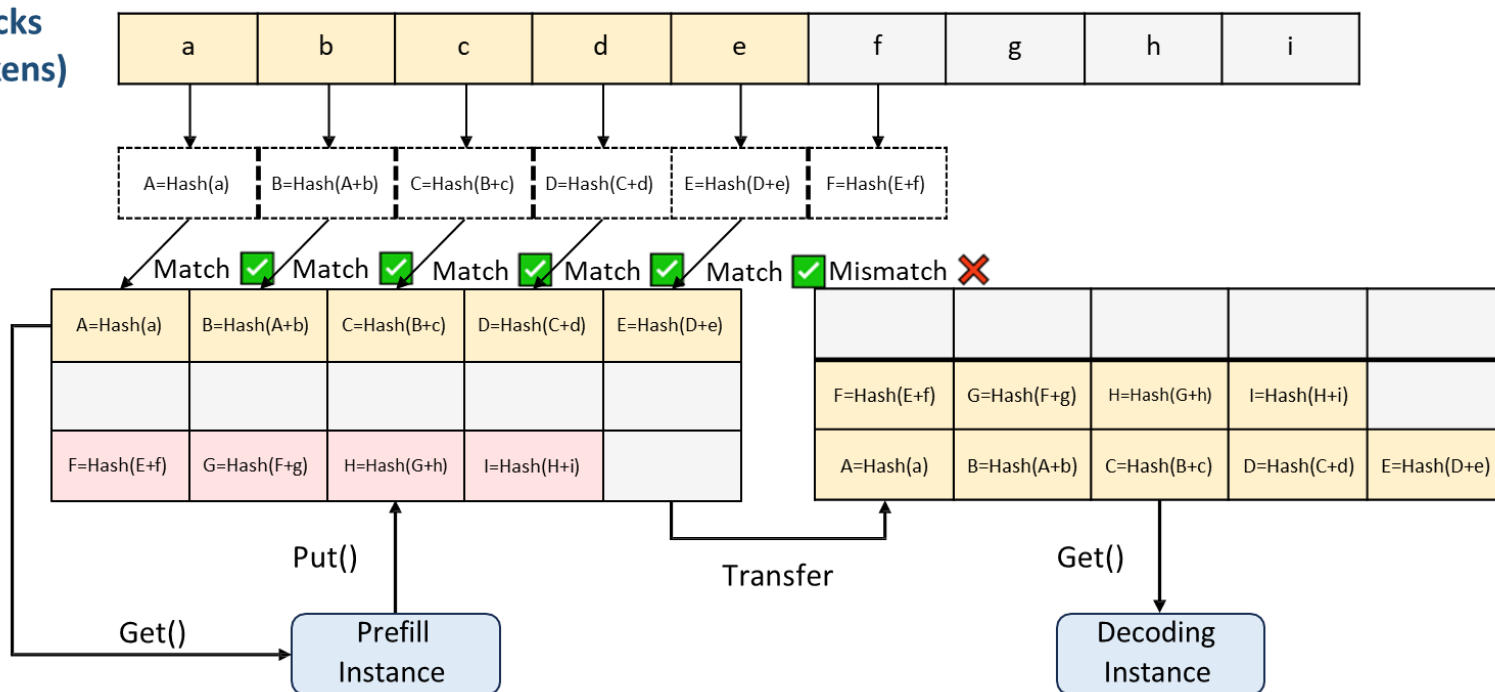
Mooncake: KV cache为中心的推理系统



- 推理流程



Token Blocks
(16~512 tokens)



小结：模型推理中的存储系统



- 推动智能增长的因素——模型参数的不断增长、推理上下文不断变长
- 模型参数数据量大、产生的中间结果多
- 大模型参数超过GPU有限容量
 - Ktransformer 单卡推理deepseek
 - LLM in flash 端侧推理
 - Powerinfer 消费级显卡推理
- KV cache需要廉价的存储
 - infinigen KV cache卸载到CPU内存
 - Impress KV cache卸载到SSD
 - Mooncake 以KV cache为中心的分布式推理系统



目录

- 大模型训练中的存储
- 大模型推理中的存储
- 大模型指导存储系统设计

AI coding



- AI辅助编程
 - Claude code
 - Codex
 - 通义灵码
 - ...

AI能否替代开发者对系统进行调优和维护？

让AI直接生成一个完整的系统是否可以？生成后是否能够不断自我迭代？



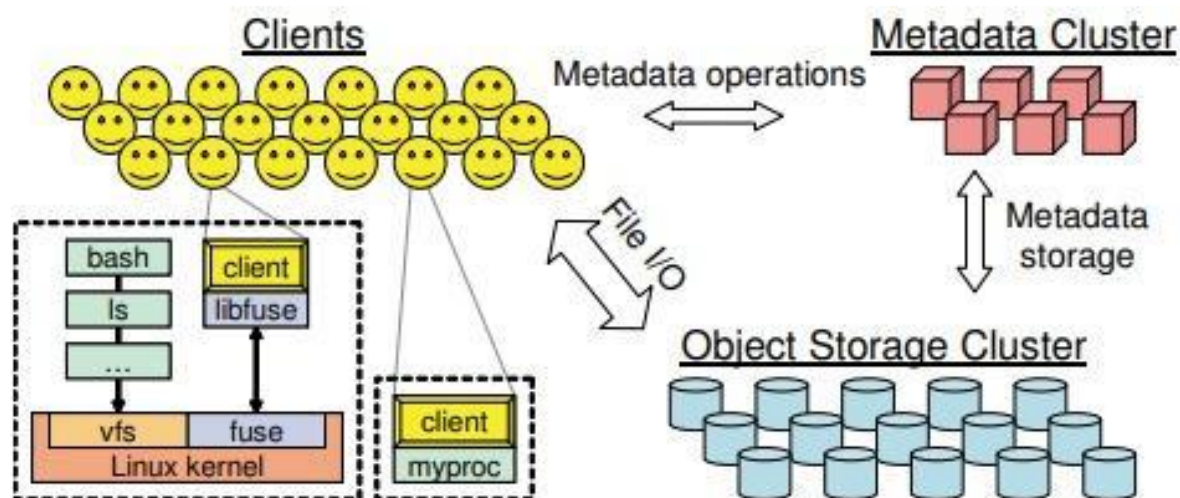
HPC 存储调优的需求

并行文件系统（Lustre/Ceph）是 HPC 核心支撑

可调参数极多：Lustre 159 个、Ceph 1536 个

I/O 性能直接决定科学计算效率

调优目标：为应用匹配最优参数配置



Ceph架构



传统调优的困境

人工调优：

效率高但极度依赖专家、成本高、难以复用

传统自动调优（启发式 / ML/RL）：

迭代次数：数百 ~ 数十万次

探索成本极高，生产环境不可用

LLM 直接使用：

存在领域幻觉、参数理解错误

使用RAG+LLM agent进行调优？

STELLAR架构



两大模块：离线参数提取 + 在线自主

调优流程：手册 RAG 解析 → I/O 日志分析 → 调优迭代 → 规则总结

RAG+LLM 智能体：解决存储领域知识幻觉

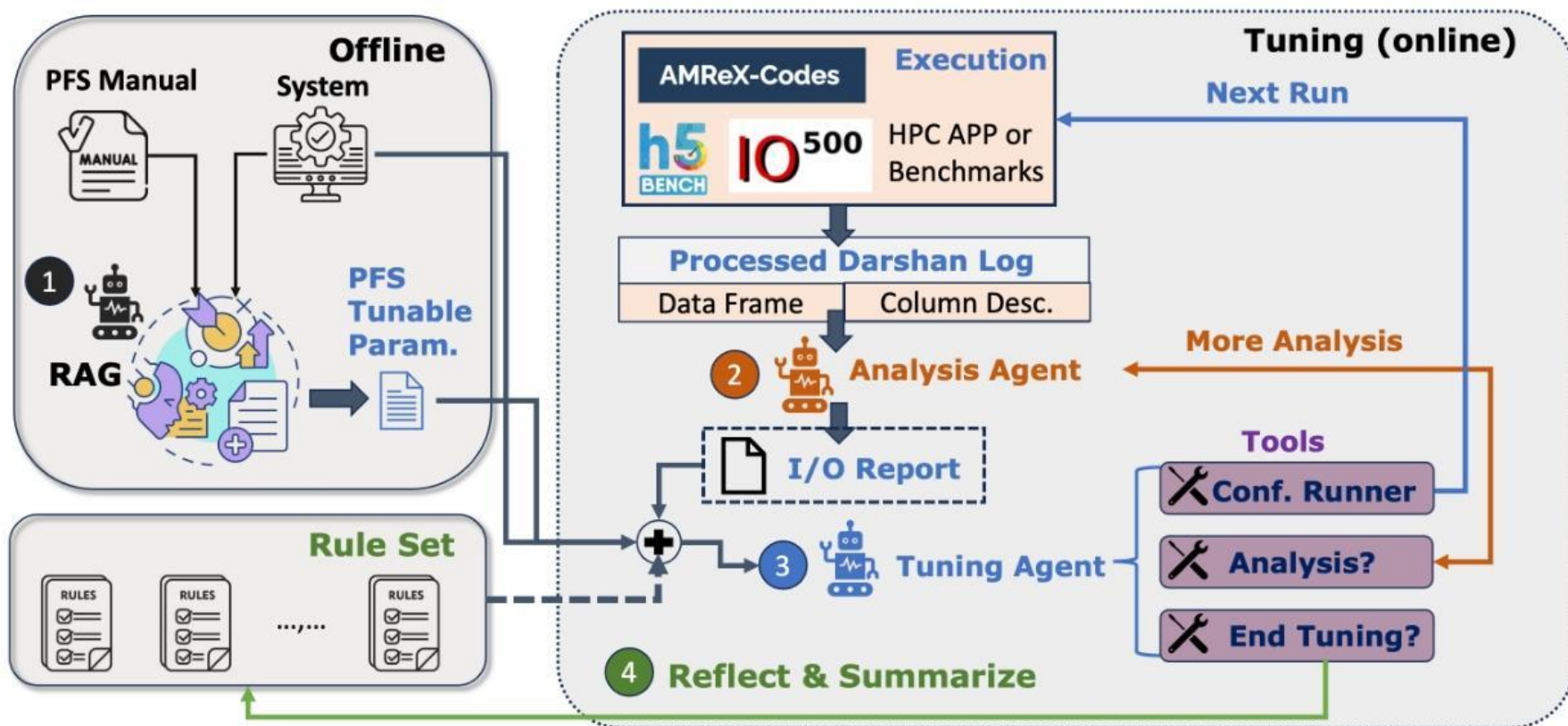
双智能体闭环：自动分析 I/O、自主调优、自主停止

规则集积累：提升泛化能力

STELLAR架构



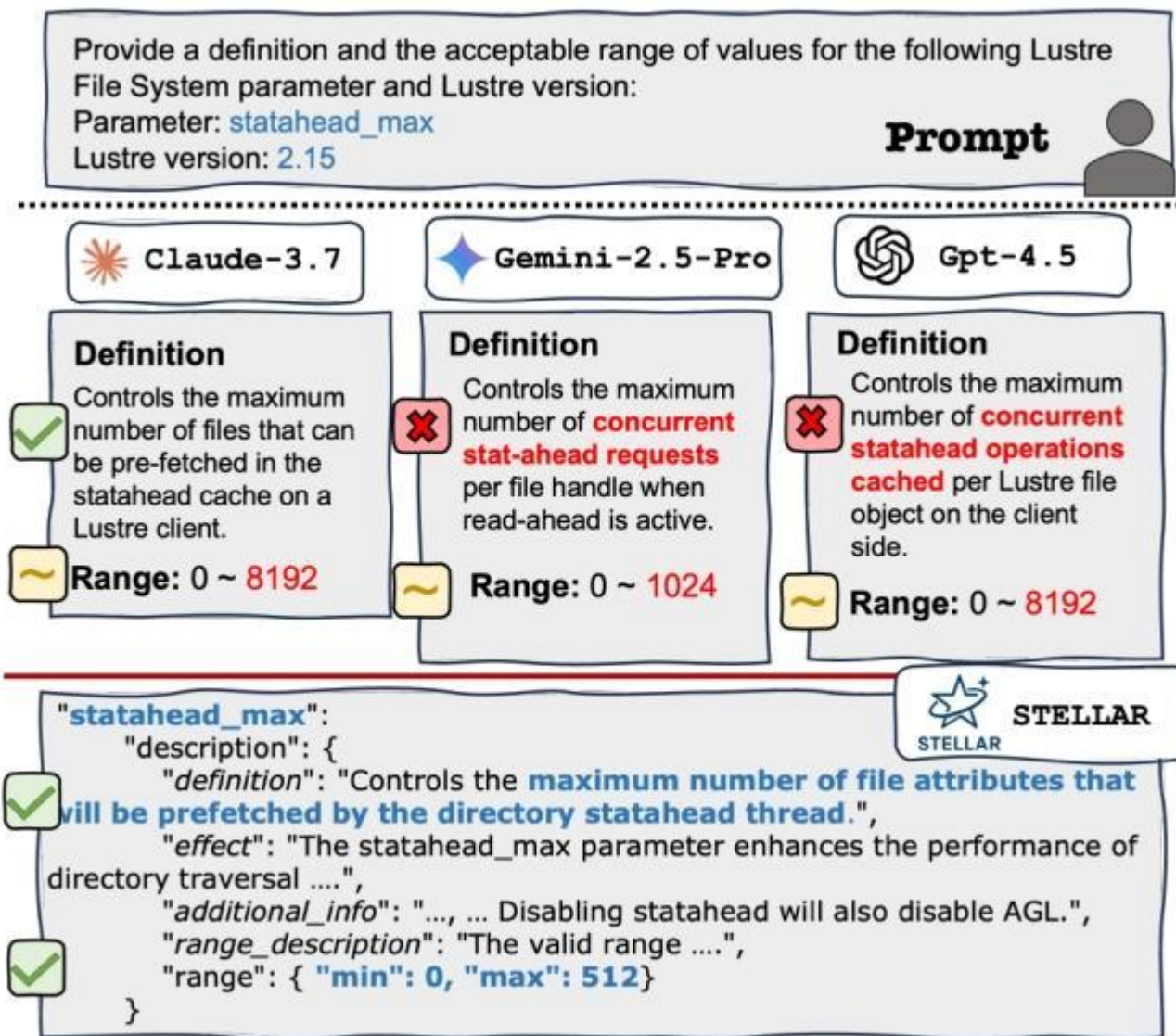
调优流程：手册 RAG 解析 → I/O 日志分析 → 调优迭代 → 规则总结





RAC

- 并行
- 手册
- 主流
- 超长 > 30



出错

RAG参数提取



步骤 1：构建文档向量索引，把超长手册变成可检索的知识库
将超过 600 页的 Lustre 手册，转化为一个可查询的知识库。
将手册按1024 token大小切分，块间重叠20 token，保证上下文连续性。

步骤2：

1. 获取系统 “可写参数” 列表：从系统路径（如 /proc/fs/lustre/）获取所有可动态调整的参数，过滤掉只读参数。
2. RAG 检索与解析：对每个参数，向量库查询其官方定义。
3. 自动筛选 “高价值” 参数：过滤标准：排除二进制参数（如校验和，影响安全）。排除低性能影响参数。排除文档缺失参数（无法验证）。

双智能体调优



模拟人类专家调优流程

人类专家调优分两步：

看懂应用 I/O 行为（分析员角色）

根据行为给出参数配置（调优师角色）

STELLAR 用两个专业化 LLM 智能体分工协作：

Analysis Agent（分析智能体）：看懂 I/O 日志

Tuning Agent（调优智能体）：决策、配置、迭代

双智能体调优-分析智能体



自动读懂 Darshan 日志，提取 I/O 模式生成清晰、可用于调优的I/O Report

接受 Tuning Agent 的追问，补充分析

分析内容：

- 文件大小分布（大文件 / 小文件）
- I/O 类型：顺序 / 随机
- 访问模式：共享文件 / 独立文件
- 读写比例、进程间 I/O 均衡性
- 典型 I/O 瓶颈（如小文件过多、预读不足）

双智能体调优-调优智能体



读取：参数手册（RAG 输出）+ I/O 报告 + 历史规则

推理：应该调整哪些参数、改成什么值

决策：下一步做什么（调用工具）

判断：是否继续调优 / 停止

反思：总结调优经验成规则

动态交互：Tuning Agent 拿到初始 I/O 报告发现信息不够→
向 Analysis Agent 提问→ 返回更细的 I/O 特征，Tuning
Agent 生成配置→ 继续优化或停止

调优结果



KernelEvolve - Meta



KernelEvolve 是 Meta 开发的一个智能体内核代码生成框架。它利用 AI 智能体代替人类工程师，自动为各种不同的 AI 芯片（如 NVIDIA GPU、AMD 以及 Meta 自研的 MTIA 芯片）编写和优化底层高性能算子代码。

人工算子的挑战：

- 模型多样性：从简单的推荐模型到复杂的 Transformer 结构，需求各异。
- 算子多样性：除了矩阵乘法，还有 200 多种数据预处理、归一化等各种复杂的小算子。
- 硬件异构性：Meta 的数据中心同时运行着多代 NVIDIA GPU、AMD GPU 和自研的 MTIA 芯片。

KernelEvolve - Meta

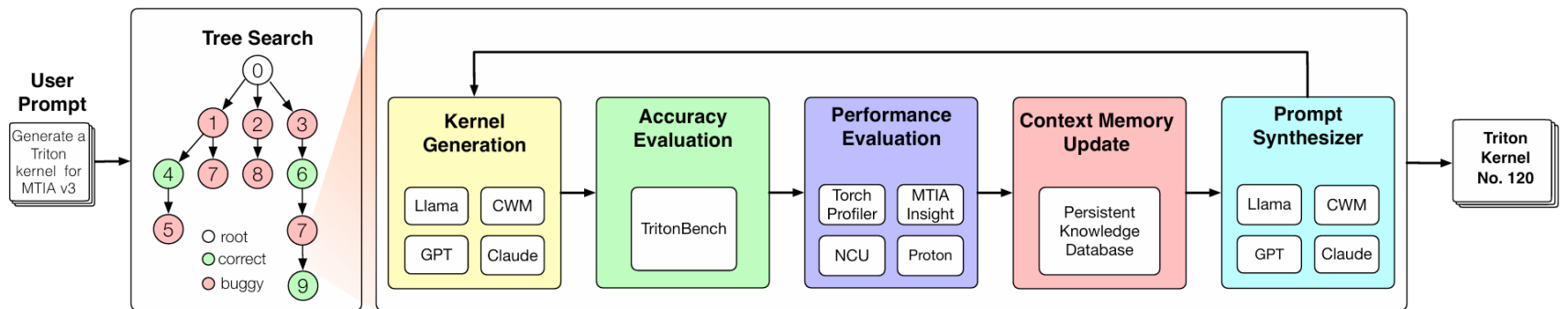
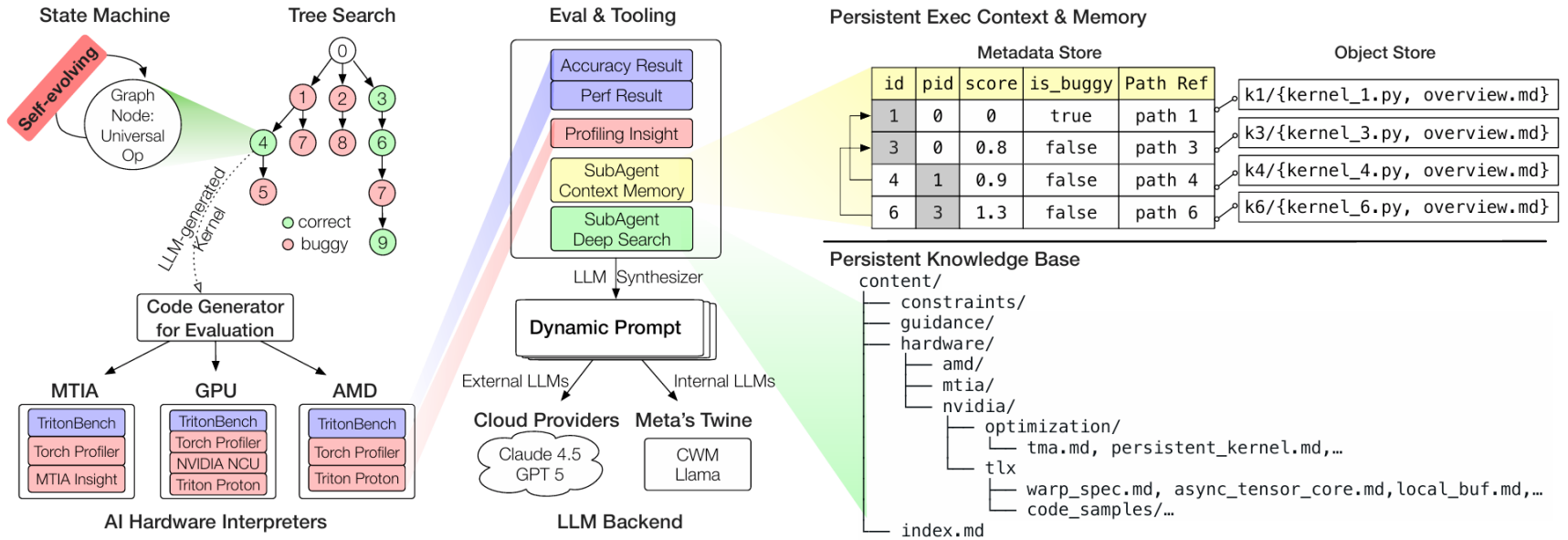


智能体架构：使用类似“思维树”的搜索方式，在海量的代码实现空间里寻找最优解。

多级抽象：它能跨越从高级的 Triton/CuTe DSL 到低级的硬件无关语言，自动适配不同的硬件指令集。

闭环进化：系统会不断尝试生成代码，通过“适应度函数”（性能测试）来筛选最快的代码。如果跑得不够快，它会根据反馈自动修改、继续演化。

KernelEvolve - Meta



KernelEvolve: Scaling Agentic Kernel Coding for Heterogeneous AI Accelerators at Meta

KernelEvolve - Meta



- ✓ 4 | read_file
/tmp/alpha_kernel_workspace_kzg1xux...
- ✓ 5 | LLM Thought
Perfect! The file is empty and ready for ...
- ✓ 6 | write_to_file
/tmp/alpha_kernel_workspace_kzg1xux...
- ✓ 7 | LLM Thought
Let me use the edit tool instead:
- ✓ 8 | replace_in_file
/tmp/alpha_kernel_workspace_kzg1xux...
- ✓ 9 | LLM Thought
Perfect! The implementation has been s...
- ✓ 10 | lint
/tmp/alpha_kernel_workspace_kzg1xux...
- ✓ 11 | LLM Thought
Good! The linter found some minor styl...
- ✓ 12 | replace_in_file
/tmp/alpha_kernel_workspace_kzg1xux...
- ✓ 13 | LLM Thought
Now let me fix the second occurrence i...
- ✓ 14 | replace_in_file
/tmp/alpha_kernel_workspace_kzg1xux...
- ✓ 15 | LLM Thought
Perfect! Now let me run the linter again ...
- ✓ 16 | lint
/tmp/alpha_kernel_workspace_kzg1xux...
- ✓ 17 | LLM Thought
Excellent! The code is now lint-free. Let...
- ✓ 18 | read_file

✓ Step 17 | LLM Thought

The LLM thought

Excellent! The code is now lint-free. Let me verify the final implementation by reading the file one more time to ensure everything is correct:

🗨️ Prompts and Responses

✓ Step 18 | read_file

Tool call **read_file** with arguments

```
{
  "file_path": "/tmp/alpha_kernel_workspace_kzg1xuxa/new.py"
}
```

resulted in the following observation

```
49|# =====
50|# Triton Kernel Implementation
51|# =====
52|
53|
54|@triton.autotune(
55|    configs=[
56|        triton.Config({"BLOCK_SIZE": 256}, num_warps=4, num_stages=1),
57|        triton.Config({"BLOCK_SIZE": 512}, num_warps=4, num_stages=1),
58|    ],
59|    key=["BLOCK_SIZE"],
60|)
```

✓ Step 19 | LLM Thought

✓ Step 20 | Agent Finished

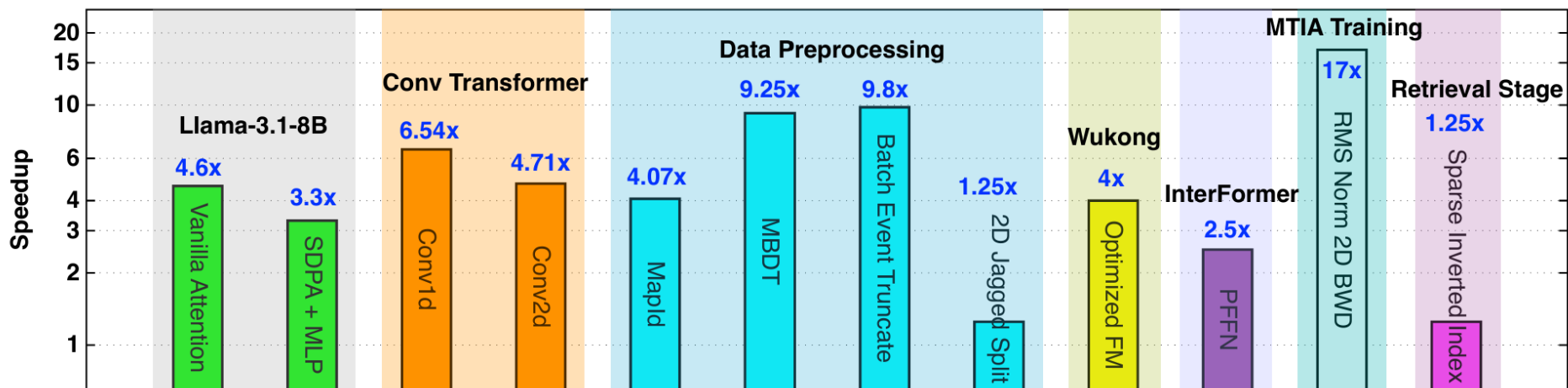
Successfully created a complete, optimized Triton kernel implementation for the Swish activation function in /tmp/alpha_kernel_workspace_kzg1xuxa/new.py.

Summary of Actions Taken:

KernelEvolve - Meta



KernelEvolve带来的加速效果

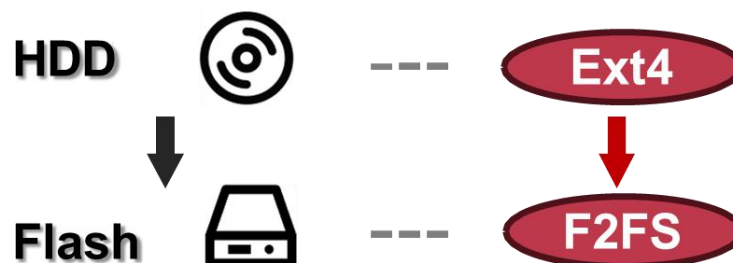


文件系统的发展



- 文件系统不断演进
- 耗费大量人力维护

为硬件优化



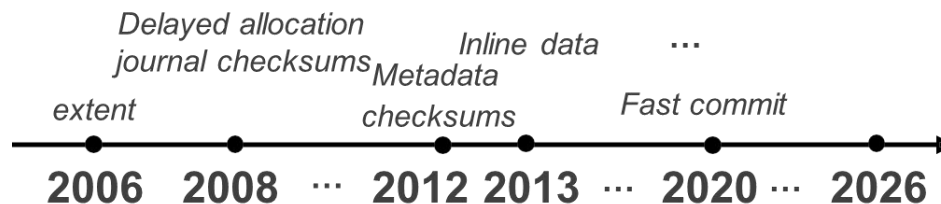
为应用优化

Fire-Flyer File System

Build passing LICENSE MIT

The Fire-Flyer File System (3FS) is a high-performance distributed file system designed to address the challenges of AI training and inference workloads. It leverages modern SSDs

不断有新技术加入文件系统



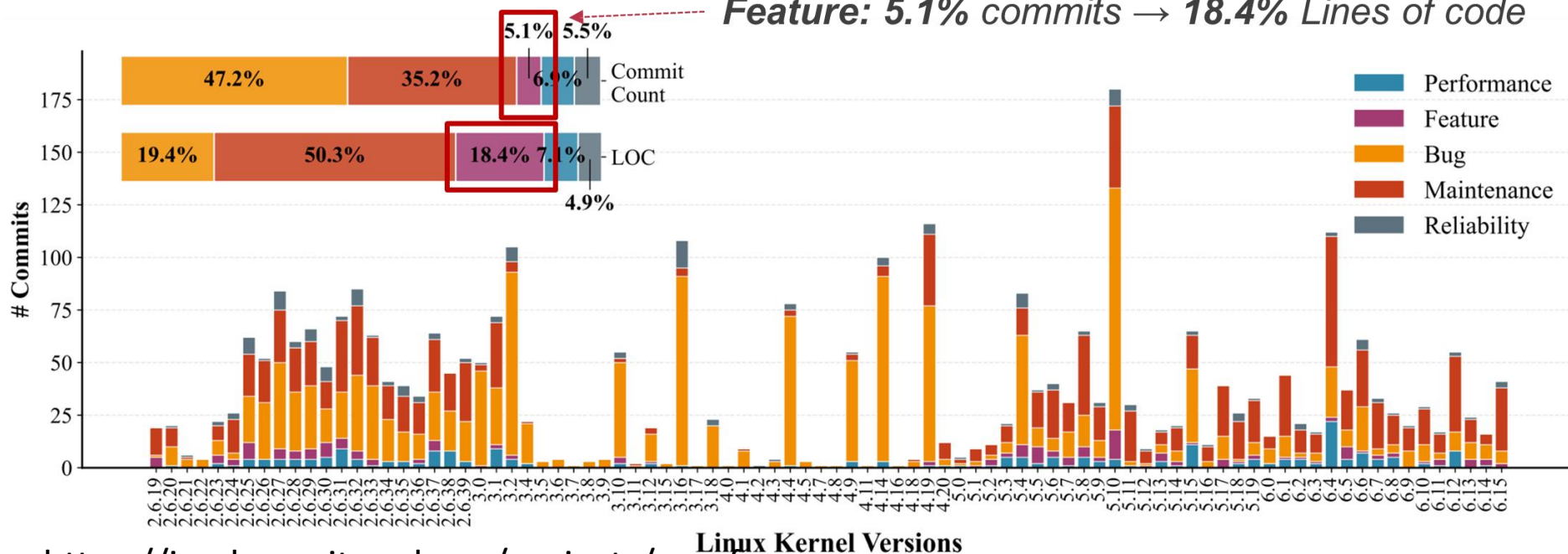
文件系统的发展



- 文件系统不断演进
- 耗费大量人力维护

From Linux 2.6.19 to 6.15: ~**3,157** ext4-related commits

Feature: 5.1% commits → 18.4% Lines of code



<https://ipads.se.sjtu.edu.cn/projects/specfs>

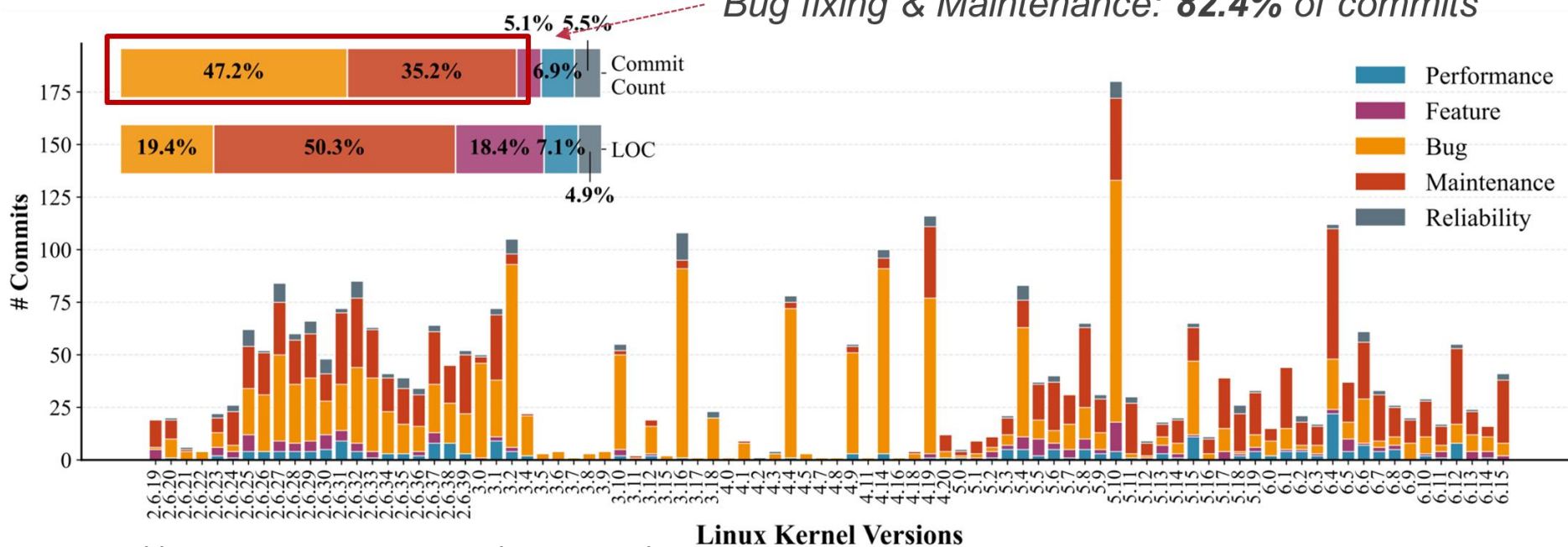
文件系统的发展



- 文件系统不断演进
- 耗费大量人力维护

From Linux 2.6.19 to 6.15: ~**3,157** ext4-related commits

Bug fixing & Maintenance: **82.4%** of commits



<https://ipads.se.sjtu.edu.cn/projects/specfs>

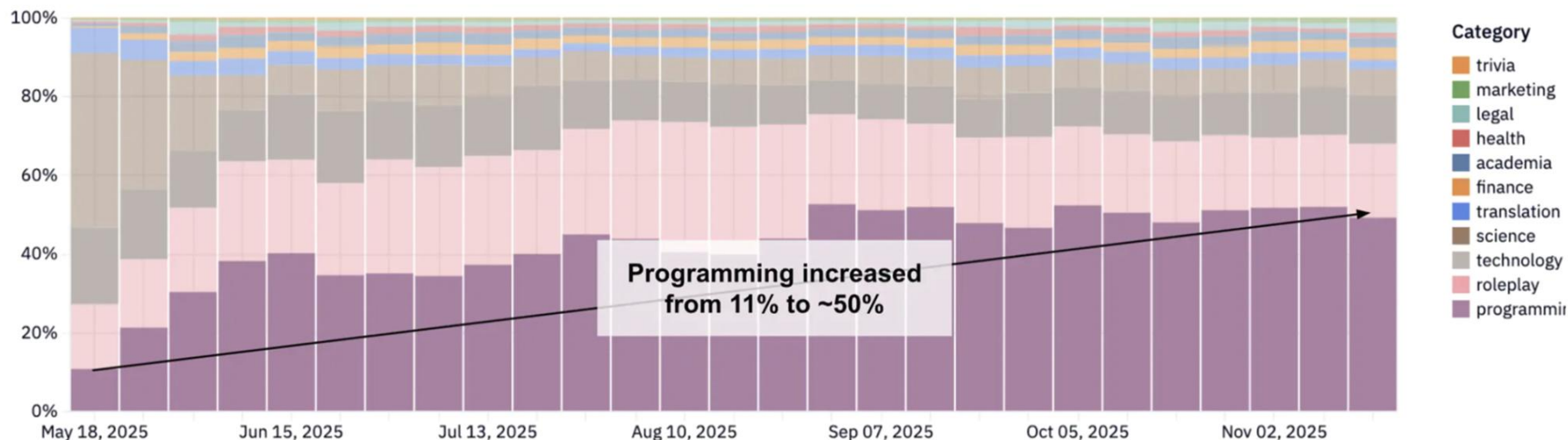
LLM coding的迅速发展



使用大模型减少文件系统的开发和维护成本？

Dominant Categories Over Time

OpenRoute

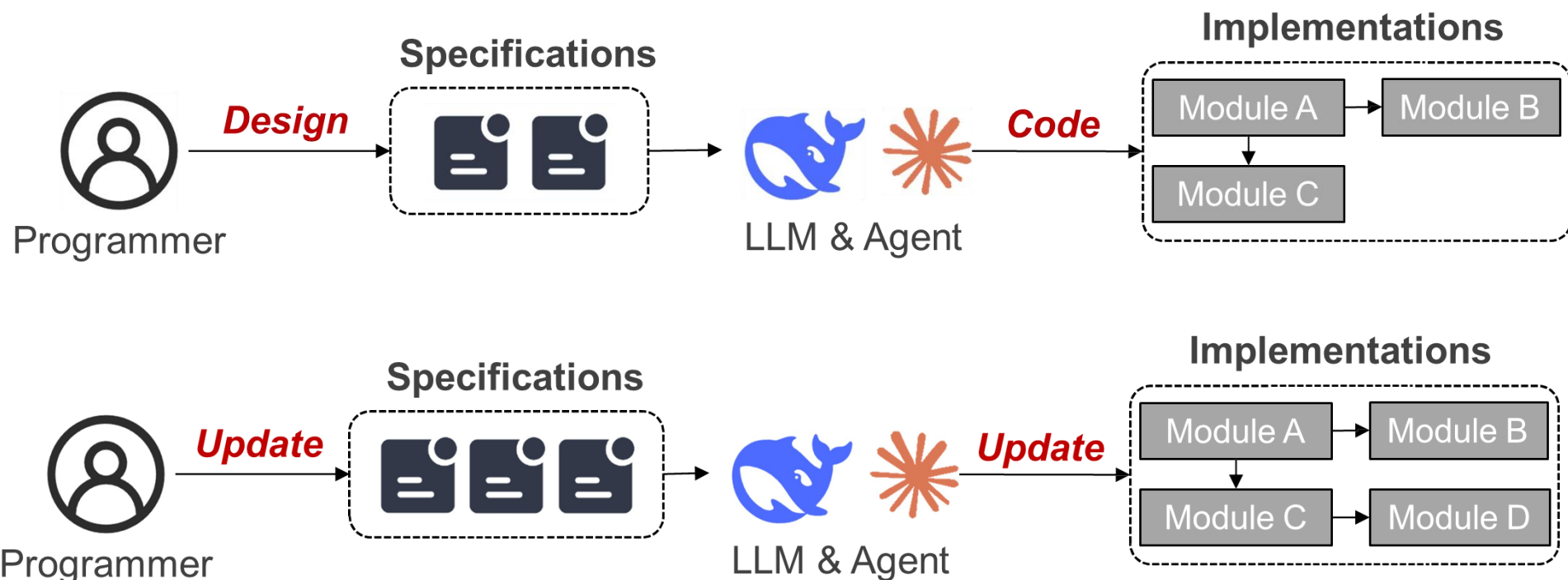


<https://ipads.se.sjtu.edu.cn/projects/specfs>

生成文件系统



使用LLM生成文件系统，并使用LLM不断维护



生成文件系统



- 挑战I：语义鸿沟 - 如何系统化指定功能？
使用自然语言往往有歧义
无法让大模型准确实现指定功能

- “No dependency error”
- “Avoid race conditions with locking”
-



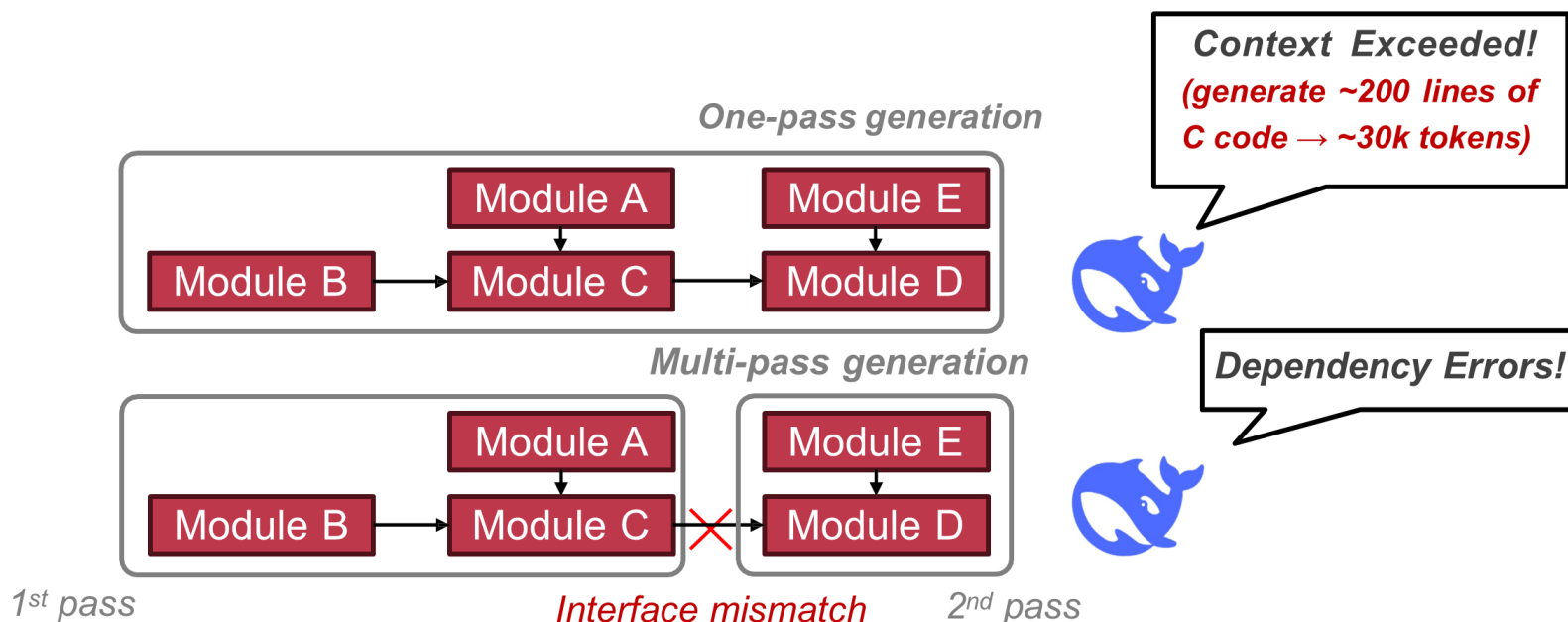
???

<https://ipads.se.sjtu.edu.cn/projects/specfs>

生成文件系统



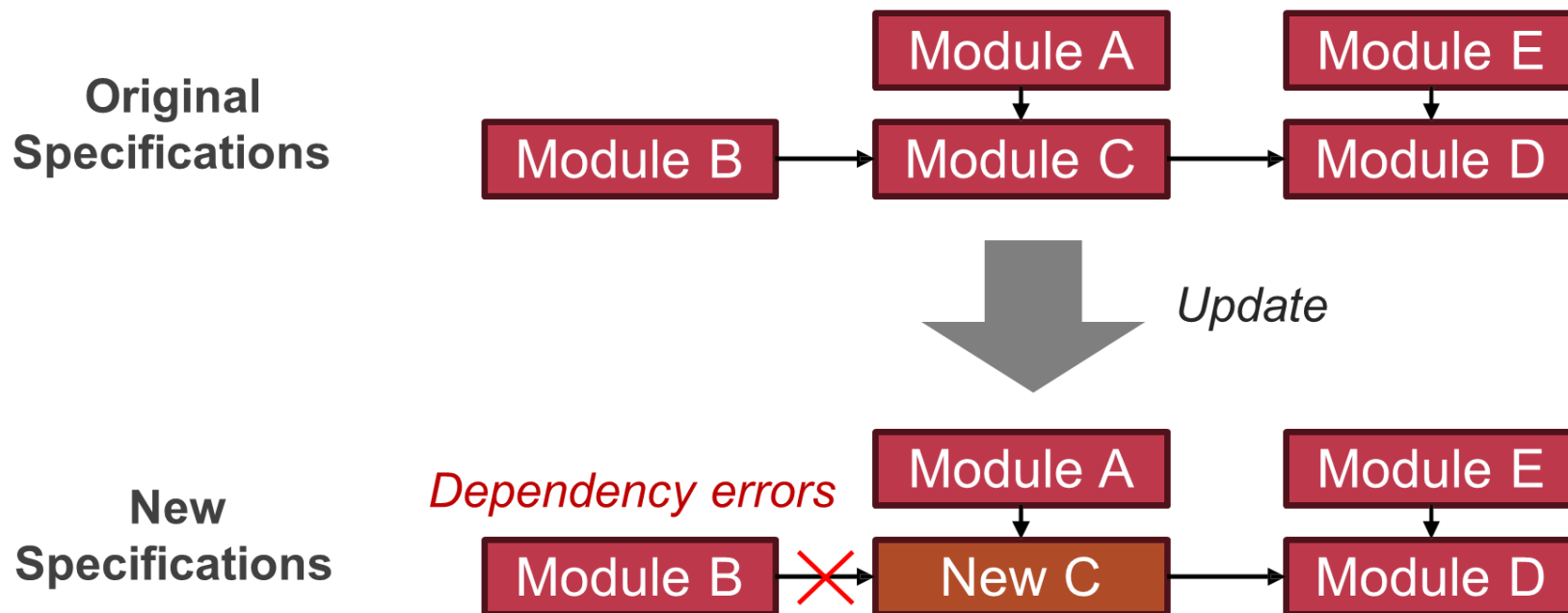
- 挑战II：组件组合 - 上下文超限、接口不匹配
- 文件系统代码量大，组件众多
- 一次性生成超过LLM上下文长度
- 组合生成，接口难以匹配



生成文件系统



- 挑战III：向后兼容性 - 规范更新如何避免级联问题
大模型替换新的组件，如何避免级联错误

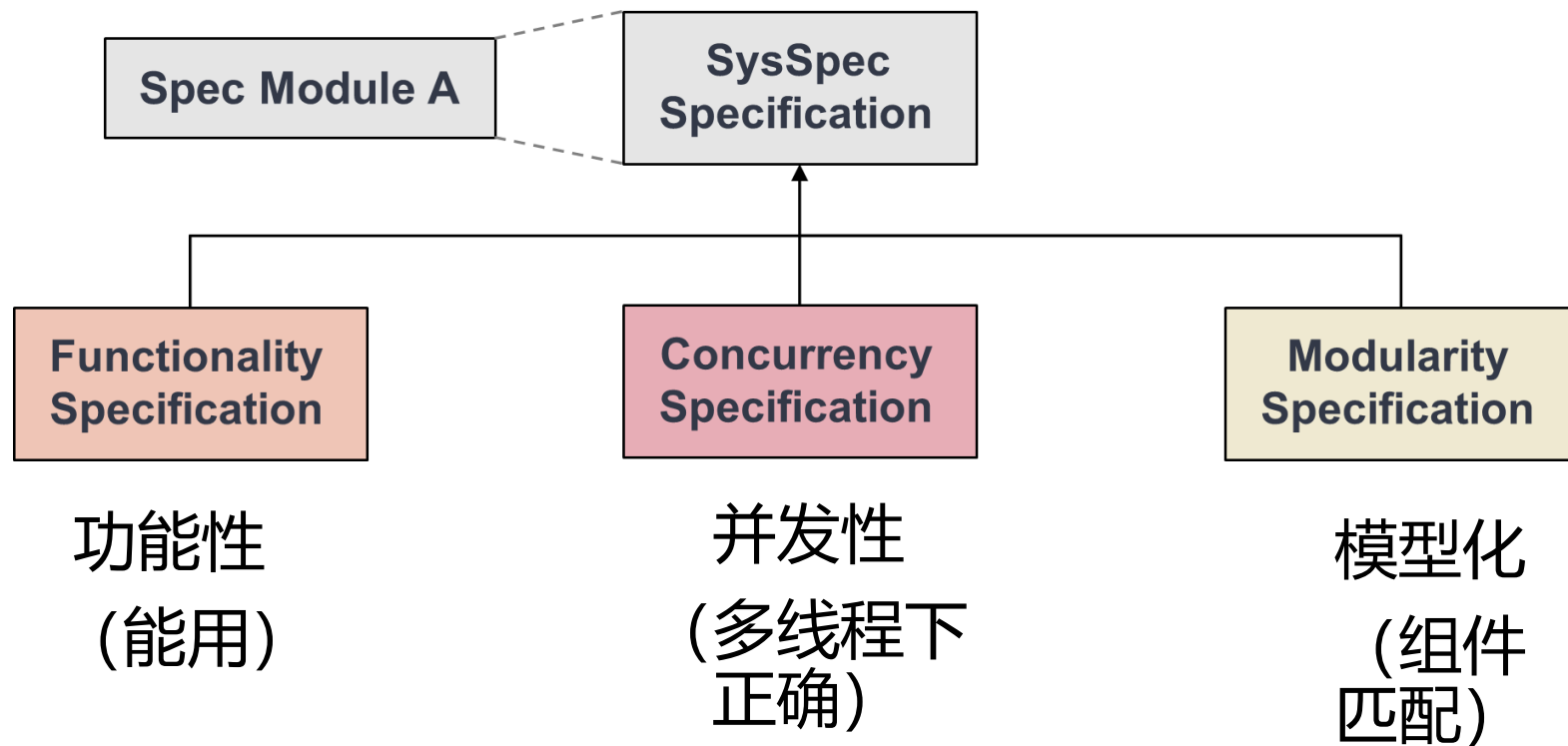


<https://ipads.se.sjtu.edu.cn/projects/specfs>

生成文件系统



SysSpec: 利用形式化验证技术, 通过编写 Specification(spec)指导文件系统的生成



<https://ipads.se.sjtu.edu.cn/projects/specfs>

生成文件系统



Functionality Specification

[Pre-condition]:
Required state before execution

[Post-condition]:
Guaranteed state upon completion

[Invariants]:
properties that must hold true
across state transitions

Hoare Style Specification

[Pre-condition]
`path`: a NULL-terminated string array `name`: a valid string
[Post-condition]
Case 1 Successful traversal and insertion
-New `inode` created
-Entry inserted into target directory
-Return `0`
Case 2 Traversal or insertion failure Return `-1`
[Invariants]
The root `inum` always exists

Functionality Specification of `atomfs_ins`

<https://ipads.se.sjtu.edu.cn/projects/specfs>

Concurrency Specification

The state of locking of
each function call

[Locking Specification of locate] **Pre-condition:**
cur is locked.

Post-condition: suppose return target.

- **If target is NULL:** no lock owned
 - **If target is not NULL:** only target is owned
- [Locking Specifications of check_ins]

.....

[Locking Specification of atomfs_ins]

.....

Concurrency Specification of atomfs_ins

<https://ipads.se.sjtu.edu.cn/projects/specfs>

生成文件系统



SpecFS: 从零开始生成的并发文件系统
借鉴 AtomFS [SOSP' 19] 的模型
几乎实现了全部功能的生成并保证正确性

LLM	AtomFS	Features
Gemini-2.5/3.0-pro	100% (45/45)	100% (70/70)
Deepseek-V3.1	100% (45/45)	100% (70/70)
GPT-5-minimal	100% (45/45)	100% (70/70)
Qwen3.5-397B-A17B	100% (45/45)	100% (70/70)
Qwen3-32B	82.2% (37/45)	92.9% (65/70)

<https://ipads.se.sjtu.edu.cn/projects/specfs>

谢谢！